



Data analysis and visualisation

Piotr Jastrzębski

2026-05-19

Table of contents

1	Data Analysis and Visualization	7
I	Introduction	8
2	A bit of theory...	9
2.1	Rational thinking test	9
2.2	Data analysis	9
2.3	Data visualization	9
2.4	Data analysis - basic concepts	10
2.4.1	Modern meanings of the word “statistics”:	10
2.4.2	“Massiveness”	10
2.4.3	Division of statistics	11
2.4.4	Population	11
2.4.5	Statistical unit	11
2.4.6	Statistical features	11
2.4.7	Scales	13
2.5	Types of statistical studies	14
2.6	Stages of a statistical study	15
2.7	Analysis of existing data	15
2.8	Data analysis process	16
2.8.1	Defining requirements	16
2.8.2	Data collection	16
2.8.3	Data processing	16
2.8.4	Proper data analysis	17
2.8.5	Reporting and distribution of results	17
2.9	Where to get data?	17
2.10	“Tidy data” concept	17
2.10.1	“Tidy data” principles	18
2.10.2	Examples of untidy data	18
2.11	A few tips for good presentations	18
2.11.1	Lie factor	19
2.11.2	Lie factor	19
2.12	References	20

3	Configuration	21
II	NumPy	23
4	NumPy - Getting Started	24
4.1	Importing the NumPy library	24
5	List vs Array	26
6	Array Attributes	27
7	Data Types	29
8	Creating Arrays	30
9	Indexing, “slicing”	37
10	Modifying shape and size	43
11	Broadcasting	49
12	Universal functions (ufunc) and other mathematical functions	52
12.1	Basic arithmetic operations	52
12.2	Trigonometric and related functions	53
12.3	Exponential and logarithmic functions	53
12.4	Rounding functions and absolute values	54
12.5	Statistical and aggregate functions	55
13	String operations	56
13.1	Creating arrays with strings	56
13.2	Basic text modification functions	56
13.2.1	<code>numpy.strings.upper</code> and <code>numpy.strings.lower</code>	56
13.2.2	<code>numpy.strings.capitalize</code>	57
13.2.3	<code>numpy.strings.title</code>	57
13.3	Joining and splitting text	58
13.3.1	<code>numpy.strings.add</code>	58
13.3.2	<code>numpy.char.join</code>	58
13.3.3	<code>numpy.char.split</code>	58
13.4	Searching and replacing substrings	59
13.4.1	<code>numpy.strings.find</code> and <code>numpy.strings.rfind</code>	59
13.4.2	<code>numpy.strings.replace</code>	59
13.5	Removing unwanted characters	60
13.5.1	<code>numpy.strings.strip</code> , <code>numpy.strings.lstrip</code> and <code>numpy.strings.rstrip</code>	60

14 Linear Algebra	61
14.1 Dot Product	61
14.2 Matrix Multiplication	61
14.3 Matrix-Vector Multiplication	62
14.4 Solving Systems of Linear Equations	62
14.5 Matrix Determinant	63
14.6 Eigenvalues and Eigenvectors	63
14.7 Singular Value Decomposition (SVD)	64
14.8 Matrix/Vector Norm	64
14.9 Inverse Matrix	65
14.10The <code>numpy.inner</code> Function - Inner Product	66
14.11The <code>numpy.outer</code> Function - Outer Product	66
14.12The <code>numpy.matmul</code> Function - Matrix Multiplication	67
15 Advanced Filtering	68
15.1 The <code>nonzero()</code> Function	68
15.2 The <code>where()</code> Function	69
15.3 The <code>indices()</code> and <code>ix_()</code> Functions	69
15.3.1 <code>indices()</code>	69
15.3.2 <code>ix_()</code>	70
15.4 <code>ogrid</code> and Grid Operations	71
15.5 The <code>ravel_multi_index()</code> and <code>unravel_index()</code> Functions	72
15.6 Diagonal Indices	72
15.7 The <code>take()</code> Function	73
15.8 The <code>take_along_axis()</code> Function	73
15.9 The <code>choose()</code> Function	74
15.10The <code>compress()</code> Function	74
15.11The <code>diag()</code> and <code>diagonal()</code> Functions	75
15.12The <code>select()</code> Function	75
15.13The <code>place()</code> Function	76
15.14The <code>put()</code> Function	76
15.15The <code>put_along_axis()</code> Function	77
15.16The <code>putmask()</code> Function	77
15.17The <code>fill_diagonal()</code> Function	77
16 NumPy - Miscellaneous	79
16.1 Constants	79
16.2 <code>numpy.inf</code>	80
16.3 <code>numpy.nan</code>	81
16.4 <code>numpy.newaxis</code>	81
16.5 Statistics and Aggregation	82

III Data exploration	84
17 Stages of data exploration	85
IV Pandas	86
18 Pandas - getting started	87
18.1 Import:	87
18.2 Basic objects	87
19 Pandas - indexing	92
20 Loading data	95
20.1 Handling csv files	95
20.2 Handling Excel files	95
20.3 Handling <code>sqlite3</code>	96
20.4 Sample files for exercises	96
21 Pandas - sorting	97
22 Pandas - time series	100
23 Pandas - regular expressions	104
23.1 What are regular expressions?	104
23.2 Basic regex syntax	104
23.3 Key functions of the <code>re</code> module	106
23.4 Groups and references	107
23.5 Flags	107
23.6 Regular expressions in Pandas	108
23.7 <code>str.contains()</code> — filtering with regex	109
23.8 <code>str.extract()</code> — extracting groups	110
23.9 <code>str.extractall()</code> — all matches	110
23.10 <code>str.replace()</code> with regex — replacing patterns	111
23.11 <code>str.findall()</code> — list of matches	111
23.12 Polish characters in regex	112
24 Pandas - other	114
24.1 Filling missing values	114
24.2 Handling missing data	115
24.3 Removing duplicates	116
24.4 Replacing values	117
24.5 Discretization and binning	118
24.6 Detecting and filtering outliers	120

24.7	Changing the column type	121
24.8	Changing characters in categories	123
24.9	Manipulation operations	124
V	Structure Analysis	146
25	Structure Analysis	147
25.1	Measures of location	147
25.1.1	Arithmetic mean	148
25.1.2	Mode	148
25.1.3	Median	149
25.1.4	Quantiles	150
25.2	Measures of variability	151
25.2.1	Range	151
25.2.2	Interquartile range	152
25.2.3	Quartile deviation	152
25.2.4	Variance	152
25.2.5	Standard deviation	152
25.3	Measures of asymmetry	153
25.4	Measures of concentration	154
25.5	Speeding up operations:	154
VI	Matplotlib	157
26	Matplotlib - start	158
27	Matplotlib - line chart	161
28	Matplotlib - extras part 1	168
28.1	Legend parameters	168
28.2	Styles, line colors	170
29	Matplotlib - subplots	177

1 Data Analysis and Visualization

The current version pertains to classes held in the 2025/26 academic year.

Some of the texts have been machine-translated into English. In case of any ambiguities, please get in touch via email.

Part I

Introduction

2 A bit of theory...

2.1 Rational thinking test

- If 5 machines produce 5 devices in 5 minutes, how long will it take 100 machines to make 100 devices?
- A patch of water lilies is growing on a pond. Every day the patch doubles in size. If it takes 48 days for the lilies to cover the entire pond, how many days does it take for them to cover half the pond?
- A baseball bat and a ball cost \$1.10 together. The bat costs one dollar more than the ball. How much does the ball cost?

2.2 Data analysis

Data analysis is the process of examining, cleaning, transforming, and modeling data in order to discover useful information, draw conclusions, and support decision-making. It is a multi-stage process that includes:

- Collecting data from various sources
- Cleaning data by removing errors, missing values, and inconsistencies
- Exploring data to understand its structure and characteristics
- Transforming data into the appropriate format
- Applying statistical methods and machine learning algorithms
- Interpreting results in the context of a specific business or scientific problem

Data analysis is used in nearly every field, from business and finance to social sciences, medicine, and scientific research. The goal of data analysis is to transform raw data into knowledge that can be used to make better decisions.

2.3 Data visualization

Data visualization is the graphical representation of information and data. It uses visual elements such as charts, maps, and dashboards to present relationships between data in a way that is easy to understand and interpret. Good data visualization:

- Presents complex information in an accessible and intuitive way
- Reveals patterns, trends, and outliers that may be difficult to notice in raw data
- Supports the data analysis process by enabling quick review of large datasets
- Facilitates communication of analysis results to various audiences, including non-technical individuals
- Helps tell stories contained in data (data storytelling)

The most popular types of data visualization include bar charts, line charts, pie charts, heatmaps, hierarchical trees, word clouds, and interactive dashboards. The choice of the appropriate visualization form depends on the type of data, the purpose of the presentation, and the target audience.

Data visualization is a key element of the data analysis process because it allows for quick drawing of conclusions and making decisions based on data. It is a bridge between complex data and human understanding.

2.4 Data analysis - basic concepts

2.4.1 Modern meanings of the word “statistics”:

- a set of numerical data showing the development of processes and phenomena, e.g. population statistics.
- all activities related to collecting and processing numerical data, e.g. statistics on a certain problem conducted by the Central Statistical Office.
- numerical characteristics, e.g. sample statistics such as arithmetic mean, standard deviation, etc.
- a scientific discipline - the science of methods for studying mass phenomena.

2.4.2 “Massiveness”

Mass phenomena/processes - a large number of units are subject to study. They are divided into:

- economic (e.g. production, consumption, services, advertising),
- social (e.g. traffic accidents, political views),
- demographic (e.g. births, aging, migrations).

2.4.3 Division of statistics

Statistics - a scientific discipline - division:

- descriptive statistics - deals with matters related to collecting, presenting, analyzing, and interpreting numerical data. Observation covers the entire population under study.
- mathematical statistics - generalization of the results of studying a part of the population (sample) to the entire population.

2.4.4 Population

Statistical population: a set of objects subject to statistical study. It consists of units that are similar to each other, logically related, but not identical. They share certain common features and certain properties that allow them to be differentiated.

- examples:
 - study of the height of Polish citizens - inhabitants of Poland
 - level of education in schools of the Warmian-Masurian Voivodeship - schools of the Warmian-Masurian Voivodeship.
- division:
 - general population - covers the entirety,
 - sample population (sample) - covers a part of the population.

2.4.5 Statistical unit

Statistical unit: each element of a statistical population.

- examples:
 - UWM students - a UWM student
 - inhabitants of Poland - every person living in Poland
 - machines produced in a factory - every machine

2.4.6 Statistical features

Statistical features

- properties characterizing statistical units in a given statistical population.
- they are divided into constant and variable.

Constant features

- properties that are common to all units of a given statistical population.
- division:
 - substantive - who or what is the subject of the statistical study,
 - temporal - when the study was conducted or what time period the study covers,
 - spatial - what territory (place or area) the study concerns.
- example: students of the Faculty of Mathematics and Computer Science (WMiI) at UWM in Olsztyn in the academic year 2017/2018:
 - substantive feature: possession of a student ID card,
 - temporal feature - students studying in the academic year 2017/2018,
 - spatial feature - location: WMiI UWM in Olsztyn.

Variable features

- properties that differentiate statistical units in a given population.
- example: UWM students - variable features: age, sex, type of secondary school completed, eye color, height.

Important:

- only variable features are subject to observation,
- a constant feature in one population may be a variable feature in another population.

Example: UWM students have an ID card issued by UWM. Students of all universities in Poland have ID cards issued by different institutions.

Division of variable features:

- measurable (quantitative) features - can be expressed as a number with a specific unit of measurement.
- non-measurable (qualitative) features - described verbally, representing certain categories.

Example: student population. Measurable features: age, weight, height, number of absences. Non-measurable features: sex, eye color, field of study.

For practical reasons, numerical codes are often assigned to non-measurable features. However, they should not be confused with measurable features. E.g. 1 - primary education, 2 - vocational education, etc.

Division of measurable features:

- continuous - can take any value within a defined interval, e.g. height, age, apartment area.
- discrete - can take specific (discrete) numerical values without intermediate values, e.g. number of people in a household, number of employees in a given company.

Discrete features usually have integer values, although this is not always required, e.g. the number of full-time positions in a company (including part-time positions).

2.4.7 Scales

Measurement scale

- a system that allows the results of statistical measurements to be organized in a certain way.
- division:
 - nominal scale,
 - ordinal scale,
 - interval scale,
 - ratio scale.

Nominal scale

- a scale in which a statistical unit is classified into a specific category.
- values in this scale have no ordering.
- example:

Religion	Code
Christianity	1
Islam	2
Buddhism	3

Ordinal scale

- values have a clearly defined order, but distances between them are not given,
- allows elements to be ranked.
- examples:

Education	Code
Primary	1
Secondary	2
Higher	3

Income	Code
Low	1
Medium	2
High	3

Interval scale

- feature values are expressed through specific numerical values,
- allows comparison of units (something is larger or smaller),
- it is not possible to examine ratios (determining how many times a given value is larger or smaller than another).
- example:

City	Temperature in $^{\circ}C$	Temperature in $^{\circ}F$
Warsaw	15	59
Olsztyn	10	50
Gdańsk	5	41
Szczecin	20	68

Ratio scale

- values are expressed through numerical values,
- it is possible to determine the smaller or larger relationship between values,
- it is possible to determine the ratio (quotient) between values,
- an absolute zero exists.
- example:

Product	Price in PLN
Bread	3
Butter	8
Pears	5

2.5 Types of statistical studies

- complete study - covers all units of the statistical population.
 - statistical census,
 - ongoing registration,

- statistical reporting.
- partial studies - only a part of the population is observed. Conducted when a complete study is impractical or impossible.
 - monographic method,
 - representative method.

2.6 Stages of a statistical study

- designing and organizing the study: establishing the purpose, subject, object, scope, source, and duration of the study;
- statistical observation;
- processing statistical material: quality control of statistical material, grouping of obtained data, presentation of data results;
- statistical analysis.

2.7 Analysis of existing data

Analysis of existing data - the process of processing data in order to obtain useful information and conclusions from them. Depending on the type of data and the problems posed, this may involve the use of statistical, exploratory, and other methods.

Using existing data is an example of non-reactive research - methods of studying social behavior that do not influence those behaviors. Such data include: documents, archives, reports, chronicles, censuses, parish records, diaries, memoirs, internet blogs, audio memoirs, oral history archives, and others. (Wikipedia)

Existing data can be classified according to (Makowska ed. 2013):

- Nature: Quantitative, Qualitative
- Form: Processed data, Raw data
- Method of creation: Primary, Secondary
- Dynamics: Continuous event registration, Registration at time intervals, One-time registration
- Level of objectivity: Objective, Subjective
- Source of origin: Public data, Private data

Data analysis is a process of inspecting, organizing, transforming, and modeling data in order to obtain useful information, draw conclusions, and support the decision-making process. Data analysis has many aspects and approaches, encompassing various techniques under different names, in different business, scientific, and social domains. A practical approach to defining

data is that data are numbers, characters, images, or other recording methods, in a form that can be evaluated to determine or make a decision about a specific action. Many people believe that data in itself has no meaning - only processed and interpreted data becomes information.

2.8 Data analysis process

Analysis refers to breaking down the entirety of information into its distinct components for individual examination. Data analysis is the process of obtaining raw data and transforming it into information useful for decision-making by users. Data is collected and analyzed to answer questions, test hypotheses, or disprove theories. There are several phases that can be distinguished in the data analysis process. The phases are iterative, as feedback from subsequent phases may cause additional work in earlier phases.

2.8.1 Defining requirements

Before proceeding with data analysis, the quality requirements for data must be precisely defined. The input data to be analyzed are determined based on the requirements of those directing the analysis or clients (who will use the final product of the analysis). The general type of entity from which data will be collected is referred to as the experimental unit (e.g. a person or a population of people). Data can be numerical or categorical (i.e. text labels). The requirements definition phase should answer 2 fundamental questions:

- what do we want to measure?
- how do we want to measure it?

2.8.2 Data collection

Data is collected from various sources. Requirements regarding the type and quality of data may be communicated by analysts to “data custodians,” such as information technology staff within an organization. Data may also be collected automatically from various types of sensors in the environment - such as traffic cameras, satellites, and devices recording images, sound, and physical parameters. Another method is obtaining data through interviews, gathering from online sources, or directly from documentation.

2.8.3 Data processing

Collected data must be processed or organized in a logical manner for analysis. For example, they may be placed in tables for further analysis - in a spreadsheet or other software. Data cleaning: After the processing and organizing phase, data may be incomplete, contain duplicates,

or contain errors. The need for data cleaning arises from problems related to data entry and storage. Data cleaning is the process of preventing and correcting detected errors. Common tasks include record matching, identifying inaccuracies, overall review of existing data quality, removing duplicates, and column segmentation. It is also extremely important to pay attention to data whose values are above or below previously established thresholds (extremes).

2.8.4 Proper data analysis

There are several methods that can be used for this purpose, for example data mining, business intelligence, data visualization, or exploratory research. The latter method is a way of analyzing datasets to determine their distinct characteristics. In this way, data can be used to test the original hypothesis. Descriptive statistics is another method for analyzing collected information. Data is examined to find its most important features. In descriptive statistics, analysts use several basic tools - the mean or average of a set of numbers can be used. This helps determine the overall trend, although it does not provide great accuracy when assessing the overall picture of collected data. In this phase, modeling and the creation of mathematical formulas also take place - they are applied to identify relationships between variables, such as correlation or causation.

2.8.5 Reporting and distribution of results

This phase involves determining in what form to communicate results. The analyst may consider various data visualization techniques to clearly and effectively convey the findings of the analysis to the audience. Data visualization uses graphical forms such as charts and tables. Tables are useful for users who can search for specific records, while charts (e.g. bar charts or line charts) provide a quantitative view of the analyzed dataset.

2.9 Where to get data?

Free data repositories:

- Local Data Bank of the Central Statistical Office (GUS) - [link](#)
- Open Data - [link](#)
- World Bank - [link](#)

2.10 “Tidy data” concept

The tidy data concept:

- WICKHAM, Hadley . Tidy Data. Journal of Statistical Software, [S.l.], v. 59, Issue 10, p. 1 - 23, sep. 2014. ISSN 1548-7660. Date accessed: 25 oct. 2018. doi:<http://dx.doi.org/10.18637/jss.v059.i10>.

2.10.1 “Tidy data” principles

Ideal data is presented in a table:

Name	Age	Height	Eye color
Adam	26	167	Brown
Sylwia	34	164	Hazel
Tomasz	42	183	Blue

What should we pay attention to?

- one observation (statistical unit) = one row in the table/matrix/data frame
- values of a given feature are found in columns
- one type/kind of observation in one table/matrix/data frame

2.10.2 Examples of untidy data

Name	Age	Height	Brown	Blue	Hazel
Adam	26	167	1	0	0
Sylwia	34	164	0	0	1
Tomasz	42	183	0	1	0

Column headers must correspond to features, not to variable values.

2.11 A few tips for good presentations

Edward Tufte, professor at Yale

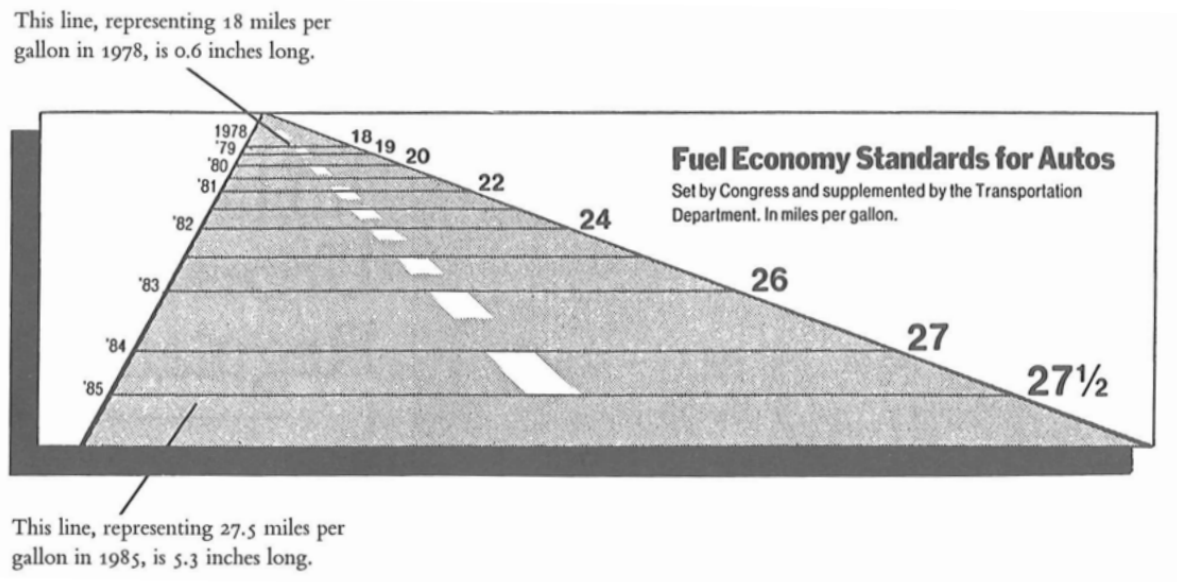
1. Present data richly.
2. Do not hide data, show the truth.
3. Do not use junk charts.
4. Show the variability of data, do not design it.

5. A chart should have the lowest possible lie factor.
6. PowerPoint is evil!

2.11.1 Lie factor

- the ratio of the effect visible on the chart to the effect indicated by the data on which the chart was drawn.

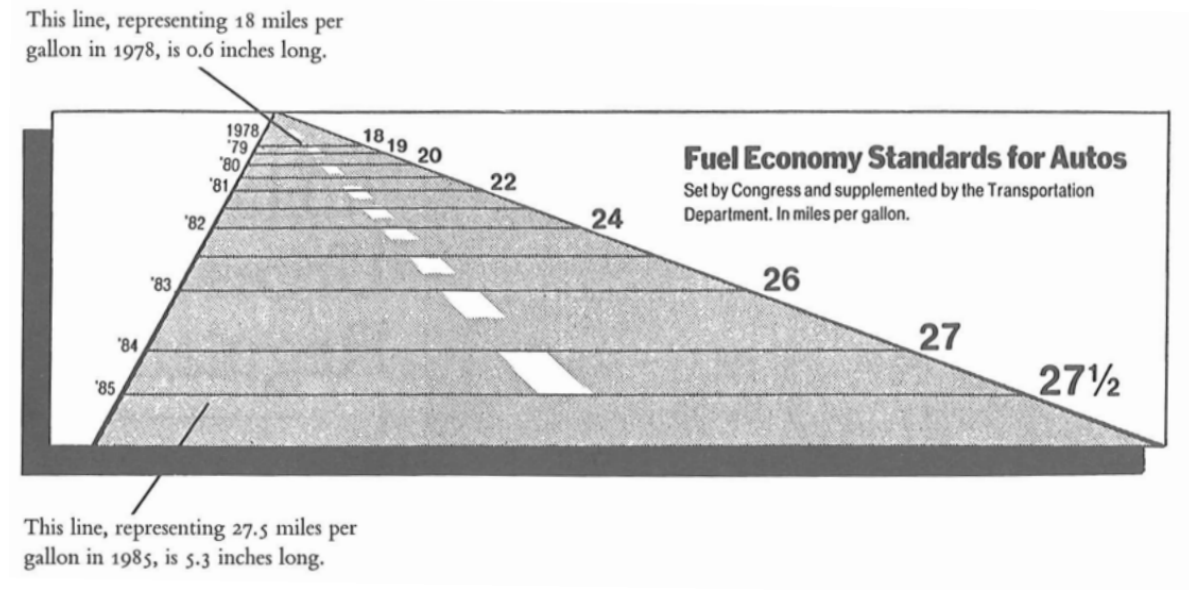
2.11.2 Lie factor



[Tufte, 1991] Edward Tufte, The Visual Display of Quantitative Information, Second Edition, Graphics Press, USA, 1991, p. 57 – 69.

$$\text{LieFactor} = \frac{\text{size of the effect visible on the chart}}{\text{size of the effect resulting from the data}}$$

$$\text{effect size} = \frac{|\text{second value} - \text{first value}|}{\text{first value}}$$



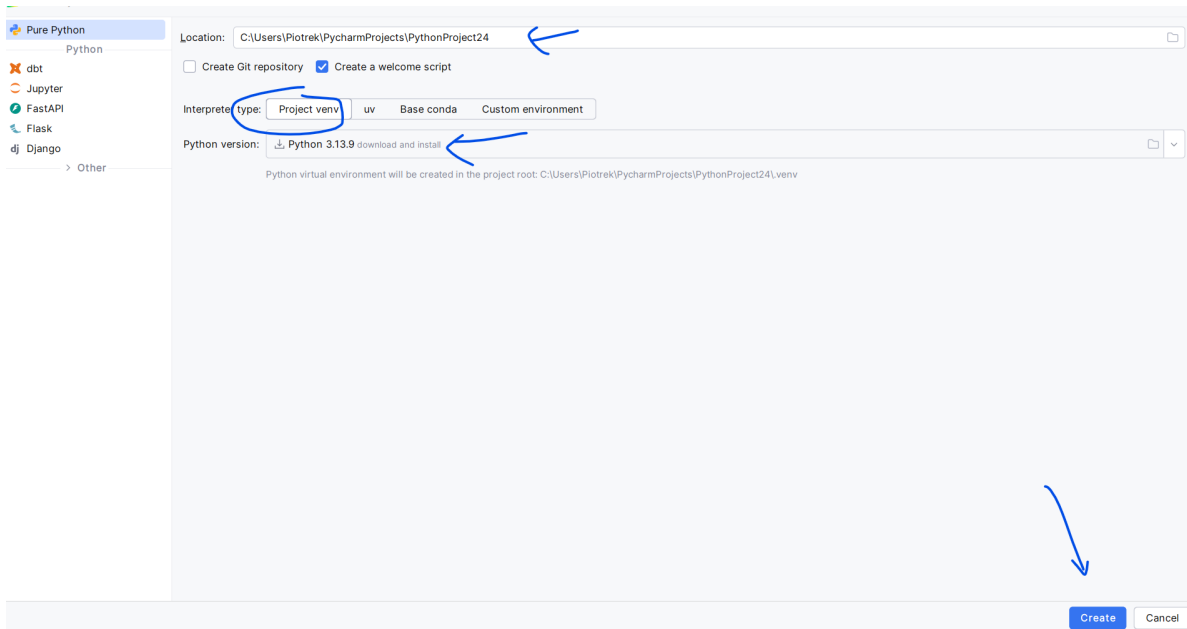
$$\text{LieFactor} = \frac{\frac{5.3-0.6}{0.6}}{\frac{27.5-18}{18}} \approx 14.8$$

2.12 References

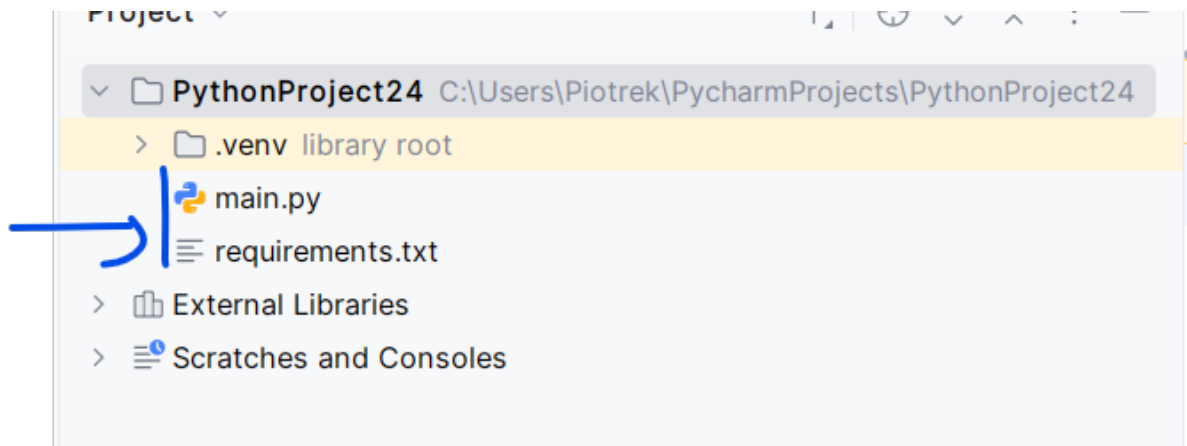
- <https://pl.wikipedia.org/wiki/Wizualizacja>
- https://mfiles.pl/pl/index.php/Analiza_danych, accessed online 1.04.2019.
- Walesiak M., Gatnar E., Statystyczna analiza danych z wykorzystaniem programu R, PWN, Warszawa, 2009.
- Wasilewska E., Statystyka opisowa od podstaw, Podręcznik z zadaniami, Wydawnictwo SGGW, Warszawa, 2009.
- https://en.wikipedia.org/wiki/Cognitive_reflection_test, accessed online 20.03.2023.
- <https://qlikblog.pl/edward-tufte-dobre-praktyki-prezentacji-danych/>, accessed online 20.03.2023.

3 Configuration

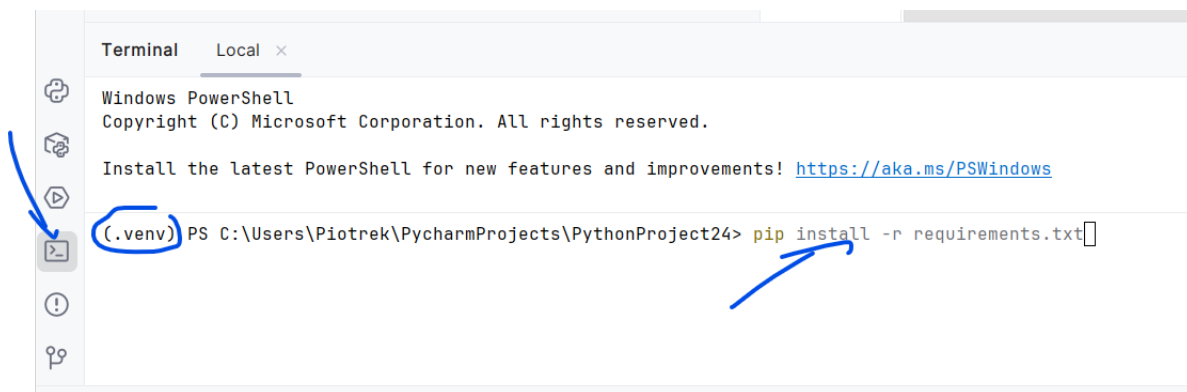
1. In the first step, we create a new project in the chosen location. We select venv and one of the Python versions, e.g. 3.13. Finally, click the Create button.



2. We add the requirements file <https://piojas.pl/wp-content/uploads/2026/02/requirements.txt> to the project's root directory. The file name is important.



3. We open the terminal in PyCharm and type the command: `pip install -r requirements.txt`



Part II

NumPy

4 NumPy - Getting Started

NumPy is a Python library for scientific computing.

Applications:

- linear algebra
- advanced mathematical (numerical) computations
- numerical integration
- solving equations

4.1 Importing the NumPy library

```
import numpy as np
```

The fundamental object in the NumPy library is an N-dimensional array called `ndarray`. Each element of the array is treated as a `dtype` type.

```
np.array(object, dtype=None, *, copy=True, order='K', subok=False, ndmin=0, like=None)
```

- `object` - an object (or sequence) to be converted into an array
- `dtype` - type
- `copy` - whether objects should be copied, default is `True`
- `order` - memory layout: C (rows), F (columns), A, K
- `subok` - handled by subclasses (if `True`), default is `False`
- `ndmin` - minimum number of dimensions of the array
- `like` - creation based on a reference array

```
import numpy as np
```

```
a = np.array([1, 2, 3])
```

①

```
print("a:", a)
```

```
print("type of a:", type(a))
```

②

```
b = np.array([1, 2, 3.0])
```

③


```

print("b:", b)
c = np.array([[1, 2], [3, 4]])
print("c:", c)
d = np.array([1, 2, 3], ndmin=2)
print("d:", d)
e = np.array([1, 2, 3], dtype=complex)
print("e:", e)
f = np.array(np.asmatrix('1 2; 3 4'))
print("f:", f)
g = np.array(np.asmatrix('1 2; 3 4'), subok=True)
print("g:", g)
print(type(g))

```

- ① Creating an array with default settings.
- ② Checking the type.
- ③ One of the elements is of a different type. Therefore, upcasting to the “largest” type occurs.
- ④ Here we get a 2x2 array.
- ⑤ In this line we get an array with dimensions 1x3 (shape (1, 3)).
- ⑥ Forcing the **complex** type — a type with a wider range.
- ⑦ Using the matrix subtype.
- ⑧ Preserving the matrix type.

```

a: [1 2 3]
type of a: <class 'numpy.ndarray'>
b: [1. 2. 3.]
c: [[1 2]
     [3 4]]
d: [[1 2 3]]
e: [1.+0.j 2.+0.j 3.+0.j]
f: [[1 2]
     [3 4]]
g: [[1 2]
     [3 4]]
<class 'numpy.matrix'>

```

5 List vs Array

```
import numpy as np
import time

start_time = time.time()
my_arr = np.arange(1000000)
my_list = list(range(1000000))
start_time = time.time()
my_arr2 = my_arr * 2
print("--- %s seconds ---" % (time.time() - start_time))
start_time = time.time()
my_list2 = [x * 2 for x in my_list]
print("--- %s seconds ---" % (time.time() - start_time))
```

```
--- 0.0020012855529785156 seconds ---
--- 0.036900997161865234 seconds ---
```

6 Array Attributes

Attribute	Description
shape	tuple with information about the number of elements for each dimension
size	total number of elements in the array
ndim	number of dimensions of the array
nbytes	number of bytes the array occupies in memory
dtype	data type stored in the array (e.g. <code>int64</code> , <code>float64</code>)

```
import numpy as np

tab1 = np.array([2, -3, 4, -8, 1])
print("type:", type(tab1))
print("shape:", tab1.shape)
print("size:", tab1.size)
print("ndim:", tab1.ndim)
print("nbytes:", tab1.nbytes)
print("dtype:", tab1.dtype)
```

```
type: <class 'numpy.ndarray'>
shape: (5,)
size: 5
ndim: 1
nbytes: 40
dtype: int64
```

```
import numpy as np

tab2 = np.array([[2, -3], [4, -8]])
print("type:", type(tab2))
print("shape:", tab2.shape)
print("size:", tab2.size)
```

```
print("ndim:", tab2.ndim)
print("nbytes:", tab2.nbytes)
print("dtype:", tab2.dtype)
```

```
type: <class 'numpy.ndarray'>
shape: (2, 2)
size: 4
ndim: 2
nbytes: 32
dtype: int64
```

NumPy does not support ragged arrays. The following code will produce an error:

```
import numpy as np

tab3 = np.array([[2, -3], [4, -8, 5], [3]])
```

7 Data Types

Integer types	int,int8,int16,int32,int64
Unsigned integer types	uint,uint8,uint16,uint32,uint64
Boolean type	bool
Floating-point types	float, float16, float32, float64, float128
Complex floating-point types	complex, complex64, complex128, complex256
String	str

```
import numpy as np

tab = np.array([[2, -3], [4, -8]])
print(tab)
tab2 = np.array([[2, -3], [4, -8]], dtype=int)
print(tab2)
tab3 = np.array([[2, -3], [4, -8]], dtype=float)
print(tab3)
tab4 = np.array([[2, -3], [4, -8]], dtype=complex)
print(tab4)
```

```
[[ 2 -3]
 [ 4 -8]]
[[ 2 -3]
 [ 4 -8]]
[[ 2. -3.]
 [ 4. -8.]]
[[ 2.+0.j -3.+0.j]
 [ 4.+0.j -8.+0.j]]
```

8 Creating Arrays

`np.array` - arguments are cast to an array (anything iterable) - it's worth checking the size/shape

```
import numpy as np

tab = np.array([2, -3, 4])
print(tab)
print("size:", tab.size)
tab2 = np.array((4, -3, 3, 2))
print(tab2)
print("size:", tab2.size)
tab3 = np.array({3, 3, 2, 5, 2})
print(tab3)
print("size:", tab3.size)
tab4 = np.array({"pl": 344, "en": 22})
print(tab4)
print("size:", tab4.size)
```

```
[ 2 -3  4]
size: 3
[ 4 -3  3  2]
size: 4
{2, 3, 5}
size: 1
{'pl': 344, 'en': 22}
size: 1
```

`np.zeros` - creates an array filled with zeros

```
import numpy as np

tab = np.zeros(4)
print(tab)
print(tab.shape)
```

```

tab2 = np.zeros([2, 3])
print(tab2)
print(tab2.shape)
tab3 = np.zeros([2, 3, 4])
print(tab3)
print(tab3.shape)

```

```

[0. 0. 0. 0.]
(4,)
[[0. 0. 0.]
 [0. 0. 0.]]
(2, 3)
[[[0. 0. 0. 0.]
   [0. 0. 0. 0.]
   [0. 0. 0. 0.]]
  [0. 0. 0. 0.]
  [0. 0. 0. 0.]]]
(2, 3, 4)

```

`np.ones` - creates an array filled with ones (this is not the equivalent of an identity matrix!)

```

import numpy as np

tab = np.ones(4)
print(tab)
print(tab.shape)
tab2 = np.ones([2, 3])
print(tab2)
print(tab2.shape)
tab3 = np.ones([2, 3, 4])
print(tab3)
print(tab3.shape)

```

```

[1. 1. 1. 1.]
(4,)
[[1. 1. 1.]
 [1. 1. 1.]]
(2, 3)
[[[1. 1. 1. 1.]
   [1. 1. 1. 1.]
   [1. 1. 1. 1.]]
  [1. 1. 1. 1.]
  [1. 1. 1. 1.]]]

```

```

[1.  1.  1.  1.]
[1.  1.  1.  1.]]

[[1.  1.  1.  1.]
 [1.  1.  1.  1.]
 [1.  1.  1.  1.]]]
(2, 3, 4)

```

`np.diag` - creates an array corresponding to a diagonal matrix

```

import numpy as np

print("tab0")
tab0 = np.diag([3, 4, 5])
print(tab0)
print("tab1")
tab1 = np.array([[2, 3, 4], [3, -4, 5], [3, 4, -5]])
print(tab1)
tab2 = np.diag(tab1)
print("tab2")
print(tab2)
tab3 = np.diag(tab1, k=1)
print("tab3")
print(tab3)
print("tab4")
tab4 = np.diag(tab1, k=-2)
print(tab4)
print("tab5")
tab5 = np.diag(np.diag(tab1))
print(tab5)

```

```

tab0
[[3 0 0]
 [0 4 0]
 [0 0 5]]
tab1
[[ 2  3  4]
 [ 3 -4  5]
 [ 3  4 -5]]
tab2
[ 2 -4 -5]
tab3

```



```
[3 5]
tab4
[3]
tab5
[[ 2  0  0]
 [ 0 -4  0]
 [ 0  0 -5]]
```

`np.arange` - array filled with evenly spaced values

Syntax: `numpy.arange([start, ]stop, [step, ]dtype=None)`

Works similarly to the `range` function, but allows fractional numbers.

```
import numpy as np

a = np.arange(3)
print(a)
b = np.arange(3.0)
print(b)
c = np.arange(3, 7)
print(c)
d = np.arange(3, 11, 2)
print(d)
e = np.arange(0, 1, 0.1)
print(e)
f = np.arange(3, 11, 2, dtype=float)
print(f)
g = np.arange(3, 10, 2)
print(g)
```

```
[0 1 2]
[0. 1. 2.]
[3 4 5 6]
[3 5 7 9]
[0.  0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9]
[3. 5. 7. 9.]
[3 5 7 9]
```

`np.linspace` - array filled with evenly spaced values on a linear scale

`np.linspace(start, stop, num)` generates an array of evenly spaced values. For example, `np.linspace(2.0, 3.0, num=5)` produces 2, 2.25, 2.5, 2.75, 3 — the interval is divided

into `num - 1` equal parts. Setting `endpoint=False` removes the last point (from the right side), and the interval is then divided into `num` equal parts instead. The `dtype` parameter forces the output type — typically set to `int`, in which case the results may have the fractional part truncated.

```
import numpy as np

a = np.linspace(2.0, 3.0, num=5)
print(a)
b = np.linspace(2.0, 3.0, num=5, endpoint=False)
print(b)
c = np.linspace(10, 20, num=4)
print(c)
d = np.linspace(10, 20, num=4, dtype=int)
print(d)
```

```
[2.  2.25 2.5  2.75 3.  ]
[2.  2.2 2.4 2.6 2.8]
[10.          13.33333333 16.66666667 20.          ]
[10 13 16 20]
```

`np.logspace` - array filled with values on a logarithmic scale

Syntax: `numpy.logspace(start, stop, num=50, endpoint=True, base=10.0, dtype=None, axis=0)`

The function `np.logspace(2.0, 3.0, num=4)` generates 4 numbers spaced evenly on a logarithmic scale between 10^2 and 10^3 . The default base is 10. First, the exponents are computed as equally spaced points in the interval $[2.0, 3.0]$, which gives approximately 2, 2.33, 2.66, and 3.0. Then each exponent is used as the power of 10, producing the output values $10^2 = 100$, $10^{2.33} \approx 215.44$, $10^{2.66} \approx 464.16$, and $10^3 = 1000$. An important consequence is that while the exponents are uniformly distributed, the resulting values are not — the distances between consecutive output numbers grow larger as the values increase. This is the key distinction between `np.logspace` and `np.linspace`: the former creates equal spacing in log-scale (i.e. between the exponents), not in the actual output values.

```
import numpy as np

a = np.logspace(2.0, 3.0, num=4)
print(a)
b = np.logspace(2.0, 3.0, num=4, endpoint=False)
print(b)
```

```
c = np.logspace(2.0, 3.0, num=4, base=2.0)
print(c)
```

```
[ 100.          215.443469   464.15888336 1000.          ]
[100.          177.827941   316.22776602 562.34132519]
[4.           5.0396842   6.34960421 8.           ]
```

`np.empty` - empty (uninitialized) array - specific values are not “guaranteed”

```
import numpy as np

a = np.empty(3)
print(a)
b = np.empty(3, dtype=int)
print(b)
```

```
[0.  1.  2.]
[          0 4607182418800017408 4611686018427387904]
```

`np.identity` - array resembling an identity matrix

`np.eye` - array with ones on the diagonal (remaining values are zeros)

```
import numpy as np

print("a")
a = np.identity(4)
print(a)
print("b")
b = np.eye(4, k=1)
print(b)
print("c")
c = np.eye(4, k=2)
print(c)
print("d")
d = np.eye(4, k=-1)
print(d)
```

```
a
[[1.  0.  0.  0.]
```

```
[0. 1. 0. 0.]  
[0. 0. 1. 0.]  
[0. 0. 0. 1.]]
```

b

```
[[0. 1. 0. 0.]  
 [0. 0. 1. 0.]  
 [0. 0. 0. 1.]  
 [0. 0. 0. 0.]]
```

c

```
[[0. 0. 1. 0.]  
 [0. 0. 0. 1.]  
 [0. 0. 0. 0.]  
 [0. 0. 0. 0.]]
```

d

```
[[0. 0. 0. 0.]  
 [1. 0. 0. 0.]  
 [0. 1. 0. 0.]  
 [0. 0. 1. 0.]]
```

9 Indexing, “slicing”

```
import numpy as np

a = np.array([2, 5, -2, 4, -7, 8, 9, 11, -23, -4, -7, 16, 1])
print("1:", a[5])
print("2:", a[-2])
print("3:", a[3:6])
print("4:", a[:])
print("5:", a[0:-1])
print("6:", a[:5])
```

①
②
③
④
⑤
⑥

- ① Access the element at index 5.
- ② Access the second element from the end.
- ③ Access elements with indices from 3 to 5 (inclusive) — the principle of half-open intervals (closed on the left, open on the right).
- ④ Access all elements.
- ⑤ Access all elements except the last one.
- ⑥ Access from the beginning up to the element at index 4.

```
1: 8
2: 16
3: [ 4 -7  8]
4: [  2  5 -2  4 -7  8  9 11 -23 -4 -7 16  1]
5: [  2  5 -2  4 -7  8  9 11 -23 -4 -7 16]
6: [ 2  5 -2  4 -7]
```

```
import numpy as np

print("1:", a[4:])
print("2:", a[4:-1])
print("3:", a[4:10:2])
print("4:", a[::-1])
print("5:", a[:2])
print("6:", a[::-2])
```

①
②
③
④
⑤
⑥

- ① Access elements from index 4 to the end.
- ② Access elements from index 4 to the end, excluding the last one.
- ③ Access elements whose indices form an arithmetic sequence from 4 to 10 (including 4, but excluding 10) with a step of 2.
- ④ Access elements in reverse order (from end to beginning).
- ⑤ Access elements at even indices from the beginning.
- ⑥ Access every second element, counting from the end (reversed order).

```
1: [ -7   8   9  11 -23  -4  -7  16   1]
2: [ -7   8   9  11 -23  -4  -7  16]
3: [ -7   9 -23]
4: [  1  16  -7  -4 -23  11   9   8  -7   4  -2   5   2]
5: [  2  -2  -7   9 -23  -7   1]
6: [  1  -7 -23   9  -7  -2   2]
```

```
import numpy as np

a = np.array([[3, 4, 5], [-3, 4, 8], [3, 2, 9]])
b = a[:, 1:]
print(b)
print(np.shape(b))
c = a[1]
print(c)
print(np.shape(c))
d = a[1, :]
print(d)
print(np.shape(d))
```

```
[[4 5]
 [4 8]]
(2, 2)
[-3  4  8]
(3,)
[-3  4  8]
(3,)
```

```
import numpy as np

a = np.array([[3, 4, 5], [-3, 4, 8], [3, 2, 9]])
e = a[1:2, :]
print(e)
```

```

print(np.shape(e))
f = a[:, :2]
print(f)
print(np.shape(f))
g = a[1, :2]
print(g)
print(np.shape(g))
h = a[1:2, :2]
print(h)
print(np.shape(h))

```

```

[[-3  4  8]]
(1, 3)
[[ 3  4]
 [-3  4]
 [ 3  2]]
(3, 2)
[-3  4]
(2,)
[[-3  4]]
(1, 2)

```

Note — “slicing” creates a so-called **view**, not a copy. Changes to the view affect the original array and vice versa.

```

import numpy as np

a = np.array([[3, 4, 5], [-3, 4, 8], [3, 2, 9]])
b = a[1:2, 1:]
print(b)
a[1][1] = 9
print(a)
print(b)
b[0][0] = -11
print(a)
print(b)

```

```

[[4 8]]
[[ 3  4  5]
 [-3  9  8]
 [ 3  2  9]]

```

```
[[9 8]]
[[ 3  4  5]
 [-3 -11  8]
 [ 3  2  9]]
[[-11  8]]
```

Solution — use the `.copy()` method:

```
import numpy as np

a = np.array([[3, 4, 5], [-3, 4, 8], [3, 2, 9]])
b = a[1:2, 1:].copy()
print(b)
a[1][1] = 9
print(a)
print(b)
b[0][0] = -11
print(a)
print(b)
```

```
[[4 8]]
[[ 3  4  5]
 [-3  9  8]
 [ 3  2  9]]
[[4 8]]
[[ 3  4  5]
 [-3  9  8]
 [ 3  2  9]]
[[-11  8]]
```

Fancy indexing (indexing with an array of indices)

```
import numpy as np

a = np.array([2, 5, -2, 4, -7, 8, 9, 11, -23, -4, -7, 8, 1])
b = a[np.array([1, 3, 7])]
print(b)
c = a[[1, 3, 7]]
print(c)
```

```
[ 5  4 11]
[ 5  4 11]
```


Boolean indexing (boolean masks)

```
import numpy as np

a = np.array([2, 5, -2, 4, -7, 8, 9, 11, -23, -4, -7, 8, 1])
b = a > 0
print(b)
c = a[a > 0]
print(c)
d = a[(a > 5) & (a%2 !=0)] # the & operator stands for AND
print(d)
e = a[(a > 5) | (a%2 !=0)] # the | operator stands for OR
print(e)
f = a[(a > 5) ^ (a%2 !=0)] # the ^ operator stands for XOR
print(f)
g = a[~(a > 0)]
print(g)
```

```
[ True  True False  True False  True  True  True False False False  True
  True]
[ 2  5  4  8  9 11  8  1]
[ 9 11]
[ 5 -7  8  9 11 -23 -7  8  1]
[ 5 -7  8 -23 -7  8  1]
[-2 -7 -23 -4 -7]
```

Note — fancy and boolean indexing return a copy, not a view (unlike “slicing”).

```
import numpy as np

a = np.array([2, 5, -2, 4, -7, 8, 9, 11, -23, -4, -7, 8, 1])
b = a[a > 0]
print(b)
b[0] = -5
print(a)
print(b)
a[1] = 20
print(a)
print(b)
```

```
[ 2  5  4  8  9 11  8  1]
```

```

[ 2  5 -2  4 -7  8  9 11 -23 -4 -7  8  1]
[-5 5  4  8  9 11  8  1]
[ 2 20 -2  4 -7  8  9 11 -23 -4 -7  8  1]
[-5 5  4  8  9 11  8  1]

```

10 Modifying shape and size

```
import numpy as np

print("a")
a = np.array([[3, 4, 5], [-3, 4, 8], [3, 2, 9]])
print(a)
print("b")
b = np.reshape(a, (1, 9))
print(b)
print("c")
c = a.reshape(9)
print(c)
```

```
a
[[ 3  4  5]
 [-3  4  8]
 [ 3  2  9]]
b
[[ 3  4  5 -3  4  8  3  2  9]]
c
[ 3  4  5 -3  4  8  3  2  9]
```

```
import numpy as np

print("a")
a = np.array([[3, 4, 5], [-3, 4, 8], [3, 2, 9]])
print(a)
print("d")
d = a.flatten()    # always returns a copy
print(d)
print("e")
e = a.ravel()      # returns a view (shared memory with original)
print(e)
print("f")
```

```
f = np.ravel(a)
print(f)
```

```
a
[[ 3  4  5]
 [-3  4  8]
 [ 3  2  9]]
d
[ 3  4  5 -3  4  8  3  2  9]
e
[ 3  4  5 -3  4  8  3  2  9]
f
[ 3  4  5 -3  4  8  3  2  9]
```

```
import numpy as np

print("g")
g = np.array([[1, 3, 4]])
print(g)
print("h")
h = np.squeeze(g)
print(h)
print("i")
a = np.array([[3, 4, 5], [-3, 4, 8], [3, 2, 9]])
print(a)
i = a.T
print(i)
print("j")
j = np.transpose(a)
print(j)
```

```
g
[[1 3 4]]
h
[1 3 4]
i
[[ 3  4  5]
 [-3  4  8]
 [ 3  2  9]]
[[ 3 -3  3]
 [ 4  4  2]]
```

```

[ 5  8  9]]
j
[[ 3 -3  3]
 [ 4  4  2]
 [ 5  8  9]]

```

```

import numpy as np

print("n")
n = np.array([3, -4, 5, -2])
print(n)
print("k")
k = np.hstack((n, n, n))
print(k)
print("l")
l = np.vstack((n, n, n))
print(l)
print("m")
m = np.dstack((n, n, n))
print(m)

```

```

n
[ 3 -4  5 -2]
k
[ 3 -4  5 -2  3 -4  5 -2  3 -4  5 -2]
l
[[ 3 -4  5 -2]
 [ 3 -4  5 -2]
 [ 3 -4  5 -2]]
m
[[[ 3  3  3]
  [-4 -4 -4]
  [ 5  5  5]
  [-2 -2 -2]]]

```

```

import numpy as np

a = np.array([[1, 2], [3, 4]])
b = np.array([[5, 6]])
print("r1")
r1 = np.concatenate((a, b))      # axis=0 is the default
print(r1)

```

```

print("r2")
r2 = np.concatenate((a, b), axis=0) # same as above (explicit)
print(r2)
print("r3")
r3 = np.concatenate((a, b.T), axis=1)
print(r3)
print("r4")
r4 = np.concatenate((a, b), axis=None)
print(r4)

```

```

r1
[[1 2]
 [3 4]
 [5 6]]
r2
[[1 2]
 [3 4]
 [5 6]]
r3
[[1 2 5]
 [3 4 6]]
r4
[1 2 3 4 5 6]

```

```

import numpy as np

a = np.array([[1, 2], [3, 4]])
print("r1")
r1 = np.resize(a, (2, 3))
print(r1)
print("r2")
r2 = np.resize(a, (1, 4))
print(r2)
print("r3")
r3 = np.resize(a, (2, 4))
print(r3)

```

```

r1
[[1 2 3]
 [4 1 2]]
r2

```

```
[[1 2 3 4]]
r3
[[1 2 3 4]
 [1 2 3 4]]
```

```
import numpy as np

a = np.array([[1, 2], [3, 4]])
b = np.array([[5, 6]])
print("r1")
r1 = np.append(a, b)
print(r1)
print("r2")
r2 = np.append(a, b, axis=0)
print(r2)
```

```
r1
[1 2 3 4 5 6]
r2
[[1 2]
 [3 4]
 [5 6]]
```

```
import numpy as np

a = np.array([[1, 2], [3, 7]])
print("r1")
r1 = np.insert(a, 1, 4)
print(r1)
print("r2")
r2 = np.insert(a, 2, 4)
print(r2)
print("r3")
r3 = np.insert(a, 1, 4, axis=0)
print(r3)
print("r4")
r4 = np.insert(a, 1, 4, axis=1)
print(r4)
```

```
r1
[1 4 2 3 7]
```

```
r2
[1 2 4 3 7]
r3
[[1 2]
 [4 4]
 [3 7]]
r4
[[1 4 2]
 [3 4 7]]
```

```
import numpy as np

a = np.array([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]])
print("r1")
r1 = np.delete(a, 1, axis=1)
print(r1)
print("r2")
r2 = np.delete(a, 2, axis=0)
print(r2)
```

```
r1
[[ 1  3  4]
 [ 5  7  8]
 [ 9 11 12]]
r2
[[1 2 3 4]
 [5 6 7 8]]
```


11 Broadcasting

The variants discussed below are illustrative examples.

Variant 1 — scalar-array — performing an operation on every element of the array

When a scalar is combined with an array, NumPy broadcasts the scalar to match the array's shape, then performs the operation element-wise. In the code above, array **a** has shape 3×2 and contains $\begin{bmatrix} 1 & 2 \\ 5 & 6 \\ 9 & 10 \end{bmatrix}$. The expression **a** + 4 conceptually expands the scalar 4 into a 3×2 array filled with 4s, so the result **b** is $\begin{bmatrix} 5 & 6 \\ 9 & 10 \\ 13 & 14 \end{bmatrix}$. The same principle applies to **2** ** **a**, where the scalar 2 is broadcast across every element of **a**, producing $\begin{bmatrix} 2 & 4 \\ 32 & 64 \\ 512 & 1024 \end{bmatrix}$ — each element is computed as 2 raised to the corresponding value in **a**.

```
import numpy as np

a = np.array([[1, 2], [5, 6], [9, 10]])
b = a + 4
print(b)
c = 2 ** a
print(c)
```

```
[[ 5  6]
 [ 9 10]
 [13 14]]
[[ 2  4]
 [32 64]
 [512 1024]]
```

Variant 2 — two arrays — “when one of the arrays can be expanded” (both dimensions are equal or one of them is equal to 1)

When two arrays of different but compatible shapes are combined, NumPy stretches the smaller one to match the larger. Array **a** has shape 2×2 and **b** is a 1D array of shape (2,), which NumPy treats as a single row. In **a** + **b**, the row $[9, 2]$ is broadcast across both rows of **a**, giving $\begin{bmatrix} 10 & 4 \\ 14 & 8 \end{bmatrix}$. Similarly, **a** / **b** divides each row of **a** element-wise by $[9, 2]$, producing $\begin{bmatrix} 1/9 & 1 \\ 5/9 & 3 \end{bmatrix}$. Array **c** has shape 2×1 , meaning it acts as a column. In **a** + **c**, the value 4 is added across the entire first row and -2 across the second, yielding $\begin{bmatrix} 5 & 6 \\ 3 & 4 \end{bmatrix}$. Likewise, **c** /

`a` divides the column values by each element in the corresponding row, so the first row becomes $[4/1, 4/2]$ and the second $[-2/5, -2/6]$. The general rule is that dimensions are compatible when they are equal or one of them is 1 — the dimension of size 1 is then stretched to match the other.

```
import numpy as np

a = np.array([[1, 2], [5, 6]])
b = np.array([9, 2])
r1 = a + b
print(r1)
r2 = a / b
print(r2)
c = np.array([[4], [-2]])
r3 = a + c
print(r3)
r4 = c / a
print(r4)
```

```
[[10  4]
 [14  8]]
[[0.11111111 1.          ]
 [0.55555556 3.          ]]
[[5 6]
 [3 4]]
[[ 4.          2.          ]
 [-0.4        -0.33333333]]
```

Variant 3 — “column” and “row”

Array `a` is created as a row $[[5, 2, -3]]$ and then transposed with `.T`, turning it into a column of shape 3×1 . Array `b` is a 1D array of shape $(5,)$, treated by NumPy as a single row. When these two are combined, broadcasting stretches the column across 5 columns and the row across 3 rows, producing a 3×5 result in every case. In `a + b`, each value from the column $[5, 2, -3]$ is added to every element of the row $[3, -2, 1, 2, 4]$, giving $[[8, 3, 6, 7, 9], [5, 0, 3, 4, 6], [0, -5, -2, -1, 1]]$. The expression `b + a` yields the same result, since addition is commutative. For `a * b`, each column value is multiplied by every row value, producing $[[15, -10, 5, 10, 20], [6, -4, 2, 4, 8], [-9, 6, -3, -6, -12]]$. This column-times-row pattern is essentially an outer operation — every pair of elements from the two arrays is combined, and the result shape is determined by the lengths of the column and the row.

```
import numpy as np

a = np.array([[5, 2, -3]]).T
b = np.array([3, -2, 1, 2, 4])
print(a+b)
print(b+a)
print(a*b)
```

```
[[ 8  3  6  7  9]
 [ 5  0  3  4  6]
 [ 0 -5 -2 -1  1]]
[[ 8  3  6  7  9]
 [ 5  0  3  4  6]
 [ 0 -5 -2 -1  1]]
[[ 15 -10  5  10  20]
 [  6  -4  2   4   8]
 [ -9   6 -3  -6 -12]]
```

12 Universal functions (ufunc) and other mathematical functions

Universal functions (known as *ufuncs*) are among the most important tools in NumPy. They are functions that operate element-by-element on arrays, often implemented in C, which ensures high computational performance. Thanks to *ufuncs*, you can perform arithmetic, trigonometric, statistical, and logical operations on entire arrays in a simple and readable way, without the need to write loops in Python.

12.1 Basic arithmetic operations

NumPy automatically translates mathematical operators into the corresponding *ufuncs*. For example:

- `+` corresponds to `np.add`
- `-` corresponds to `np.subtract`
- `*` corresponds to `np.multiply`
- `/` corresponds to `np.divide`
- `**` corresponds to `np.power`

Example:

```
import numpy as np

A = np.array([1, 2, 3, 4])
B = np.array([10, 20, 30, 40])

# Element-by-element operations
sum_tab = np.add(A, B)          # same as A + B
diff_tab = np.subtract(B, A)    # same as B - A
mul_tab = np.multiply(A, 2)     # same as A * 2
pow_tab = np.power(A, 3)        # same as A ** 3

print("Sum:", sum_tab)
print("Difference:", diff_tab)
```

```
print("Multiplication by 2:", mul_tab)
print("Exponentiation:", pow_tab)
```

```
Sum: [11 22 33 44]
Difference: [ 9 18 27 36]
Multiplication by 2: [2 4 6 8]
Exponentiation: [ 1  8 27 64]
```

12.2 Trigonometric and related functions

NumPy offers a rich set of trigonometric functions:

- `np.sin`, `np.cos`, `np.tan` — basic functions,
- `np.arcsin`, `np.arccos`, `np.arctan` — inverse trigonometric functions,
- `np.sinh`, `np.cosh`, `np.tanh` — hyperbolic functions.

Example:

```
import numpy as np

x = np.linspace(0, np.pi, 5) # array [0, /4, /2, 3/4, ]
sin_values = np.sin(x)
cos_values = np.cos(x)

print("sin(x) values:", sin_values)
print("cos(x) values:", cos_values)
```

```
sin(x) values: [0.00000000e+00  7.07106781e-01  1.00000000e+00  7.07106781e-01
 1.22464680e-16]
cos(x) values: [ 1.00000000e+00  7.07106781e-01  6.12323400e-17 -7.07106781e-
01
-1.00000000e+00]
```

12.3 Exponential and logarithmic functions

- `np.exp` — exponential function,
- `np.log` — natural logarithm,
- `np.log10` — base-10 logarithm.

Example:

```
import numpy as np

A = np.array([1, np.e, np.e**2])
print("A:", A)
print("log(A):", np.log(A))
print("exp(A):", np.exp([0, 1, 2])) # exp(0)=1, exp(1)=e, exp(2)=e^2
```

```
A: [1.          2.71828183  7.3890561 ]
log(A): [0.  1.  2.]
exp(A): [1.          2.71828183  7.3890561 ]
```

12.4 Rounding functions and absolute values

- `np.round` — rounds to the nearest number,
- `np.floor` — floor,
- `np.ceil` — ceiling,
- `np.trunc` — truncation to the integer part,
- `np.abs` — absolute value.

Example:

```
import numpy as np

B = np.array([1.7, -2.5, 3.5, 4.52, -4.1])
print("B:", B)
print("floor(B):", np.floor(B))
print("ceil(B):", np.ceil(B))
print("abs(B):", np.abs(B))
print("round(B):", np.round(B))
print("rint(B):", np.rint(B))
print("round(B,1):", np.round(B,1))
```

```
B: [ 1.7 -2.5  3.5  4.52 -4.1 ]
floor(B): [ 1. -3.  3.  4. -5.]
ceil(B): [ 2. -2.  4.  5. -4.]
abs(B): [1.7  2.5  3.5  4.52 4.1 ]
round(B): [ 2. -2.  4.  5. -4.]
rint(B): [ 2. -2.  4.  5. -4.]
round(B,1): [ 1.7 -2.5  3.5  4.5 -4.1]
```

12.5 Statistical and aggregate functions

Although many statistical functions are available as array methods (e.g. `A.mean()`, `A.std()`), there are also ufuncs that operate element-by-element or accept axis parameters:

- `np.minimum`, `np.maximum` — return the element-by-element minimum/maximum of two arrays,
- `np.fmin`, `np.fmax` — similar to the above, but ignore NaN values,
- `np.sqrt` — square root,
- `np.square` — squaring.

Example:

```
import numpy as np

C1 = np.array([1, 4, 9, 16])
C2 = np.array([2, 2, 5, 20])

print("element-wise minimum of C1 and C2:", np.minimum(C1, C2))
print("element-wise maximum of C1 and C2:", np.maximum(C1, C2))
print("sqrt(C1):", np.sqrt(C1))
print("square(C2):", np.square(C2))
```

```
element-wise minimum of C1 and C2: [ 1  2  5 16]
element-wise maximum of C1 and C2: [ 2  4  9 20]
sqrt(C1): [1.  2.  3.  4.]
square(C2): [ 4  4 25 400]
```

13 String operations

In NumPy, besides the well-known numerical arrays, there is also a set of functions that allow vectorized operations on strings.

Important: The functions below are typically available in the `numpy.char` module. In the documentation, they can be found in the [String operations](#) section, but in this material we will focus on how to use them via the `numpy.strings` module interface. This is analogous to using `numpy.char`, but is the newer approach.

13.1 Creating arrays with strings

NumPy allows storing text in arrays, for example:

```
import numpy as np

arr = np.array(["python", "NumPy", "data", "Science"])
print(arr)
```

```
['python' 'NumPy' 'data' 'Science']
```

13.2 Basic text modification functions

Below are popular functions for modifying text in string arrays:

13.2.1 `numpy.strings.upper` and `numpy.strings.lower`

- `upper`: Converts all letters to uppercase.
- `lower`: Converts all letters to lowercase.


```
import numpy as np

arr = np.array(["python", "NumPy", "data", "Science"])

print(np.strings.upper(arr))
print(np.strings.lower(arr))
```

```
['PYTHON' 'NUMPY' 'DATA' 'SCIENCE']
['python' 'numpy' 'data' 'science']
```

13.2.2 `numpy.strings.capitalize`

The `capitalize` function converts the first letter of the entire string to uppercase and the rest to lowercase.

```
import numpy as np

arr = np.array(["python", "NumPy", "data", "Science"])
print(np.strings.capitalize(arr))
```

```
['Python' 'Numpy' 'Data' 'Science']
```

13.2.3 `numpy.strings.title`

The `title` function converts the first letter of each word to uppercase and the remaining letters of each word to lowercase.

```
import numpy as np

arr2 = np.array(["python data science", "machine learning", "deep learning"])
print(np.strings.title(arr2))
```

```
['Python Data Science' 'Machine Learning' 'Deep Learning']
```

13.3 Joining and splitting text

13.3.1 `numpy.strings.add`

The `add` function concatenates elements of text arrays pairwise, acting like the `+` operator on strings, but in a vectorized way. The concatenation is performed without any separator.

```
import numpy as np

arr_a = np.array(["Hello", "Data"])
arr_b = np.array(["World", "Science"])

# without separator
print(np.strings.add(arr_a, arr_b))
# with space -- it must be added manually
print(np.strings.add(np.strings.add(arr_a, np.array([" ", " "]])), arr_b))
```

```
['HelloWorld' 'DataScience']
['Hello World' 'Data Science']
```

13.3.2 `numpy.char.join`

The `join` function inserts a specified separator between individual characters within each element of the array.

```
import numpy as np

arr3 = np.array(["python", "numpy", "string"])
print(np.char.join("-", arr3))
```

```
['p-y-t-h-o-n' 'n-u-m-p-y' 's-t-r-i-n-g']
```

Note: `join` vectorizes the operation, treating each element of the array as a sequence of characters to be joined with the separator.

13.3.3 `numpy.char.split`

Allows splitting strings by a given separator. Returns an array containing lists of substrings.

```
import numpy as np

arr4 = np.array(["python-data-science", "machine-learning"])
print(np.char.split(arr4, sep="-"))
```

```
[list(['python', 'data', 'science']) list(['machine', 'learning'])]
```

13.4 Searching and replacing substrings

13.4.1 `numpy.strings.find` and `numpy.strings.rfind`

- `find`: Returns the index of the first occurrence of a substring (or -1 if not found).
- `rfind`: Returns the index of the last occurrence of a substring (or -1 if not found).

```
import numpy as np

arr5 = np.array(["banana", "data", "abracadabra"])
# find -- first occurrence
print(np.strings.find(arr5, "a"))
# rfind -- last occurrence
print(np.strings.rfind(arr5, "a"))
```

```
[1 1 0]
[ 5  3 10]
```

13.4.2 `numpy.strings.replace`

`replace` substitutes all occurrences of a substring with a new string.

```
import numpy as np

arr6 = np.array(["python", "pydata", "pypy"])
print(np.strings.replace(arr6, "py", "PY"))
```

```
['PYthon' 'PYdata' 'PYPY']
```

13.5 Removing unwanted characters

13.5.1 `numpy.strings.strip`, `numpy.strings.lstrip` and `numpy.strings.rstrip`

- `strip`: Removes specified characters from the beginning and end.
- `lstrip`: Removes specified characters from the left side (beginning).
- `rstrip`: Removes specified characters from the right side (end).

```
import numpy as np

arr7 = np.array(["  python  ", "  numpy  "])
# strip -- removes from both sides
print(np.strings.strip(arr7))
# lstrip -- removes only from the left
print(np.strings.lstrip(arr7))
# rstrip -- removes only from the right
print(np.strings.rstrip(arr7))
```

```
['python' 'numpy']
['python' 'numpy']
['python' 'numpy']
```

We can also specify custom characters to remove:

```
import numpy as np

arr8 = np.array(["###data###", "###science###"])
print(np.strings.strip(arr8, "##"))
```

```
['data' 'science']
```

14 Linear Algebra

14.1 Dot Product

For two vectors, `dot` computes their dot product.

```
import numpy as np

# Dot product of two vectors
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
result = np.dot(a, b) # 1*4 + 2*5 + 3*6
print(result) # Result: 32

# Alternative notation using the @ operator
result = a @ b
print(result) # Result: 32
```

32

32

14.2 Matrix Multiplication

For matrices (two-dimensional arrays), `dot` performs standard matrix multiplication.

```
import numpy as np
# Matrix multiplication
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])
C = np.dot(A, B)
print(C)
# Result:
# [[19 22]
#  [43 50]]
```

```
# The same using the @ operator
C = A @ B
print(C)
```

```
[[19 22]
 [43 50]]
[[19 22]
 [43 50]]
```

14.3 Matrix-Vector Multiplication

We can also multiply a matrix by a vector:

```
import numpy as np
# Matrix-vector multiplication
A = np.array([[1, 2], [3, 4]])
v = np.array([5, 6])
result = np.dot(A, v)
print(result) # Result: [17 39]
```

```
[17 39]
```

14.4 Solving Systems of Linear Equations

The `numpy.linalg.solve` function solves systems of linear equations of the form $Ax = b$:

```
import numpy as np
# Solving a system of linear equations
A = np.array([[3, 1], [1, 2]])
b = np.array([9, 8])
x = np.linalg.solve(A, b)
print(x) # Result: [2. 3.]

# Verifying the solution
np.dot(A, x) # Should be equal to b
```

```
[2. 3.]
```

```
array([9., 8.])
```

14.5 Matrix Determinant

The `numpy.linalg.det` function computes the determinant of a matrix:

```
import numpy as np
# Computing the determinant
A = np.array([[1, 2], [3, 4]])
det_A = np.linalg.det(A)
print(det_A) # Result: -2.0
```

-2.0000000000000004

14.6 Eigenvalues and Eigenvectors

The `numpy.linalg.eig` function computes the eigenvalues and eigenvectors of a matrix:

```
import numpy as np
# Computing eigenvalues and eigenvectors
A = np.array([[4, -2], [1, 1]])
eigenvalues, eigenvectors = np.linalg.eig(A)
print("Eigenvalues:", eigenvalues)
print("Eigenvectors:")
print(eigenvectors)

# Verification: A * v = lambda * v
for i in range(len(eigenvalues)):
    lambda_i = eigenvalues[i]
    v_i = eigenvectors[:, i]
    print(f"_{{i}} = {{lambda_i}}")
    print("A * v =", np.dot(A, v_i))
    print(" * v =", lambda_i * v_i)
```

Eigenvalues: [3. 2.]

Eigenvectors:

[[0.89442719 0.70710678]

[0.4472136 0.70710678]]

_0 = 3.0

A * v = [2.68328157 1.34164079]

* v = [2.68328157 1.34164079]

_1 = 2.0

```
A * v = [1.41421356 1.41421356]
* v = [1.41421356 1.41421356]
```

14.7 Singular Value Decomposition (SVD)

SVD is one of the key methods used in data analysis:

```
import numpy as np
# SVD decomposition
A = np.array([[1, 2], [3, 4], [5, 6]])
U, s, Vh = np.linalg.svd(A)
print("Matrix U:")
print(U)
print("Singular values:", s)
print("Matrix V^H:")
print(Vh)

# Reconstructing matrix A
S = np.zeros((A.shape[0], A.shape[1]))
S[:len(s), :len(s)] = np.diag(s)
A_reconstructed = U @ S @ Vh
print("Reconstructed matrix A:")
print(A_reconstructed)
```

```
Matrix U:
[[-0.2298477  0.88346102  0.40824829]
 [-0.52474482  0.24078249 -0.81649658]
 [-0.81964194 -0.40189603  0.40824829]]
Singular values: [9.52551809 0.51430058]
Matrix V^H:
[[-0.61962948 -0.78489445]
 [-0.78489445  0.61962948]]
Reconstructed matrix A:
[[1. 2.]
 [3. 4.]
 [5. 6.]]
```

14.8 Matrix/Vector Norm

NumPy offers various types of vector and matrix norms:


```

import numpy as np

v = np.array([3, 4])
# L1 (sum of absolute values): |3| + |4| = 7
print("L1 norm:", np.linalg.norm(v, 1))
# L2 / Euclidean (square root of sum of squares): sqrt(9+16) = 5
print("L2 norm (Euclidean):", np.linalg.norm(v))
# Maximum norm (largest absolute value): max(3,4) = 4
print("Maximum norm:", np.linalg.norm(v, np.inf))

A = np.array([[1, 2], [3, 4]])
# Frobenius (like L2, but for matrices): sqrt(1+4+9+16) = 5.477
print("Frobenius norm:", np.linalg.norm(A, 'fro'))

```

```

L1 norm: 7.0
L2 norm (Euclidean): 5.0
Maximum norm: 4.0
Frobenius norm: 5.477225575051661

```

14.9 Inverse Matrix

The `numpy.linalg.inv` function computes the inverse of a matrix:

```

import numpy as np
# Inverse matrix
A = np.array([[1, 2], [3, 4]])
A_inv = np.linalg.inv(A)
print("Inverse matrix:")
print(A_inv)

# Verification: A * A^(-1) = I
print("A * A^(-1):")
print(np.dot(A, A_inv)) # Should be close to the identity matrix

```

```

Inverse matrix:
[[-2.   1.]
 [ 1.5 -0.5]]
A * A^(-1):
[[1.0000000e+00 0.0000000e+00]
 [8.8817842e-16 1.0000000e+00]]

```

14.10 The `numpy.inner` Function - Inner Product

The `inner` function computes the inner product of two arrays:

```
import numpy as np
# Inner product
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
result = np.inner(a, b)
print(result)  # 1*4 + 2*5 + 3*6 = 32

# For 2D arrays: inner(A, B) computes A @ B.T (not A @ B!)
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])
result = np.inner(A, B)
print(result)
# [[1*5+2*6, 1*7+2*8],    = [[17, 23],
#   [3*5+4*6, 3*7+4*8]]    [39, 53]]
print("Comparison with A @ B.T:")
print(A @ B.T)
```

```
32
[[17 23]
 [39 53]]
Comparison with A @ B.T:
[[17 23]
 [39 53]]
```

14.11 The `numpy.outer` Function - Outer Product

The `outer` function computes the outer product of two vectors:

```
import numpy as np
# Outer product
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
result = np.outer(a, b)
print(result)
# Result:
# [[ 4  5  6]
```

```
# [ 8 10 12]
# [12 15 18]]
```

```
[[ 4  5  6]
 [ 8 10 12]
 [12 15 18]]
```

14.12 The `numpy.matmul` Function - Matrix Multiplication

The `matmul` function (and the equivalent `@` operator) is the recommended way to multiply matrices. For 2D arrays, it works identically to `dot`. Differences appear for 3D+ arrays (`matmul` treats them as stacks of matrices) and for scalars (`matmul` does not accept scalars, `dot` does).

```
import numpy as np

# For 2D -- identical results
a = np.array([[1, 2], [3, 4]])
b = np.array([[5, 6], [7, 8]])

print("dot:", np.dot(a, b)[0])
print("matmul:", np.matmul(a, b)[0])

# Difference: matmul does not accept scalars
try:
    np.matmul(a, 3)
except ValueError as e:
    print("matmul with scalar:", e)

print("dot with scalar:", np.dot(a, 3))
```

```
dot: [19 22]
```

```
matmul: [19 22]
```

```
matmul with scalar: matmul: Input operand 1 does not have enough dimensions (has 0, gufunc c
```

```
dot with scalar: [[ 3  6]
```

```
 [ 9 12]]
```

15 Advanced Filtering

15.1 The `nonzero()` Function

Returns the indices of non-zero elements in an array. The result is returned as a tuple of arrays, one for each dimension of the array.

```
import numpy as np

arr = np.array([[3, 0, 0], [0, 4, 0], [5, 6, 0]])
indices = np.nonzero(arr)
print(indices)  # (array([0, 1, 2, 2]), array([0, 1, 0, 1]))

# Extracting non-zero values
values = arr[indices]
print(values)  # [3 4 5 6]

# Alternatively, you can use:
position_indices = np.argwhere(arr != 0)
print(position_indices)
# [[0 0]
#   [1 1]
#   [2 0]
#   [2 1]]
```

```
(array([0, 1, 2, 2]), array([0, 1, 0, 1]))
[3 4 5 6]
[[0 0]
 [1 1]
 [2 0]
 [2 1]]
```

The `np.nonzero()` function returns the indices of all non-zero elements in an array. The result is a tuple of arrays, one per dimension: the first array contains the row indices and the second contains the column indices. For example, given `arr = np.array([[3, 0, 0], [0, 4, 0],`

[5, 6, 0]]), calling `np.nonzero(arr)` returns (`array([0, 1, 2, 2])`, `array([0, 1, 0, 1])`). This means the non-zero elements are located at positions (0,0), (1,1), (2,0), and (2,1), corresponding to the values 3, 4, 5, and 6 respectively.

15.2 The `where()` Function

Returns elements chosen from `x` or `y` depending on a condition. It works like a conditional element selection (equivalent to a ternary expression).

```
import numpy as np

# Replace non-positive values with 0 (zero effectively unchanged)
arr = np.array([1, -2, 3, -4, 5])
result = np.where(arr > 0, arr, 0)
print(result)  # [1 0 3 0 5]

# In a 2D array -- replace negative values with -1
arr_2d = np.array([[1, -2, 3], [-4, 5, -6]])
result_2d = np.where(arr_2d < 0, -1, arr_2d)
print(result_2d)
# [[ 1 -1  3]
#  [-1  5 -1]]
```

```
[1 0 3 0 5]
[[ 1 -1  3]
 [-1  5 -1]]
```

The `np.where(condition, x, y)` function takes three arguments: a condition, a value to use when the condition is true, and a value to use when the condition is false. For each element, it evaluates the condition and returns the corresponding value from `x` if the result is true, or from `y` if the result is false. It acts as an element-wise ternary operator for NumPy arrays.

15.3 The `indices()` and `ix_()` Functions

15.3.1 `indices()`

Creates an array representing grid indices.

```
import numpy as np

# Creating a 3x4 index grid
grid = np.indices((3, 4))
print(grid.shape)  # (2, 3, 4)
print(grid[0])  # row indices
# [[0 0 0 0]
#   [1 1 1 1]
#   [2 2 2 2]]
print(grid[1])  # column indices
# [[0 1 2 3]
#   [0 1 2 3]
#   [0 1 2 3]]
```

```
(2, 3, 4)
[[0 0 0 0]
 [1 1 1 1]
 [2 2 2 2]]
[[0 1 2 3]
 [0 1 2 3]
 [0 1 2 3]]
```

The `np.indices()` function generates an array representing the index grid for a given shape. For a 3×4 grid, `np.indices((3, 4))` returns a result of shape (2, 3, 4), where the first component (`grid[0]`) contains the row indices and the second component (`grid[1]`) contains the column indices. In `grid[0]`, each row is filled with its own row number (all zeros in the first row, all ones in the second, all twos in the third), while in `grid[1]`, each row contains the sequence of column numbers 0, 1, 2, 3 repeated identically across all rows. Together, these two arrays allow you to reference every position in the grid by its (row, column) pair.

15.3.2 `ix_()`

Constructs an open mesh from multiple sequences, which is useful for multidimensional indexing.

```
import numpy as np
x = np.array([0, 1, 2])
y = np.array([3, 4, 5, 6])
indices = np.ix_(x, y)

# Creates an open index grid for broadcasting
```

```

print(indices[0].shape, indices[1].shape) # (3, 1) (1, 4)

# Using it to select a subarray
arr = np.arange(16).reshape(4, 4)
print(arr)
# [[ 0  1  2  3]
#   [ 4  5  6  7]
#   [ 8  9 10 11]
#   [12 13 14 15]]

subarray = arr[np.ix_([0, 2, 3], [0, 2])]
print(subarray)
# [[ 0  2]
#   [ 8 10]
#   [12 14]]

```

```

(3, 1) (1, 4)
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]
 [12 13 14 15]]
[[ 0  2]
 [ 8 10]
 [12 14]]

```

15.4 ogridd and Grid Operations

ogrid allows creating open grids, which is more memory-efficient than full grids.

```

import numpy as np

# Grid of points in the range from -2 to 2 with step 0.1
x, y = np.ogrid[-2:2:0.1, -2:2:0.1]
mask = x**2 + y**2 <= 1 # Circle with radius 1
print(mask.shape) # (40, 40)

```

```

(40, 40)

```

15.5 The `ravel_multi_index()` and `unravel_index()` Functions

These functions convert between multidimensional and flat indices.

```
import numpy as np

# Converting multidimensional indices to flat indices
multi_indices = np.array([[0, 0], [1, 1], [2, 1]])
dimensions = (3, 3)
flat_indices = np.ravel_multi_index(multi_indices.T, dimensions)
print(flat_indices)  # [0 4 7]

# Converting flat indices to multidimensional indices
flat_indices = np.array([0, 3, 8])
shape = (3, 3)
multi_indices = np.unravel_index(flat_indices, shape)
print(multi_indices)  # (array([0, 1, 2]), array([0, 0, 2]))
```

```
[0 4 7]
(array([0, 1, 2]), array([0, 0, 2]))
```

15.6 Diagonal Indices

NumPy offers many functions for working with matrix diagonals.

```
import numpy as np

# Getting the main diagonal indices
n = 4
diag_indices = np.diag_indices(n)
print(diag_indices)  # (array([0, 1, 2, 3]), array([0, 1, 2, 3]))

# Using them to set the main diagonal
arr = np.zeros((4, 4))
arr[diag_indices] = 1  # Setting ones on the main diagonal
print(arr)
# [[1. 0. 0. 0.]
#  [0. 1. 0. 0.]
#  [0. 0. 1. 0.]
#  [0. 0. 0. 1.]]
```



```
# Getting indices from an existing array
arr2 = np.ones((3, 3))
diag_indices2 = np.diag_indices_from(arr2)
print(diag_indices2)
```

```
(array([0, 1, 2, 3]), array([0, 1, 2, 3]))
[[1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 0. 1.]]
(array([0, 1, 2]), array([0, 1, 2]))
```

15.7 The take() Function

Retrieves elements from an array along a specified axis based on indices.

```
import numpy as np

arr = np.array([10, 20, 30, 40, 50])
indices = np.array([0, 2, 4])
result = np.take(arr, indices)
print(result) # [10 30 50]

# In multidimensional arrays, we can choose the axis
arr_2d = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
row_indices = np.array([0, 2])
result_2d = np.take(arr_2d, row_indices, axis=0)
print(result_2d)
# [[1 2 3]
#   [7 8 9]]
```

```
[10 30 50]
[[1 2 3]
 [7 8 9]]
```

15.8 The take_along_axis() Function

Retrieves values from an array along a specified axis, using an index array with a compatible shape. Useful, for example, for applying results from `argsort()` or `argmin()`.

```
import numpy as np

arr = np.array([[10, 30, 20], [60, 40, 50]])
sort_indices = np.argsort(arr, axis=1)
result = np.take_along_axis(arr, sort_indices, axis=1)
print(result)
# [[10 20 30]
#   [40 50 60]]
```

```
[[10 20 30]
 [40 50 60]]
```

15.9 The choose() Function

Constructs an array by selecting elements from a list of arrays.

```
import numpy as np

choices = [np.array([0, 1, 2, 3]),
            np.array([10, 11, 12, 13]),
            np.array([20, 21, 22, 23])]
indices = np.array([0, 2, 1, 0]) # Selects which choice array to take the element from
result = np.choose(indices, choices)
print(result) # [ 0 21 12  3]
```

```
[ 0 21 12  3]
```

15.10 The compress() Function

Returns selected elements of an array along a specified axis.

```
import numpy as np
arr = np.array([[1, 2, 3], [4, 5, 6]])
mask = np.array([True, False, True])
result = np.compress(mask, arr, axis=1)
print(result)
# [[1 3]
#   [4 6]]
```

```
[[1 3]
 [4 6]]
```

15.11 The `diag()` and `diagonal()` Functions

Functions for working with diagonals.

```
import numpy as np

# Creating a diagonal array
diag_arr = np.diag([1, 2, 3, 4])
print(diag_arr)
# [[1 0 0 0]
#   [0 2 0 0]
#   [0 0 3 0]
#   [0 0 0 4]]

# Extracting the diagonal from an array
arr = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
diag = np.diag(arr)
print(diag) # [1 5 9]

# Extracting a diagonal shifted by 1
diag_offset = np.diagonal(arr, offset=1)
print(diag_offset) # [2 6]
```

```
[[1 0 0 0]
 [0 2 0 0]
 [0 0 3 0]
 [0 0 0 4]]
[1 5 9]
[2 6]
```

15.12 The `select()` Function

Returns an array built from elements of a list of choices, depending on conditions.

```
import numpy as np

arr = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
conditions = [arr < 3, arr < 6, arr < 9]
choices = [100, 200, 300]
result = np.select(conditions, choices, default=400)
print(result)
# [[100 100 200]
#   [200 200 300]
#   [300 300 400]]
```

```
[[100 100 200]
 [200 200 300]
 [300 300 400]]
```

15.13 The place() Function

Changes elements of an array based on a mask and provided values.

```
import numpy as np

arr = np.arange(5)
mask = np.array([True, False, True, False, True])
np.place(arr, mask, [-1, -2, -3]) # Cyclically uses values [-1, -2, -3]
print(arr) # [-1  1 -2  3 -3]
```

```
[-1  1 -2  3 -3]
```

15.14 The put() Function

Replaces specified elements of an array with given values.

```
import numpy as np

arr = np.arange(5)
indices = [0, 2, 4]
np.put(arr, indices, [10, 20, 30])
print(arr) # [10  1 20  3 30]
```

```
[10  1 20  3 30]
```

15.15 The `put_along_axis()` Function

Inserts values into a target array along a specified axis, using an index array with a compatible shape.

```
import numpy as np

arr = np.array([[10, 30, 20], [60, 40, 50]])
indices = np.argmax(arr, axis=1)
indices = np.expand_dims(indices, axis=1) # Reshape to shape (2, 1)
np.put_along_axis(arr, indices, 99, axis=1)
print(arr)
# [[99 30 20]
#   [60 99 50]]
```

```
[[99 30 20]
 [60 99 50]]
```

15.16 The `putmask()` Function

Changes elements of an array based on a mask. Unlike `place()`, values are assigned positionally (with cyclic extension to the array size), not sequentially to elements satisfying the mask.

```
import numpy as np

arr = np.arange(5)
mask = np.array([True, False, True, False, True])
np.putmask(arr, mask, [-1, -2, -3]) # Values assigned positionally (cyclically)
print(arr) # [-1  1 -3  3 -2]
```

```
[-1  1 -3  3 -2]
```

15.17 The `fill_diagonal()` Function

Fills the main diagonal of an array with a given value.

```

import numpy as np

arr = np.zeros((4, 4))
np.fill_diagonal(arr, 5)
print(arr)
# [[5. 0. 0. 0.]
#  [0. 5. 0. 0.]
#  [0. 0. 5. 0.]
#  [0. 0. 0. 5.]]

arr_rect = np.zeros((4, 4, 4))
np.fill_diagonal(arr_rect, 9)
print(arr_rect[0]) # Only element [0][0] set to 9 (super-diagonal i,i,i)

```

```

[[5. 0. 0. 0.]
 [0. 5. 0. 0.]
 [0. 0. 5. 0.]
 [0. 0. 0. 5.]]
[[9. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]]

```

16 NumPy - Miscellaneous

16.1 Constants

NumPy provides several well-known mathematical constants that can be useful in scientific and engineering computations. Built-in constants such as the number Pi or the base of the natural logarithm `e` make it easier to write readable and concise code.

1. `numpy.pi`

- Represents the number Pi (π) with high precision.
- Pi is the ratio of a circle's circumference to its diameter.
- Approximately: 3.141592653589793

2. `numpy.e`

- Represents the base of the natural logarithm, `e`.
- `e` is used in many fields, such as mathematical analysis, probability theory, and statistics.
- Approximately: 2.718281828459045

3. `numpy.eulergamma`

- Represents the Euler-Mascheroni constant, usually denoted as γ (gamma).
- It appears in mathematical analysis, particularly in number theory and the study of harmonic series.
- Approximately: 0.5772156649015329

```
import numpy as np

# Circle radius
r = 5.0

# Circle circumference: 2 * pi * r
circumference = 2 * np.pi * r
```

```
print("Circle circumference:", circumference)

# Circle area:    * r^2
area = np.pi * r**2
print("Circle area:", area)
```

```
Circle circumference: 31.41592653589793
Circle area: 78.53981633974483
```

```
import numpy as np

# Sample point x
x = 1.0

# Value of the function e^x
exp_value = np.e**x
print("e^x for x=1:", exp_value)

# Comparison with the np.exp function
exp_compare = np.exp(x)
print("Comparison with np.exp(1):", exp_compare)
```

```
e^x for x=1: 2.718281828459045
Comparison with np.exp(1): 2.718281828459045
```

16.2 numpy.inf

- **Description:** `np.inf` represents an infinite value (∞).
- It often appears in computations when the value of an expression tends to infinity (e.g., division by zero, certain limits, etc.).
- For example, `1.0 / 0.0` will return a warning and may consequently yield the value `inf`.

```
import numpy as np

# Using it to create boolean masks
arr = np.array([1, 2, np.inf, 4, 5])
mask = np.isinf(arr)
print("Mask of elements with value inf:", mask)
```


Mask of elements with value inf: [False False True False False]

16.3 numpy.nan

- **Description:** `np.nan` stands for “Not a Number” (NaN), meaning an undefined or unrepresentable value in the number system.
- It appears when the result of a numerical operation is undefined, e.g., `0.0/0.0`, `inf - inf`, or when data loading errors occur.
- Arithmetic operations with `nan` typically also return `nan`.

```
import numpy as np

# Replacing nan values in an array
data = np.array([1, 2, np.nan, 4, np.nan])
print("Original data:", data)

# Filling nan values with zero
data_no_nan = np.nan_to_num(data, nan=0.0)
print("Data without nan:", data_no_nan)
```

Original data: [1. 2. nan 4. nan]
Data without nan: [1. 2. 0. 4. 0.]

16.4 numpy.newaxis

- **Description:** `np.newaxis` is a special “constant”/object used to change array dimensions by increasing the number of dimensions by 1.

```
import numpy as np

# We have a 1D array
vec = np.array([1, 2, 3, 4])
print("Original array:", vec, "Shape:", vec.shape)

# Adding a new dimension as the row dimension
vec_as_col = vec[:, np.newaxis]
print("Array as column:\n", vec_as_col, "Shape:", vec_as_col.shape)
```

```
# Adding a dimension at the beginning
vec_as_row = vec[np.newaxis, :]
print("Array as row:\n", vec_as_row, "Shape:", vec_as_row.shape)

# Another example: adding a dimension to easily broadcast operations
a = np.array([10, 20, 30])
b = np.array([1, 2])
# Without a new dimension, trying to add a to b would fail,
# because the shapes are not compatible.
# With a new dimension, a has shape (3,1) and b (2,), which allows broadcasting
sum_matrix = a[:, np.newaxis] + b
print("Operation with broadcasting:\n", sum_matrix)
```

Original array: [1 2 3 4] Shape: (4,)

Array as column:

```
[[1]
 [2]
 [3]
 [4]] Shape: (4, 1)
```

Array as row:

```
[[1 2 3 4]] Shape: (1, 4)
```

Operation with broadcasting:

```
[[11 12]
 [21 22]
 [31 32]]
```

16.5 Statistics and Aggregation

Function	Description
np.mean	Mean of all values in the array.
np.std	Standard deviation.
np.var	Variance.
np.sum	Sum of all elements.
np.prod	Product of all elements.
np.cumsum	Cumulative sum of all elements.
np.cumprod	Cumulative product of all elements.
np.min, np.max	Minimum/maximum value in the array.
np.argmin, np.argmax	Index of the minimum/maximum value in the array.
np.all	Checks whether all elements are non-zero.

Function	Description
<code>np.any</code>	Checks whether at least one element is non-zero.

Part III

Data exploration

17 Stages of data exploration

- **Data collection:**
 - Gathering data from various sources (databases, CSV files, APIs, etc.).
- **Data understanding:**
 - Analysis of data structure, data types, and their meaning.
 - Exploration of preliminary relationships and trends.
- **Data cleaning:**
 - Removing missing values, errors, and anomalies in the data.
 - Handling missing values and duplicates.
- **Data transformation:**
 - Normalization, standardization, encoding of categorical variables.
 - Creating new variables (features).
- **Data reduction:**
 - Selection of relevant features or reducing data dimensionality (e.g., PCA).

Part IV

Pandas

18 Pandas - getting started

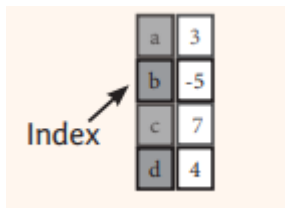
Pandas is a Python library used for data analysis and manipulation

18.1 Import:

```
import pandas as pd
```

18.2 Basic objects

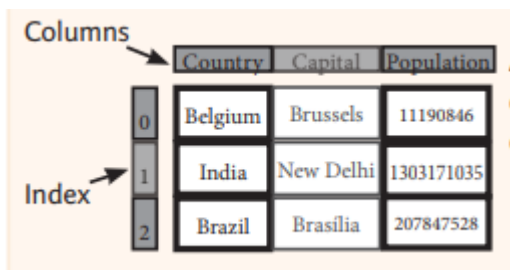
Series



A diagram illustrating a Pandas Series. It consists of a vertical column of four cells. The first cell contains 'a' and the second '3'. The third cell contains 'b' and the fourth '-5'. The fifth cell contains 'c' and the sixth '7'. The seventh cell contains 'd' and the eighth '4'. An arrow labeled 'Index' points to the first cell ('a').

a	3
b	-5
c	7
d	4

DataFrame



A diagram illustrating a Pandas DataFrame. It is a table with three columns and three rows. The columns are labeled 'Country', 'Capital', and 'Population'. The rows are indexed 0, 1, and 2. An arrow labeled 'Columns' points to the 'Country' header. An arrow labeled 'Index' points to the index value '0'.

	Country	Capital	Population
0	Belgium	Brussels	11190846
1	India	New Delhi	1303171035
2	Brazil	Brasilia	207847528

```
import pandas as pd

s = pd.Series([3, -5, 7, 4])
print(s)
print("values")
print(s.to_numpy())
print(type(s.to_numpy()))
print(s.index)
print(type(s.index))
```

```
0    3
1   -5
2    7
3    4
dtype: int64
values
[ 3 -5  7  4]
<class 'numpy.ndarray'>
RangeIndex(start=0, stop=4, step=1)
<class 'pandas.RangeIndex'>
```

```
import pandas as pd
import numpy as np

s = pd.Series([3, -5, 7, 4], index=['a', 'b', 'c', 'd'])
print(s)
print(s['b'])
s['b'] = 8
print(s)
print(s[s > 5])
print(s * 2)
print(np.sin(s))
```

```
a    3
b   -5
c    7
d    4
dtype: int64
-5
a    3
b    8
```



```

c      7
d      4
dtype: int64
b      8
c      7
dtype: int64
a      6
b     16
c     14
d      8
dtype: int64
a     0.141120
b     0.989358
c     0.656987
d    -0.756802
dtype: float64

```

```

import pandas as pd

d = {'key1': 350, 'key2': 700, 'key3': 70}
s = pd.Series(d)
print(s)

```

```

key1      350
key2      700
key3       70
dtype: int64

```

```

import pandas as pd

d = {'key1': 350, 'key2': 700, 'key3': 70}
k = ['key0', 'key2', 'key3', 'key1']
s = pd.Series(d, index=k)
print(s)
s.name = "Value"
s.index.name = "Key"
print(s)

```

```

key0      NaN
key2     700.0
key3     70.0

```

```

key1    350.0
dtype: float64
Key
key0     NaN
key2    700.0
key3     70.0
key1    350.0
Name: Value, dtype: float64

```

```

import pandas as pd

data = {'Country': ['Belgium', 'India', 'Brazil'],
        'Capital': ['Brussels', 'New Delhi', 'Brasília'],
        'Population': [11190846, 1303171035, 207847528]}
frame = pd.DataFrame(data)
print(frame)
data2 = pd.DataFrame(data, columns=['Country', 'Population', 'Capital'])
print(data2)

```

	Country	Capital	Population
0	Belgium	Brussels	11190846
1	India	New Delhi	1303171035
2	Brazil	Brasília	207847528

	Country	Population	Capital
0	Belgium	11190846	Brussels
1	India	1303171035	New Delhi
2	Brazil	207847528	Brasília

```

import pandas as pd

data = {'Country': ['Belgium', 'India', 'Brazil'],
        'Capital': ['Brussels', 'New Delhi', 'Brasília'],
        'Population': [11190846, 1303171035, 207847528]}
df_data = pd.DataFrame(data, columns=['Country', 'Population', 'Capital'])
print("Shape:", df_data.shape)
print("---")
print("Index:", df_data.index)
print("---")
print("columns:", df_data.columns)
print("---")
df_data.info()

```

```
print("--")
print(df_data.count())
```

Shape: (3, 3)

--

Index: RangeIndex(start=0, stop=3, step=1)

--

columns: Index(['Country', 'Population', 'Capital'], dtype='str')

--

<class 'pandas.DataFrame'>

RangeIndex: 3 entries, 0 to 2

Data columns (total 3 columns):

#	Column	Non-Null Count	Dtype
0	Country	3 non-null	str
1	Population	3 non-null	int64
2	Capital	3 non-null	str

dtypes: int64(1), str(2)

memory usage: 204.0 bytes

--

Country 3

Population 3

Capital 3

dtype: int64

19 Pandas - indexing

```
import pandas as pd

data = {'Country': ['Belgium', 'India', 'Brazil'],
        'Capital': ['Brussels', 'New Delhi', 'Brasília'],
        'Population': [11190846, 1303171035, 207847528]}
data2 = pd.DataFrame(data, columns=['Country', 'Population', 'Capital'])
print(data2.iloc[[0], [0]])
print("--")
print(data2.loc[[0], ['Country']])
print("--")
print(data2.loc[2])
print("--")
print(data2.loc[:, 'Capital'])
print("--")
print(data2.loc[1, 'Capital'])
```

```
Country
0 Belgium
--
Country
0 Belgium
--
Country      Brazil
Population    207847528
Capital      Brasília
Name: 2, dtype: object
--
0    Brussels
1    New Delhi
2    Brasília
Name: Capital, dtype: str
--
New Delhi
```

1. loc:

- This is a label-based indexing method, meaning it uses column label names and row indices to select data.
- It works based on index labels and column labels, which allows for more convenient data filtering.
- It supports both individual labels and label ranges.
- It also works with non-numeric labels.
- Example usage: `df.loc[1:3, ['A', 'B']]` - returns rows from index 1 to 3 (inclusive) and columns 'A' and 'B'.

2. iloc:

- This is a position-based indexing method, meaning it uses numeric column and row indices to select data.
- It works based on numeric indices for both rows and columns.
- It supports individual indices and index ranges.
- When using index ranges, the range is half-open, meaning the right end is not included.
- Example usage: `df.iloc[1:3, 0:2]` - returns rows from index 1 to 3 (excluding 3) and columns from index 0 to 2 (excluding 2).

```
import pandas as pd

data = {'Country': ['Belgium', 'India', 'Brazil'],
        'Capital': ['Brussels', 'New Delhi', 'Brasília'],
        'Population': [11190846, 1303171035, 207847528]}
data2 = pd.DataFrame(data, columns=['Country', 'Population', 'Capital'])
print(data2['Population'])
print("---")
print(data2[data2['Population'] > 1200000000])
print("---")
```

```
0      11190846
1    1303171035
2     207847528
Name: Population, dtype: int64
--
   Country  Population   Capital
1   India  1303171035  New Delhi
--
```

Exercises: (ex9.py)

Practice indexing on the data frame below (you don't need to use all rows):

- **Categorical columns:**
 - **Region** – sales region
 - **Product** – product type
 - **Sales_Channel** – sales channel (online, retail store, wholesale)
- **Numeric columns:**
 - **Units_Sold** – number of units sold
 - **Revenue** – revenue in thousands of GBP
 - **Profit** – profit in thousands of GBP

Region	Product	Sales_Channel	Units_Sold	Revenue	Profit
North	Electronics	Online	120	60.5	15.2
South	Furniture	Retail	80	45.0	12.0
East	Clothing	Online	200	35.0	8.5
West	Electronics	Wholesale	150	70.0	20.5
North	Furniture	Retail	90	50.5	13.2
South	Clothing	Online	300	55.0	10.0
East	Electronics	Retail	110	62.0	16.0
West	Furniture	Online	70	30.0	7.5
North	Clothing	Wholesale	250	40.0	9.0
South	Electronics	Retail	130	75.0	22.0

20 Loading data

20.1 Handling csv files

The `pandas.read_csv` function

Documentation: [link](#)

Selected arguments:

- `filepath` - access path
- `sep=_NoDefault.no_default, delimiter=None` - separator
- `header='infer'` - header - by default the column names, alternatively `header=None` means no header
- `index_col=None` - sets a column as the index (row names)
- `thousands=None` - thousands separator
- `decimal='.'` - decimal separator

Saving with `pandas.DataFrame.to_csv`

Documentation: [link](#)

20.2 Handling Excel files

The `pandas.read_excel` function

https://pandas.pydata.org/docs/reference/api/pandas.read_excel.html

**** Important:** you need to install the `openpyxl` library to import `.xlsx` files and `xlrd` to import `.xls` files (in most cases you don't need to import them explicitly in the code)

Selected arguments:

- `io` - access path
- `sheet_name=0` - sheet name
- `header='infer'` - header - by default the column names, alternatively `header=None` means no header
- `index_col=None` - sets a column as the index (row names)
- `thousands=None` - thousands separator

- `decimal='.'` - decimal separator

20.3 Handling sqlite3

```
import pandas as pd
from sqlite3 import connect

conn = connect('sales_data2.db')
data = pd.read_sql("SELECT * FROM sales_data", con=conn)
```

20.4 Sample files for exercises

<https://github.com/pjastr/DAV-files>

21 Pandas - sorting

```
import pandas as pd

# Sample data frame
data = pd.DataFrame({
    'Name': ['Alice', 'Tom', 'Charlie'],
    'Age': [25, 42, 35],
    'Salary': [50000, 60000, 70000]
})

# Sorting by the 'Age' column
s1 = data.sort_values(by='Age')
print(s1)

# Sorting in reverse order
s2 = data.sort_values(by='Salary', ascending=False)
print(s2)

# Sorting by 'Age' ascending, then 'Salary' descending
s3 = data.sort_values(by=['Age', 'Salary'], ascending=[True, False])
print(s3)
```

	Name	Age	Salary
0	Alice	25	50000
2	Charlie	35	70000
1	Tom	42	60000

	Name	Age	Salary
2	Charlie	35	70000
1	Tom	42	60000
0	Alice	25	50000

	Name	Age	Salary
0	Alice	25	50000
2	Charlie	35	70000
1	Tom	42	60000

```
import pandas as pd

# Sample data frame
data = pd.DataFrame({
    'Name': ['Alice', 'Tom', 'Charlie'],
    'Age': [25, 41, 35],
    'Salary': [50000, 60000, 70000]
})

# Inplace sorting (replacing the existing variable) - currently not recommended
data.sort_values(by='Age', inplace=True)
print(data)
```

	Name	Age	Salary
0	Alice	25	50000
2	Charlie	35	70000
1	Tom	41	60000

```
import pandas as pd

df2 = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Charlie', 'Dave'],
    'Age': [25, 30, None, 35],
    'Salary': [50000, None, 70000, 60000]
})

# Sorting with NaN at the end
s2 = df2.sort_values(by='Age', na_position='last')
print(s2)

# Sorting with NaN at the beginning
s3 = df2.sort_values(by='Age', na_position='first')
print(s3)
```

	Name	Age	Salary
0	Alice	25.0	50000.0
1	Bob	30.0	NaN
3	Dave	35.0	60000.0
2	Charlie	NaN	70000.0

	Name	Age	Salary
2	Charlie	NaN	70000.0
0	Alice	25.0	50000.0

1	Bob	30.0	NaN
3	Dave	35.0	60000.0

```
import pandas as pd

df3 = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Charlie', 'Dave'],
    'Age': [25, 30, None, 35],
    'Salary': [50000, None, 70000, 60000]
})

s3 = df3.sort_values(by='Name', key=lambda x: x.str.len())
print(s3)
```

	Name	Age	Salary
1	Bob	30.0	NaN
3	Dave	35.0	60000.0
0	Alice	25.0	50000.0
2	Charlie	NaN	70000.0

22 Pandas - time series

Converting a string to `datetime` format (date-time)

```
import pandas as pd

data = {'date': ['2023-01-01', '2023-01-02', '2023-01-03'], 'value': [10, 15, 20]}
data_frame = pd.DataFrame(data)
data_frame['date'] = pd.to_datetime(data_frame['date'])
print(data)
```

```
{'date': ['2023-01-01', '2023-01-02', '2023-01-03'], 'value': [10, 15, 20]}
```

The `errors` argument in the `pd.to_datetime` function controls how the function should behave when it encounters invalid data while attempting to convert values into `datetime` objects. Possible values for `errors` are:

1. **'raise'** (default): Raises an exception if it encounters an invalid data format.
2. **'coerce'**: Replaces invalid values with `NaT` (Not a Time).
3. **'ignore'**: Returns the input data unchanged when it encounters an error (option deprecated in later versions).

Code to paste into the environment:

```
import pandas as pd

data = {'date': ['2023-01-01', 'invalid', '2023-01-03'], 'value': [10, 15, 20]}
data_frame = pd.DataFrame(data)
data_frame['date'] = pd.to_datetime(data_frame['date'], errors='ignore')
```

The `format` argument in the `pandas.to_datetime` function allows you to specify the exact date and time format to be used for parsing input values. This is useful when input data has a fixed, specific format, which can speed up processing and reduce the risk of misinterpreting dates.

```

import pandas as pd

# Sample input data with different formats
data1 = ['01-01-2025', '15-03-2025', '30-12-2025'] # Format: DD-MM-YYYY
data2 = ['2025/01/01', '2025/03/15', '2025/12/30'] # Format: YYYY/MM/DD

# Conversion with a specified format (DD-MM-YYYY)
df1 = pd.DataFrame(data1)
df1[0] = pd.to_datetime(df1[0], format='%d-%m-%Y')
print("Conversion with format '%d-%m-%Y':")
print(df1)

# Conversion with a specified format (YYYY/MM/DD)
df2 = pd.DataFrame(data2)
df2[0] = pd.to_datetime(df2[0], format='%Y/%m/%d')
print("\nConversion with format '%Y/%m/%d':")
print(df2)

```

Conversion with format '%d-%m-%Y':

```

0
0 2025-01-01
1 2025-03-15
2 2025-12-30

```

Conversion with format '%Y/%m/%d':

```

0
0 2025-01-01
1 2025-03-15
2 2025-12-30

```

Format code	Description	Example
%Y	Year in 4-digit format	2025
%y	Year in 2-digit format	25
%m	Month (digits, 2-digit)	01 (January), 12 (December)
%d	Day of the month (2-digit)	01, 15, 31
%B	Full month name	January, December
%b	Abbreviated month name	Jan, Dec
%A	Full weekday name	Monday, Sunday
%a	Abbreviated weekday name	Mon, Sun

Format code	Description	Example
%H	Hour in 24-hour format	00, 12, 23
%I	Hour in 12-hour format	01, 11
%p	AM/PM	AM, PM
%M	Minutes	00, 30, 59
%S	Seconds	00, 30, 59

Polish month names in nominative or abbreviated form.

Note: That isn't grammatically correct. Traditional format (DMY): dd.mm.yyyy, often with dots as separators; the more official form is d yyyy, or, less frequently, d yyyy. Official format (YMD): the ISO 8601 YYYY-MM-DD format is used in Poland in official documents, banks, computer systems, and on the internet.

```
import locale
import pandas as pd

locale.setlocale(locale.LC_ALL, 'PL')
# locale.setlocale(locale.LC_TIME, 'pl_PL.UTF-8') # On Linux/Mac systems
# locale.setlocale(locale.LC_TIME, 'Polish_Poland.1250') # On Windows

data = ['10 styczeń 2025', '15 grudzień 2025', '5 marzec 2025']
data_frame = pd.DataFrame(data)
data_frame[0] = pd.to_datetime(data_frame[0], format='%d %B %Y')
print(data_frame)

data2 = ['10 sty 2025', '15 gru 2025', '5 mar 2025']
df2 = pd.DataFrame(data2)
df2[0] = pd.to_datetime(df2[0], format='%d %b %Y')
print(df2)
```

```

0
0 2025-01-10
1 2025-12-15
2 2025-03-05
0
0 2025-01-10
1 2025-12-15
2 2025-03-05
```

Exercise: (extime.py)

Load the following files and transform the date column:

- date_sale.csv
- date_temp.csv

Hint:

```
import pandas as pd  
data = pd.read_csv("date_sale.csv", parse_dates=["Sale_Date"], date_format="%d-%m-%Y")
```

23 Pandas - regular expressions

23.1 What are regular expressions?

Regular expressions (regex) are text patterns used for searching, matching, and manipulating strings. In Python, they are supported by the built-in `re` module, and Pandas integrates them directly into operations on text columns through the `.str` accessor.

Before a regular expression, we always use the `r` prefix (raw string) so that the backslash `\` is not interpreted as a Python escape character.

```
import re

text = "Contact us at info@example.com or sales@example.com"
emails = re.findall(r"[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}", text)
print(emails)
# ['info@example.com', 'sales@example.com']
```

```
['info@example.com', 'sales@example.com']
```

23.2 Basic regex syntax

Here are the most important elements of regular expression syntax:

Metacharacters:

Metacharacter	Meaning
.	Any character (except <code>\n</code>)
^	Start of string
\$	End of string
*	0 or more repetitions
+	1 or more repetitions
?	0 or 1 repetition
	Alternative (or)
()	Capturing group

Metacharacter	Meaning
[]	Character set
{}	Exact number of repetitions

Character classes:

Shortcut	Meaning	ASCII equivalent
\d	Digit	[0-9]
\D	Non-digit	[^0-9]
\w	“Word” character	[a-zA-Z0-9_]
\W	Non-word character	[^a-zA-Z0-9_]
\s	Whitespace	[\t\n\r\f\v]
\S	Non-whitespace	[^ \t\n\r\f\v]
\b	Word boundary	—

Quantifiers:

Pattern	Meaning
a{3}	Exactly 3 occurrences of a
a{2,5}	Between 2 and 5 occurrences of a
a{2,}	At least 2 occurrences of a
a*?	0 or more (lazy)
a+?	1 or more (lazy)

```
import re

text = "<b>bold</b> and <i>italic</i>"

# greedy - matches as much as possible
greedy = re.findall(r"<.+>", text)
print("Greedy:", greedy)

# lazy - matches as little as possible
lazy = re.findall(r"<.+?>", text)
print("Lazy:", lazy)
```

```
Greedy: ['<b>bold</b> and <i>italic</i>']
Lazy: ['<b>', '</b>', '<i>', '</i>']
```

23.3 Key functions of the re module

Here is a Markdown table explaining the key functions of the `re` module:

Function	Description
<code>re.search()</code>	Searches for the first match in the entire string. Returns a <code>Match</code> object or <code>None</code> .
<code>re.match()</code>	Searches for a match only at the beginning of the string.
<code>re.findall()</code>	Returns a list of all matches (as strings).
<code>re.finditer()</code>	Like <code>findall</code> , but returns an iterator of <code>Match</code> objects.
<code>re.sub()</code>	Replaces matches with a new string or the result of a function.
<code>re.split()</code>	Splits a string by a pattern (more flexible than <code>str.split()</code>).
<code>re.fullmatch()</code>	Checks whether the entire string matches the pattern — ideal for validation.
<code>re.compile()</code>	Compiles a pattern for repeated use — speeds up execution.

```
import re

text = "Temperature is 36.6 degrees"
match = re.search(r"\d+\.\d+", text)
if match:
    print(f"Found: {match.group()} at position {match.span()}")

log = """
ERROR 2025-03-06 10:15:32 Connection timeout
INFO  2025-03-06 10:15:33 Retry attempt
ERROR 2025-03-06 10:15:35 Disk full
"""

errors = re.findall(r"ERROR\s+\S+\s+\S+\s+(.+)", log)
print("Errors:", errors)
```

```
Found: 36.6 at position (15, 19)
Errors: ['Connection timeout', 'Disk full']
```

23.4 Groups and references

Parentheses create groups — they make it possible to extract fragments of a match. Groups can be numbered, named ((?P<name>...)), or non-capturing ((?:...)).

```
import re

# Numbered groups
text = "2025-03-06"
match = re.search(r"(\d{4})-(\d{2})-(\d{2})", text)
if match:
    print(f"Year: {match.group(1)}, Month: {match.group(2)}, Day: {match.group(3)}")

# Named groups
pattern = r"(?P<year>\d{4})-(?P<month>\d{2})-(?P<day>\d{2})"
match = re.search(pattern, "Meeting on 2025-03-06 at noon")
if match:
    print(match.groupdict())

# Non-capturing group
result = re.findall(r"(?:https?|ftp)://\S+", "Visit https://example.com")
print(result)
```

```
Year: 2025, Month: 03, Day: 06
{'year': '2025', 'month': '03', 'day': '06'}
['https://example.com']
```

23.5 Flags

Flags modify the behavior of regular expressions:

Flag	Shortcut	Action
re.IGNORECASE	re.I	Ignores letter case
re.MULTILINE	re.M	^/\$ apply to each line
re.DOTALL	re.S	. also matches \n
re.VERBOSE	re.X	Allows comments within the pattern

```

import re

# Combining flags with the | operator
text = "First Line\nSecond Line\nThird Line"
matches = re.findall(r"^\w+", text, re.MULTILINE | re.IGNORECASE)
print(matches)

# VERBOSE flag - readable patterns with comments
email_pattern = re.compile(r"""
    ^
    [a-zA-Z0-9._%+-]+      # username
    @                      # @ symbol
    [a-zA-Z0-9.-]+        # domain name
    \.                    # dot
    [a-zA-Z]{2,}          # domain extension
    $
""", re.VERBOSE)

print(email_pattern.fullmatch("user@example.com"))
print(email_pattern.fullmatch("invalid@mail"))

```

```

['First', 'Second', 'Third']
<re.Match object; span=(0, 16), match='user@example.com'>
None

```

23.6 Regular expressions in Pandas

Many `.str` accessor methods in Pandas support regular expressions via the `regex=True` parameter. This allows vectorized regex operations on entire columns.

```

import pandas as pd

data = pd.DataFrame({
    'Text': [' Hello World ', 'Pandas Library43', ' Data Science ']
})

# Removing special characters (regex in str.replace)
data['Clean'] = data['Text'].str.strip().str.replace(r'[\w\s]', '', regex=True)
print(data)

```

```
# Removing numbers
data['NoDigits'] = data['Clean'].str.replace(r'\d+', '', regex=True)
print(data)
```

	Text	Clean
0	Hello World	Hello World
1	Pandas Library43	Pandas Library43
2	Data Science	Data Science

	Text	Clean	NoDigits
0	Hello World	Hello World	Hello World
1	Pandas Library43	Pandas Library43	Pandas Library
2	Data Science	Data Science	Data Science

23.7 str.contains() — filtering with regex

The `str.contains()` method allows filtering rows based on a regex pattern.

```
import pandas as pd

df = pd.DataFrame({
    'email': ['jan@gmail.com', 'anna@wp.pl', 'piotr@firma.com', 'kasia@gmail.com', 'bad-email']
})

# Filtering emails from the gmail domain
gmail_users = df[df['email'].str.contains(r'@gmail\.com$', regex=True)]
print("Gmail:\n", gmail_users)

# Email format validation
df['valid'] = df['email'].str.contains(
    r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$', regex=True
)
print("\nValidation:\n", df)
```

```
Gmail:
      email
0  jan@gmail.com
3  kasia@gmail.com

Validation:
      email  valid
```

```

0    jan@gmail.com    True
1     anna@wp.pl     True
2  piotr@firma.com    True
3  kasia@gmail.com    True
4    bad-email@@     False

```

23.8 str.extract() — extracting groups

The `str.extract()` method extracts capturing groups from a regular expression into separate columns.

```

import pandas as pd

df = pd.DataFrame({
    'date_str': ['2025-03-06', '2024-12-25', '2023-01-15']
})

# Extracting year, month, and day into separate columns
extracted = df['date_str'].str.extract(r'(?P<year>\d{4})-(?P<month>\d{2})-(?P<day>\d{2})')
print(extracted)

```

```

      year month day
0  2025     03   06
1  2024     12   25
2  2023     01   15

```

23.9 str.extractall() — all matches

The `str.extractall()` method extracts **all** occurrences of a pattern (not just the first one).

```

import pandas as pd

df = pd.DataFrame({
    'text': ['Prices: 19.99 PLN and 5.50 PLN', 'Cost: 120.00 PLN', 'No prices']
})

# Extracting all amounts
amounts = df['text'].str.extractall(r'(\d+\.\d{2})')
print(amounts)

```

		0
match		
0	0	19.99
	1	5.50
1	0	120.00

23.10 str.replace() with regex — replacing patterns

The `regex=True` parameter in `str.replace()` enables replacements based on regular expressions.

```
import pandas as pd

df = pd.DataFrame({
    'phone': ['123-456-789', '987 654 321', '(48) 555-123-456']
})

# Removing everything except digits
df['digits_only'] = df['phone'].str.replace(r'[^\\d]', '', regex=True)
print(df)
```

	phone	digits_only
0	123-456-789	123456789
1	987 654 321	987654321
2	(48) 555-123-456	48555123456

23.11 str.findall() — list of matches

The `str.findall()` method returns a list of all matches in each row.

```
import pandas as pd

df = pd.DataFrame({
    'text': ['Hashtags: #python #pandas #regex', 'No tags here', '#data is fun #science']
})

# Extracting hashtags
df['hashtags'] = df['text'].str.findall(r'#[\\w\\+\\_]+')
print(df)
```

	text	hashtags
0	Hashtags: #python #pandas #regex	[#python, #pandas, #regex]
1	No tags here	[]
2	#data is fun #science	[#data, #science]

23.12 Polish characters in regex

Working with Polish characters requires explicitly including them in patterns or using Unicode classes.

```
import pandas as pd
import re

df = pd.DataFrame({
    'text': ['Zażółć gęślą jaźń', 'Hello World', 'Łódź jest piękna']
})

# Checking which rows contain Polish characters
polish_pattern = r'[ąćęłńóśźżĄĆĘŁŃÓŚŻŻ]'
df['has_polish'] = df['text'].str.contains(polish_pattern, regex=True)
print(df)
```

	text	has_polish
0	Zażółć gęślą jaźń	True
1	Hello World	False
2	Łódź jest piękna	True

```
import pandas as pd
import re

# Replacing Polish characters with Latin ones
def replace_polish(text):
    mapping = {
        "ą": "a", "ć": "c", "ę": "e", "ł": "l", "ń": "n",
        "ó": "o", "ś": "s", "ź": "z", "ż": "z",
        "Ą": "A", "Ć": "C", "Ę": "E", "Ł": "L", "Ń": "N",
        "Ó": "O", "Ś": "S", "Ź": "Z", "Ż": "Z"
    }
    pattern = r'[ąćęłńóśźżĄĆĘŁŃÓŚŻŻ]'
    return re.sub(pattern, lambda m: mapping.get(m.group(), m.group()), text)
```



```
df = pd.DataFrame({
    'text': ['Zażółć gęślą jaźń', 'Łódź', 'żółć']
})

df['ascii'] = df['text'].apply(replace_polish)
print(df)
```

	text	ascii
0	Zażółć gęślą jaźń	Zazolc gesla jazn
1	Łódź	Lodz
2	żółć	zolc

24 Pandas - other

24.1 Filling missing values

```
import pandas as pd

s = pd.Series([3, -5, 7, 4], index=['a', 'b', 'c', 'd'])
s2 = pd.Series([7, -2, 3], index=['a', 'c', 'd'])
print(s + s2)
print("--")
print(s.add(s2, fill_value=0))
print("--")
print(s.mul(s2, fill_value=2))
```

```
a    10.0
b     NaN
c     5.0
d     7.0
dtype: float64
--
a    10.0
b    -5.0
c     5.0
d     7.0
dtype: float64
--
a    21.0
b   -10.0
c   -14.0
d    12.0
dtype: float64
```

24.2 Handling missing data

```
import numpy as np
import pandas as pd

string_data = pd.Series(['aardvark', 'artichoke', np.nan, 'avocado'])
print(string_data)
print(string_data.isna())
print(string_data.dropna())
```

```
0    aardvark
1    artichoke
2         NaN
3     avocado
dtype: str
0    False
1    False
2     True
3    False
dtype: bool
0    aardvark
1    artichoke
3     avocado
dtype: str
```

```
from numpy import nan as NA
import pandas as pd

data = pd.DataFrame([[1., 6.5, 3.], [1., NA, NA],
                    [NA, NA, NA], [NA, 6.5, 3.]])
cleaned = data.dropna()
print(cleaned)
print(data.dropna(how='all'))
data[4] = NA
print(data.dropna(how='all', axis=1))
print(data)
print(data.fillna(0))
print(data.fillna({1: 0.5, 2: 0}))
```

	0	1	2	
0	1.0	6.5	3.0	
	0	1	2	
0	1.0	6.5	3.0	
1	1.0	NaN	NaN	
3	NaN	6.5	3.0	
	0	1	2	
0	1.0	6.5	3.0	
1	1.0	NaN	NaN	
2	NaN	NaN	NaN	
3	NaN	6.5	3.0	
	0	1	2	4
0	1.0	6.5	3.0	NaN
1	1.0	NaN	NaN	NaN
2	NaN	NaN	NaN	NaN
3	NaN	6.5	3.0	NaN
	0	1	2	4
0	1.0	6.5	3.0	0.0
1	1.0	0.0	0.0	0.0
2	0.0	0.0	0.0	0.0
3	0.0	6.5	3.0	0.0
	0	1	2	4
0	1.0	6.5	3.0	NaN
1	1.0	0.5	0.0	NaN
2	NaN	0.5	0.0	NaN
3	NaN	6.5	3.0	NaN

24.3 Removing duplicates

```
import pandas as pd

data = pd.DataFrame({'k1': ['one', 'two'] * 3 + ['two'],
                     'k2': [1, 1, 2, 3, 3, 4, 4]})

print(data)
print(data.duplicated())
print(data.drop_duplicates())
```

	k1	k2
0	one	1
1	two	1

```

2 one 2
3 two 3
4 one 3
5 two 4
6 two 4
0 False
1 False
2 False
3 False
4 False
5 False
6 True
dtype: bool
   k1 k2
0 one 1
1 two 1
2 one 2
3 two 3
4 one 3
5 two 4

```

24.4 Replacing values

```

import pandas as pd
import numpy as np

data = pd.Series([1., -999., 2., -999., -1000., 3.])
print(data)
print(data.replace(-999, np.nan))
print(data.replace([-999, -1000], np.nan))
print(data.replace([-999, -1000], [np.nan, 0]))
print(data.replace({-999: np.nan, -1000: 0}))

```

```

0      1.0
1    -999.0
2      2.0
3    -999.0
4   -1000.0
5      3.0
dtype: float64

```

```

0      1.0
1      NaN
2      2.0
3      NaN
4    -1000.0
5       3.0
dtype: float64
0      1.0
1      NaN
2      2.0
3      NaN
4      NaN
5      3.0
dtype: float64
0      1.0
1      NaN
2      2.0
3      NaN
4      0.0
5      3.0
dtype: float64
0      1.0
1      NaN
2      2.0
3      NaN
4      0.0
5      3.0
dtype: float64

```

24.5 Discretization and binning

```

import pandas as pd

ages = [20, 22, 25, 27, 21, 23, 37, 31, 61, 45, 41, 32]
bins = [18, 25, 35, 60, 100]
cats = pd.cut(ages, bins)
print(cats)
print(cats.codes)
print(cats.categories)
print(pd.Series(cats).value_counts())

```

```
import pandas as pd
import numpy as np

data = np.random.randn(1000)
cats = pd.qcut(data, 4)
print(cats)
print(pd.Series(cats).value_counts())
```

```
[(-0.597, 0.0607], (-2.963, -0.597], (0.748, 3.801], (-2.963, -0.597], (-
2.963, -0.597], ..., (0.748, 3.801], (-2.963, -0.597], (0.748, 3.801], (-
0.597, 0.0607], (0.0607, 0.748]]
Length: 1000
Categories (4, interval[float64, right]): [(-2.963, -0.597] < (-0.597, 0.0607] < (0.0607, 0.
(-2.963, -0.597]      250
(-0.597, 0.0607]      250
(0.0607, 0.748]       250
(0.748, 3.801]        250
Name: count, dtype: int64
```

24.6 Detecting and filtering outliers

```
import pandas as pd
import numpy as np

data = pd.DataFrame(np.random.randn(1000, 4))
print(data.describe())
print("----")
col = data[2]
print(col[np.abs(col) > 3])
print("----")
print(data[(np.abs(data) > 3).any(axis=1)])
```

	0	1	2	3
count	1000.000000	1000.000000	1000.000000	1000.000000
mean	-0.009741	-0.004792	0.010749	-0.000584
std	0.979630	1.031484	1.029102	1.015369
min	-4.014106	-3.741276	-2.917596	-3.191150

25%	-0.640090	-0.670298	-0.668171	-0.665036
50%	0.017729	-0.041524	0.051032	0.046523
75%	0.617048	0.675106	0.683567	0.700587
max	2.847210	3.457701	3.263269	3.143610

873 3.263269

Name: 2, dtype: float64

	0	1	2	3
23	-0.006278	-3.741276	-0.597937	0.091917
156	0.996089	3.036908	-0.436185	0.239618
222	-4.014106	1.002816	-1.522266	0.516670
324	0.269773	3.457701	1.053668	-1.016401
362	-0.555566	-1.357638	1.057790	3.143610
723	1.602654	3.025094	-2.254967	-0.591341
873	0.582272	-0.531854	3.263269	-0.604219
881	-0.993190	-2.591012	0.569693	-3.191150
956	-0.255577	0.127302	0.921399	3.118090

24.7 Changing the column type

```
import pandas as pd

data = {
    'A': ['1', '2', '3', '4', '5', '6'],
    'B': ['7.5', '8.5', '9.5', '10.5', '11.5', '12.5'],
    'C': ['x', 'y', 'z', 'x', 'y', 'z']
}
data2 = pd.DataFrame(data)

# Display the original DataFrame
print("Original DataFrame:")
print(data2)

# Change the data type of column 'A' to int
data2['A'] = pd.Series(data2['A'], dtype=int)

# Change the data type of column 'B' to float
data2['B'] = pd.Series(data2['B'], dtype=float)
```

```
# Display the DataFrame after type changes
print("\nDataFrame after type changes:")
print(data2)
```

Original DataFrame:

	A	B	C
0	1	7.5	x
1	2	8.5	y
2	3	9.5	z
3	4	10.5	x
4	5	11.5	y
5	6	12.5	z

DataFrame after type changes:

	A	B	C
0	1	1.0	x
1	2	2.0	y
2	3	3.0	z
3	4	4.0	x
4	5	5.0	y
5	6	6.0	z

```
import pandas as pd

data = {
    'A': ['1', '2', '3', '4', '5', '6'],
    'B': ['7.5', '8.5', '9.5', '10.5', '11.5', '12.5'],
    'C': ['x', 'y', 'z', 'x', 'y', 'z']
}
data2 = pd.DataFrame(data)

# Display the original DataFrame
print("Original DataFrame:")
print(data2)

# Change the data type of column 'A' to int
data2['A'] = data2['A'].astype(int)

# Change the data type of column 'B' to float
data2['B'] = data2['B'].astype(float)
```

```
# Display the DataFrame after type changes
print("\nDataFrame after type changes:")
print(data2)
```

Original DataFrame:

	A	B	C
0	1	7.5	x
1	2	8.5	y
2	3	9.5	z
3	4	10.5	x
4	5	11.5	y
5	6	12.5	z

DataFrame after type changes:

	A	B	C
0	1	7.5	x
1	2	8.5	y
2	3	9.5	z
3	4	10.5	x
4	5	11.5	y
5	6	12.5	z

24.8 Changing characters in categories

```
import pandas as pd

# Creating a DataFrame
data = {
    'A': ['abc', 'def', 'ghi', 'jkl', 'mno', 'pqr'],
    'B': ['1.23', '4.56', '7.89', '0.12', '3.45', '6.78'],
    'C': ['xyz', 'uvw', 'rst', 'opq', 'lmn', 'ijk']
}
data2 = pd.DataFrame(data)

# Display the original DataFrame
print("Original DataFrame:")
print(data2)

# Change lowercase letters to uppercase in column 'A'
```

```
data2['A'] = data2['A'].str.upper()

# Replace the dot with a comma in column 'B'
data2['B'] = data2['B'].str.replace('.', ',')

# Display the DataFrame after modification
print("\nDataFrame after modification:")
print(data2)
```

Original DataFrame:

	A	B	C
0	abc	1.23	xyz
1	def	4.56	uvw
2	ghi	7.89	rst
3	jkl	0.12	opq
4	mno	3.45	lmn
5	pqr	6.78	ijk

DataFrame after modification:

	A	B	C
0	ABC	1,23	xyz
1	DEF	4,56	uvw
2	GHI	7,89	rst
3	JKL	0,12	opq
4	MNO	3,45	lmn
5	PQR	6,78	ijk

24.9 Manipulation operations

Cheat sheet https://pandas.pydata.org/Pandas_Cheat_Sheet.pdf

- merge

<https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.merge.html>

The `merge` function is used to combine two DataFrames along a common column, similar to JOIN operations in SQL.

```
DataFrame.merge(right, how='inner', on=None, left_on=None, right_on=None,
                 left_index=False, right_index=False, sort=False,
                 suffixes=('_x', '_y'), copy=True, indicator=False, validate=None)
```

Where:

- **right**: the DataFrame you want to join with the original DataFrame.
- **how**: specifies the type of join. Four types are available: 'inner', 'outer', 'left', and 'right'. 'inner' is the default value, which returns only those rows that have matching keys in both DataFrames.
- **on**: the name or list of names to be used for joining. This must be a name that occurs in both the original and the right DataFrame.
- **left_on** and **right_on**: names of columns in the left and right DataFrames to be used for joining. This can be used if the column names are not the same.
- **left_index** and **right_index**: whether indexes from the left and right DataFrames should be used for joining.
- **sort**: whether the resulting DataFrame should be sorted by the join keys.
- **suffixes**: suffixes to be added to the names of overlapping columns. The default is ('_x', '_y').
- **copy**: whether to always copy the data, even when not necessary.
- **indicator**: adds a column to the resulting DataFrame that shows the source of each row.
- **validate**: checks whether specified join rules are met.

```
import pandas as pd

df1 = pd.DataFrame({
    'A': ['A0', 'A1', 'A2', 'A3'],
    'B': ['B0', 'B1', 'B2', 'B3'],
    'key': ['K0', 'K1', 'K0', 'K1']
})

df2 = pd.DataFrame({
    'C': ['C0', 'C1'],
    'D': ['D0', 'D1']},
    index=['K0', 'K1']
)

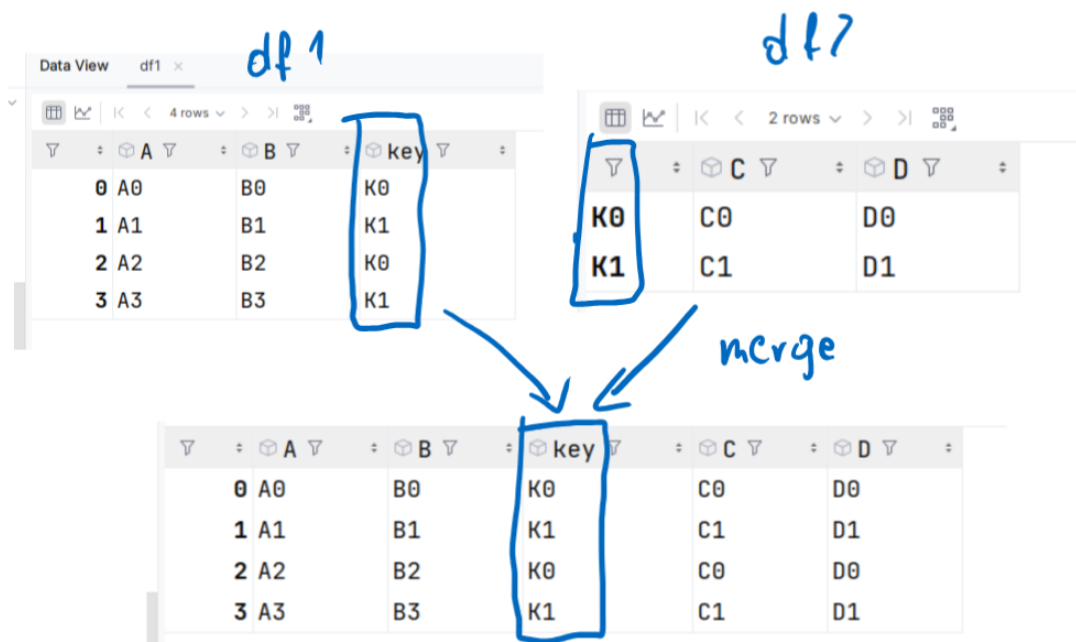
print(df1)
print(df2)
merged_df = df1.merge(df2, left_on='key', right_index=True)
print(merged_df)
```

```
   A  B key
0  A0 B0 K0
1  A1 B1 K1
```

```

2  A2  B2  K0
3  A3  B3  K1
   C    D
K0  C0  D0
K1  C1  D1
   A    B key  C    D
0  A0  B0  K0  C0  D0
1  A1  B1  K1  C1  D1
2  A2  B2  K0  C0  D0
3  A3  B3  K1  C1  D1

```



```

import pandas as pd

df1 = pd.DataFrame({
    'key': ['K0', 'K1', 'K2', 'K3'],
    'A': ['A0', 'A1', 'A2', 'A3'],
    'B': ['B0', 'B1', 'B2', 'B3']
})

df2 = pd.DataFrame({
    'key': ['K0', 'K1', 'K4', 'K5'],

```

```

        'C': ['C0', 'C1', 'C2', 'C3'],
        'D': ['D0', 'D1', 'D2', 'D3']
    })

print(df1)

print(df2)

inner_merged_df = df1.merge(df2, how='inner', on='key', suffixes=('_left', '_right'),
                             indicator=True)
outer_merged_df = df1.merge(df2, how='outer', on='key', suffixes=('_left', '_right'),
                             indicator=True)
left_merged_df = df1.merge(df2, how='left', on='key', suffixes=('_left', '_right'),
                             indicator=True)
right_merged_df = df1.merge(df2, how='right', on='key', suffixes=('_left', '_right'),
                             indicator=True)

print("Inner join")
print(inner_merged_df)

print("Outer join")
print(outer_merged_df)

print("Left join")
print(left_merged_df)

print("Right join")
print(right_merged_df)

```

```

    key  A  B
0  K0  A0 B0
1  K1  A1 B1
2  K2  A2 B2
3  K3  A3 B3
    key  C  D
0  K0  C0 D0
1  K1  C1 D1
2  K4  C2 D2
3  K5  C3 D3
Inner join
    key  A  B  C  D _merge
0  K0  A0 B0 C0 D0   both

```

```

1 K1 A1 B1 C1 D1 both
Outer join
  key  A    B    C    D    _merge
0 K0  A0  B0  C0  D0      both
1 K1  A1  B1  C1  D1      both
2 K2  A2  B2  NaN NaN  left_only
3 K3  A3  B3  NaN NaN  left_only
4 K4  NaN NaN  C2  D2  right_only
5 K5  NaN NaN  C3  D3  right_only
Left join
  key  A    B    C    D    _merge
0 K0  A0  B0  C0  D0      both
1 K1  A1  B1  C1  D1      both
2 K2  A2  B2  NaN NaN  left_only
3 K3  A3  B3  NaN NaN  left_only
Right join
  key  A    B    C    D    _merge
0 K0  A0  B0  C0  D0      both
1 K1  A1  B1  C1  D1      both
2 K4  NaN NaN  C2  D2  right_only
3 K5  NaN NaN  C3  D3  right_only

```


df1

	key	A	B
0	K0	A0	B0
1	K1	A1	B1
2	K2	A2	B2
3	K3	A3	B3

df2

	key	C	D
0	K0	C0	D0
1	K1	C1	D1
2	K4	C2	D2
3	K5	C3	D3

merge
inner

	key	A	B	C	D	_merge
0	K0	A0	B0	C0	D0	both
1	K1	A1	B1	C1	D1	both

df1

	key	A	B
0	K0	A0	B0
1	K1	A1	B1
2	K2	A2	B2
3	K3	A3	B3

df2

	key	C	D
0	K0	C0	D0
1	K1	C1	D1
2	K4	C2	D2
3	K5	C3	D3

merge
outer

	key	A	B	C	D	_merge
0	K0	A0	B0	C0	D0	both
1	K1	A1	B1	C1	D1	both
2	K2	A2	B2	NaN	NaN	left_only
3	K3	A3	B3	NaN	NaN	left_only
4	K4	NaN	NaN	C2	D2	right_only
5	K5	NaN	NaN	C3	D3	right_only

NaN - brak

df1

	key	A	B
0	K0	A0	B0
1	K1	A1	B1
2	K2	A2	B2
3	K3	A3	B3

df2

	key	C	D
0	K0	C0	D0
1	K1	C1	D1
2	K4	C2	D2
3	K5	C3	D3

merge
left

	key	A	B	C	D	_merge
0	K0	A0	B0	C0	D0	both
1	K1	A1	B1	C1	D1	both
2	K2	A2	B2	NaN	NaN	left_only
3	K3	A3	B3	NaN	NaN	left_only

df1

	key	A	B
0	K0	A0	B0
1	K1	A1	B1
2	K2	A2	B2
3	K3	A3	B3

df2

	key	C	D
0	K0	C0	D0
1	K1	C1	D1
2	K4	C2	D2
3	K5	C3	D3

merge right

	key	A	B	C	D	_merge
0	K0	A0	B0	C0	D0	both
1	K1	A1	B1	C1	D1	both
2	K4	NaN	NaN	C2	D2	right_only
3	K5	NaN	NaN	C3	D3	right_only

- join

<https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.join.html>

The join method is used to combine two DataFrames along an axis.

The basic usage of this method looks as follows:

```
DataFrame.join(other, on=None, how='left', lsuffix='', rsuffix='', sort=False)
```

Where:

- **other**: the DataFrame you want to join with the original DataFrame.
- **on**: the name or list of names of columns in the original DataFrame on which you want to join.
- **how**: specifies the type of join. Four types are available: 'inner', 'outer', 'left', and 'right'. 'left' is the default value, which returns all rows from the original DataFrame and matching rows from the second DataFrame. Values are filled with NaN if there is no match.

- `lsuffix` and `rsuffix`: suffixes to be added to repeating columns. The default is empty.
- `sort`: whether to sort the data by the key.

```
import pandas as pd

df1 = pd.DataFrame({
    'A': ['A0', 'A1', 'A2'],
    'B': ['B0', 'B1', 'B2']},
    index=['K0', 'K1', 'K2']
)

df2 = pd.DataFrame({
    'C': ['C0', 'C2', 'C3'],
    'D': ['D0', 'D2', 'D3']},
    index=['K0', 'K2', 'K3']
)

print(df1)

print(df2)

joined_df = df1.join(df2)
print(joined_df)
```

	A	B		
K0	A0	B0		
K1	A1	B1		
K2	A2	B2		
	C	D		
K0	C0	D0		
K2	C2	D2		
K3	C3	D3		
	A	B	C	D
K0	A0	B0	C0	D0
K1	A1	B1	NaN	NaN
K2	A2	B2	C2	D2

df1 df2

	A	B
K0	A0	B0
K1	A1	B1
K2	A2	B2

	C	D
K0	C0	D0
K2	C2	D2
K3	C3	D3

↓ ↙ join

	A	B	C	D
K0	A0	B0	C0	D0
K1	A1	B1	NaN	NaN
K2	A2	B2	C2	D2

- concat

<https://pandas.pydata.org/docs/reference/api/pandas.concat.html>

The `concat` method is used to combine two or more DataFrames along a specified axis.

The basic usage of this method looks as follows:

```
pandas.concat(objs, axis=0, join='outer', ignore_index=False, keys=None,
               levels=None, names=None, verify_integrity=False, sort=False,
               copy=True)
```

Where:

- **objs**: a sequence of DataFrames you want to combine.
- **axis**: the axis along which you want to combine the DataFrames. The default is 0 (combining rows, vertically), but it can also be set to 1 (combining columns, horizontally).

- **join**: specifies the type of join. Two types are available: 'outer' and 'inner'. 'outer' is the default value, which returns all columns from each DataFrame. 'inner' returns only those columns that are common to all DataFrames.
- **ignore_index**: if set to True, does not use the indexes from the DataFrames to create the index in the resulting DataFrame. Instead, it creates a new index from 0 to n-1.
- **keys**: values to associate with the objects.
- **levels**: specified indexes for the new DataFrame.
- **names**: names for the index levels (if multilevel).
- **verify_integrity**: checks whether the new, concatenated DataFrame does not have duplicate indexes.
- **sort**: whether to sort the non-concatenation axis (e.g., indexes if axis=0), regardless of the data.
- **copy**: whether to always copy the data, even when not necessary.

```
import pandas as pd

df1 = pd.DataFrame({
    'A': ['A0', 'A1', 'A2'],
    'B': ['B0', 'B1', 'B2']
})

df2 = pd.DataFrame({
    'A': ['A3', 'A4', 'A5'],
    'B': ['B3', 'B4', 'B5']
})

print(df1)

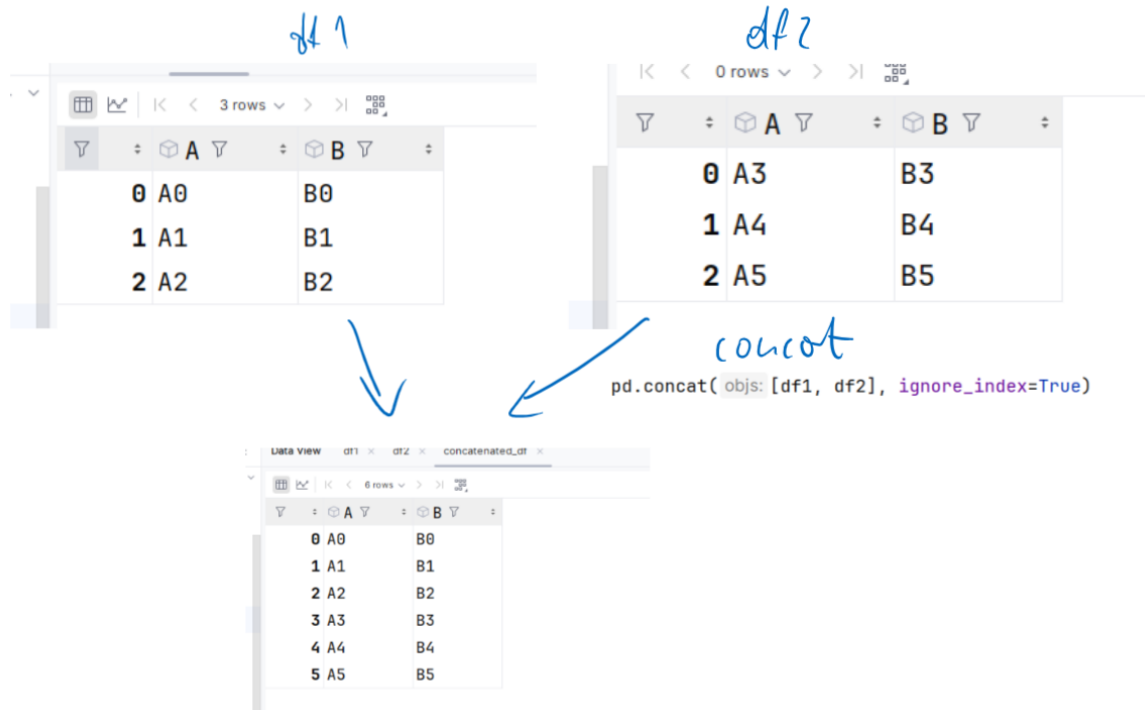
print(df2)

concatenated_df = pd.concat([df1, df2], ignore_index=True)
print(concatenated_df)
```

```

      A  B
0  A0  B0
1  A1  B1
2  A2  B2
      A  B
0  A3  B3
1  A4  B4
2  A5  B5
      A  B
```

0	A0	B0
1	A1	B1
2	A2	B2
3	A3	B3
4	A4	B4
5	A5	B5



```
import pandas as pd

df1 = pd.DataFrame({
    'A': ['A0', 'A1', 'A2'],
    'B': ['B0', 'B1', 'B2']
})

df2 = pd.DataFrame({
    'C': ['C0', 'C1', 'C2'],
    'D': ['D0', 'D1', 'D2']
})

print(df1)
```



```

print(df2)

concatenated_df_axis1 = pd.concat([df1, df2], axis=1)
concatenated_df_keys = pd.concat([df1, df2], keys=['df1', 'df2'])

print(concatenated_df_axis1)
print(concatenated_df_keys)

```

```

      A  B
0  A0  B0
1  A1  B1
2  A2  B2
      C  D
0  C0  D0
1  C1  D1
2  C2  D2
      A  B  C  D
0  A0  B0  C0  D0
1  A1  B1  C1  D1
2  A2  B2  C2  D2
      A  B  C  D
df1 0  A0  B0  NaN  NaN
    1  A1  B1  NaN  NaN
    2  A2  B2  NaN  NaN
df2 0  NaN  NaN  C0  D0
    1  NaN  NaN  C1  D1
    2  NaN  NaN  C2  D2

```

	A	B
0	A0	B0
1	A1	B1
2	A2	B2

	C	D
0	C0	D0
1	C1	D1
2	C2	D2

```
pd.concat(objs: [df1, df2], axis=1)
```

	A	B	C	D
0	A0	B0	C0	D0
1	A1	B1	C1	D1
2	A2	B2	C2	D2

	A	B
0	A0	B0
1	A1	B1
2	A2	B2

	C	D
0	C0	D0
1	C1	D1
2	C2	D2

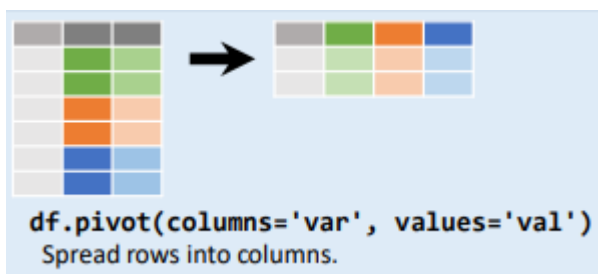
```
pd.concat(objs=[df1, df2], keys=['df1', 'df2'])
```

	A	B	C	D
df1	0 A0	B0	NaN	NaN
	1 A1	B1	NaN	NaN
	2 A2	B2	NaN	NaN
df2	0 NaN	NaN	C0	D0
	1 NaN	NaN	C1	D1
	2 NaN	NaN	C2	D2

podwojny index (multiindex)

- pivot

<https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.pivot.html>



The `pivot` method is used to transform data from a “long” format into a “wide” format.

The basic usage of this method looks as follows:

```
DataFrame.pivot(index=None, columns=None, values=None)
```

Where:

- **index:** the name of a column or a list of column names that are to become the index in the new DataFrame.
- **columns:** the name of a column whose unique values are to become columns in the new DataFrame.
- **values:** the name of a column or a list of column names that are to become the values for the new columns in the new DataFrame.

```
import pandas as pd

df = pd.DataFrame({
    'foo': ['one', 'one', 'one', 'two', 'two', 'two'],
    'bar': ['A', 'B', 'C', 'A', 'B', 'C'],
    'baz': [1, 2, 3, 4, 5, 6],
    'zoo': ['x', 'y', 'z', 'q', 'w', 't'],
})

print(df)

pivot_df = df.pivot(index='foo', columns='bar', values='baz')
print(pivot_df)
```

	foo	bar	baz	zoo
0	one	A	1	x
1	one	B	2	y
2	one	C	3	z
3	two	A	4	q
4	two	B	5	w
5	two	C	6	t

bar	A	B	C
foo			
one	1	2	3
two	4	5	6

	foo	bar	baz	zoo
0	one	A	1	x
1	one	B	2	y
2	one	C	3	z
3	two	A	4	q
4	two	B	5	w
5	two	C	6	t



```
df.pivot(index='foo', columns='bar', values='baz')
```

	A	B	C
one	1	2	3
two	4	5	6

- `wide_to_long`

https://pandas.pydata.org/docs/reference/api/pandas.wide_to_long.html

The `wide_to_long` method is used to transform data from a wide format (where each column contains multiple variables) into a long format (where each column contains a single variable with multiple measurements). This is useful when we have data spread across many columns with repeating or sequential names, and we want to transform this data in a way that facilitates analysis and visualization.

Explanation of the `wide_to_long` parameters

- **stubnames**: A list of the initial parts of column names that are to be transformed.
- **i**: The name of a column or list of columns that identify individual rows. In our example, this is `id`, which uniquely identifies a person.
- **j**: The name of the new column where the various levels of variables will be stored (in our case, `year`).
- **sep**: Optional separator (default `"`).

```

import pandas as pd

# Sample data
data = {
    'id': ['A', 'B', 'C'],
    'height_2020': [180, 175, 165],
    'weight_2020': [70, 76, 65],
    'height_2021': [181, 176, 166],
    'weight_2021': [71, 77, 66]
}

data2 = pd.DataFrame(data)

# Transformation to long format
df_long = pd.wide_to_long(data2, stubnames=['height', 'weight'], i='id', j='year', sep='_')
df_long = df_long.reset_index()

print(df_long)

```

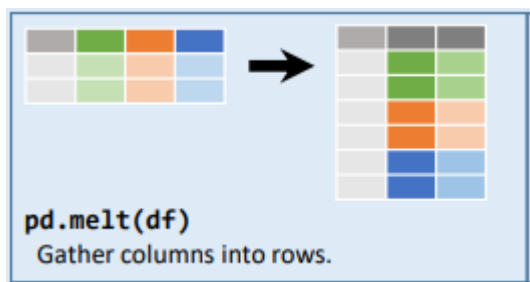
	id	year	height	weight
0	A	2020	180	70
1	B	2020	175	76
2	C	2020	165	65
3	A	2021	181	71
4	B	2021	176	77
5	C	2021	166	66

	id	height_2020	weight_2020	height_2021	weight_2021
0	A	180	70	181	71
1	B	175	76	176	77
2	C	165	65	166	66


```
df_long = pd.wide_to_long(df, stubnames=['height', 'weight'], i='id', j='year', sep='_')
df_long = df_long.reset_index()
```


	id	year	height	weight
0	A	2020	180	70
1	B	2020	175	76
2	C	2020	165	65
3	A	2021	181	71
4	B	2021	176	77
5	C	2021	166	66

- melt



<https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.melt.html>

The `melt` function is used to transform data from a wide format into a long format.

The basic usage of this method looks as follows:

```
pandas.melt(frame, id_vars=None, value_vars=None, var_name=None, value_name='value', col_level=None)
```

Where:

- **frame**: the DataFrame you want to process.
- **id_vars**: the column(s) you want to keep as identifiers. These columns will not be changed.
- **value_vars**: the column(s) you want to transform into key-value pairs. If not provided, all columns that are not **id_vars** will be used.

- **var_name**: the name of the new column that will contain the names of the columns transformed into key-value pairs. The default is 'variable'.
- **value_name**: the name of the new column that will contain the values of the columns transformed into key-value pairs. The default is 'value'.
- **col_level**: if the columns are multilevel, this is the level that will be used to transform the columns into key-value pairs.

```
import pandas as pd

data = pd.DataFrame({
    'A': ['foo', 'bar', 'baz'],
    'B': ['one', 'one', 'two'],
    'C': [2.0, 1.0, 3.0],
    'D': [3.0, 2.0, 1.0]
})
print(data)
melted_df = data.melt(id_vars=['A', 'B'], value_vars=['C', 'D'], var_name='My_Var',
                      value_name='My_Val')
print(melted_df)
```

	A	B	C	D
0	foo	one	2.0	3.0
1	bar	one	1.0	2.0
2	baz	two	3.0	1.0

	A	B	My_Var	My_Val
0	foo	one	C	2.0
1	bar	one	C	1.0
2	baz	two	C	3.0
3	foo	one	D	3.0
4	bar	one	D	2.0
5	baz	two	D	1.0

	A	B	C	D
0	foo	one	2.0	3.0
1	bar	one	1.0	2.0
2	baz	two	3.0	1.0

`df.melt(id_vars=['A', 'B'], value_vars=['C', 'D'], var_name='My_Var', value_name='My_Val')`

	A	B	My_Var	My_Val
0	foo	one	C	2.0
1	bar	one	C	1.0
2	baz	two	C	3.0
3	foo	one	D	3.0
4	bar	one	D	2.0
5	baz	two	D	1.0

Part V

Structure Analysis

25 Structure Analysis

Statistical measures

- are numerical characteristics that allow describing the properties of the distribution of the studied feature.
- other names:
 - parameters - data analyzed from the entire population,
 - sample statistics - data analyzed from a sample.

Classification of statistical measures:

- measures of location (measures of level, measures of central tendency) - allow determining where in the set of values the data from observations are located,
- measures of variability (measures of dispersion, variability, spread) - allow determining the differentiation of units,
- measures of asymmetry (measures of skewness) - allow determining asymmetry (whether most units have values greater or smaller relative to the average level),
- measures of concentration - allow determining the concentration of values relative to the mean.

25.1 Measures of location

- classical means:
 - arithmetic mean,
- positional means and quantiles:
 - mode,
 - median (second quartile),
 - quantiles (quartiles, deciles, percentiles).

25.1.1 Arithmetic mean

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

The interpretation of the arithmetic mean consists in understanding it as a representation of the “middle” or “typical” level of the feature of the studied population. The mean gives a general idea of the data, but can be misleading in the presence of extreme values (outliers), which can significantly affect its value. It is useful in many fields, from economics to social sciences, as a way to summarize data and compare different groups or data sets. It is worth remembering that the mean is not always the best choice for skewed distributions and may not adequately reflect the data distribution, especially in the presence of extreme values.

```
import pandas as pd

# Example data as a Pandas series
data = pd.Series([10, 20, 30, 40, 50, 60, 70, 80, 90, 100])

# Calculating the mean
mean = data.mean()

print(f"Mean: {mean}")
```

Mean: 55.0

25.1.2 Mode

- symbol: *Do*
- also known as modal value.
- for a discrete feature, it is the value of the feature that occurs most frequently.
- for a continuous feature, it is the value of the feature around which most measurements oscillate (the argument for which the probability density takes the largest value)

```
import pandas as pd

# Example data for a discrete variable
discrete_data = pd.Series([1, 2, 2, 3, 3, 3, 4, 4, 4, 4])

# Calculating the mode
discrete_mode = discrete_data.mode()

print(f"Mode for the discrete variable: {discrete_mode.tolist()}")
```

Mode for the discrete variable: [4]

Notes:

- it cannot always be determined precisely.
- determining the mode makes sense when the distribution is unimodal (units have one point of concentration); in the case of multimodal distributions, the mode loses its usefulness as a measure of central tendency.
- it is not sensitive to extreme values like the arithmetic mean.
- in the case of a symmetric distribution, the mode equals the mean

25.1.3 Median

- symbol: Me
- it can be determined for features expressed on any scale except the nominal scale.
- the value of the feature that the unit in the middle of the ordered sequence of observations has.
- for an odd number of observations: the value at position $\frac{n+1}{2}$
- for an even number of observations:
 - we determine the values at positions $\frac{n}{2}$ and $\frac{n}{2} + 1$
 - we calculate the mean of these values

Note: a common mistake is confusing the value of the feature with its position.

Code

```
import pandas as pd

# Example data in a DataFrame
df = pd.DataFrame({
    'Column1': [10, 20, 30, 40, 50],
    'Column2': [15, 25, 35, 45, 55]
})

# Calculating the median for each column
medians = df.median()

print("Median for each column:")
print(medians)
```

Median for each column:

Column1 30.0

Column2 35.0

dtype: float64

Interpretation of the median:

- at least half of the units have a feature value less than or equal to the median.
- the median is insensitive to extreme values.

25.1.4 Quantiles

- values of a feature that divide the population into specific parts in terms of the number of units.
- most commonly used:
 - quartiles - divide the population into 4 equal parts (the second quartile is the median)
 - deciles - divide the population into 10 equal parts
 - percentiles (centiles) - divide the population into 100 equal parts.

Quartiles

- symbols: Q_1, Q_2, Q_3

```
import pandas as pd

# Example data as a Pandas series
data = pd.Series([10, 20, 30, 40, 50, 60, 70, 80, 90, 100])

# Calculating quartiles
quartile_1 = data.quantile(0.25) # First quartile (Q1)
median = data.quantile(0.50)     # Median (Q2)
quartile_3 = data.quantile(0.75) # Third quartile (Q3)

print(f"First quartile (Q1): {quartile_1}")
print(f"Median (Q2): {median}")
print(f"Third quartile (Q3): {quartile_3}")
```

First quartile (Q1): 32.5

Median (Q2): 55.0

Third quartile (Q3): 77.5

25.2 Measures of variability

- division:
 - classical measures - based on all observations,
 - * variance,
 - * standard deviation,
 - positional measures - based on feature values occupying specific positions,
 - * range
 - * interquartile range,

25.2.1 Range

- symbol $R = \max - \min$
- also known as the empirical area of variability, amplitude of fluctuations.
- the difference between the maximum value and the minimum value.

```
import pandas as pd

# Example data as a Pandas series
data = pd.Series([10, 20, 30, 40, 50, 60, 70, 80, 90, 100])

# Calculating the maximum and minimum
maximum = data.max()
minimum = data.min()

# Calculating the difference between maximum and minimum
difference = maximum - minimum

print(f"Maximum: {maximum}")
print(f"Minimum: {minimum}")
print(f"Difference between maximum and minimum: {difference}")
```

```
Maximum: 100
Minimum: 10
Difference between maximum and minimum: 90
```

25.2.2 Interquartile range

- symbol $R_Q = Q_3 - Q_1$
- the difference between the third quartile and the first quartile.
- measures the range of the middle 50% of units.

25.2.3 Quartile deviation

- formula: $Q = \frac{Q_3 - Q_1}{2}$
- half of the interquartile range.

25.2.4 Variance

Variance provides information about how large the variability of results is in a given set of feature values. In other words, whether the results are more concentrated around the mean, whether there are small differences between the mean and individual results, or perhaps the spread of results is large, with a large difference between individual results and the mean.

- formula: $s^2 = \frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n}$ (population) or $s^2 = \frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n-1}$ (sample)

25.2.5 Standard deviation

The standard deviation measures variability with a unit consistent with the unit of the studied feature, giving the average differences of individual feature values from the arithmetic mean.

- formula: $s = \sqrt{s^2}$

```
import pandas as pd

# Example data as a Pandas series
data = pd.Series([10, 20, 30, 40, 50, 60, 70, 80, 90, 100])

# Calculating variance
variance = data.var() # Note: Pandas divides by (n-1) by default, which corresponds to sample variance

# Calculating standard deviation
standard_deviation = data.std() # By default calculated from a sample (n-1)

print(f"Variance: {variance}")
print(f"Standard deviation: {standard_deviation}")
```


Variance: 916.6666666666666

Standard deviation: 30.276503540974915

In practice, it is often assumed that the data have a normal distribution. This assumption is never completely fulfilled. The normal distribution has a non-zero probability density for every value from the set of real numbers, whereas in the real world the values of random variables are always limited - for example, there are no people with negative height. However, very often this assumption is fulfilled with sufficiently good approximation. The better it is justified, the closer to the truth the following statements may be:

- 68% of feature values lie within a distance $\leq s$ from the mean
- 95% of feature values lie within a distance $\leq 2s$ from the mean
- 99.7% of feature values lie within a distance $\leq 3s$ from the mean.

https://en.wikipedia.org/wiki/68%E2%80%9395%E2%80%9399.7_rule

25.3 Measures of asymmetry

How to recognize the type of distribution?

- symmetric distribution

$$\bar{x} = Me = Do$$

- distribution with right-sided asymmetry

$$\bar{x} > Me > Do$$

- distribution with left-sided asymmetry

$$\bar{x} < Me < Do$$

- classical coefficient of asymmetry

- formula:

$$A_s = \frac{m_3}{(s^2)^{\frac{3}{2}}}$$

25.4 Measures of concentration

Kurtosis coefficient

- also known as the coefficient of concentration, coefficient of flatness.
- formula:

$$K = \frac{m_4}{s^4} \quad m_4 = \frac{\sum_{i=1}^n (x_i - \bar{x})^4}{n}$$

Interpretation of kurtosis

- $K = 3$ - the distribution has the same concentration as the normal distribution
- $K > 3$ - stronger concentration
- $K < 3$ - weaker concentration

Sometimes the excess coefficient $K' = K - 3$ is examined.

High kurtosis means a greater number of extreme values, while low kurtosis indicates a distribution that is flatter than normal.

```
import pandas as pd

# Example data in a series
data = pd.Series([10, 20, 30, 40, 50, 60, 70, 80, 90, 100])

# Calculating skewness
skewness = data.skew()

# Calculating kurtosis
kurtosis = data.kurt()

print(f"Skewness: {skewness}")
print(f"Kurtosis: {kurtosis}")
```

Skewness: 0.0

Kurtosis: -1.2000000000000002

25.5 Speeding up operations:

```
import pandas as pd
import numpy as np

# Creating a data frame
data = {
    "Product": ["A", "B", "A", "C", "B", "A", "C", "B", "A", "C"],
    "Sales": [200, 150, 250, 300, 200, 220, 310, 180, 240, 290],
    "Cost": [120, 100, 140, 180, 110, 130, 190, 105, 125, 170],
    "Margin": [80, 50, 110, 120, 90, 90, 120, 75, 115, 120],
}

df = pd.DataFrame(data)
print(df)
```

Data frame:

Product	Sales	Cost	Margin
A	200	120	80
B	150	100	50
A	250	140	110
C	300	180	120
B	200	110	90
A	220	130	90
C	310	190	120
B	180	105	75
A	240	125	115
C	290	170	120

Mean for two selected columns

```
# Calculating the mean for the "Sales" and "Cost" columns
mean_values = df[["Sales", "Cost"]].mean()
print(mean_values)
```

The `describe()` function

```
# Generating descriptive statistics for numerical data
description = df.describe()
print(description)
```

How the `agg` function works

The `agg` function allows applying different measures to multiple columns.

```
# Applying aggregating functions
agg_results = df[["Sales", "Cost"]].agg(["mean", "sum", "max"])
print(agg_results)
```

How `groupby` works

Grouping data by a categorical column (e.g., “Product”).

```
# Grouping and calculating the mean
grouped = df.groupby("Product")[["Sales", "Cost"]].mean()
print(grouped)
```

The `numeric_only=True` option

The `numeric_only=True` option allows analyzing only numerical columns, omitting categorical ones.

```
# Calculating the sum only for numerical columns
numeric_sum = df.sum(numeric_only=True)
print(numeric_sum)
```

Part VI

Matplotlib

26 Matplotlib - start

Loading the library:

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
```

<https://matplotlib.org/>

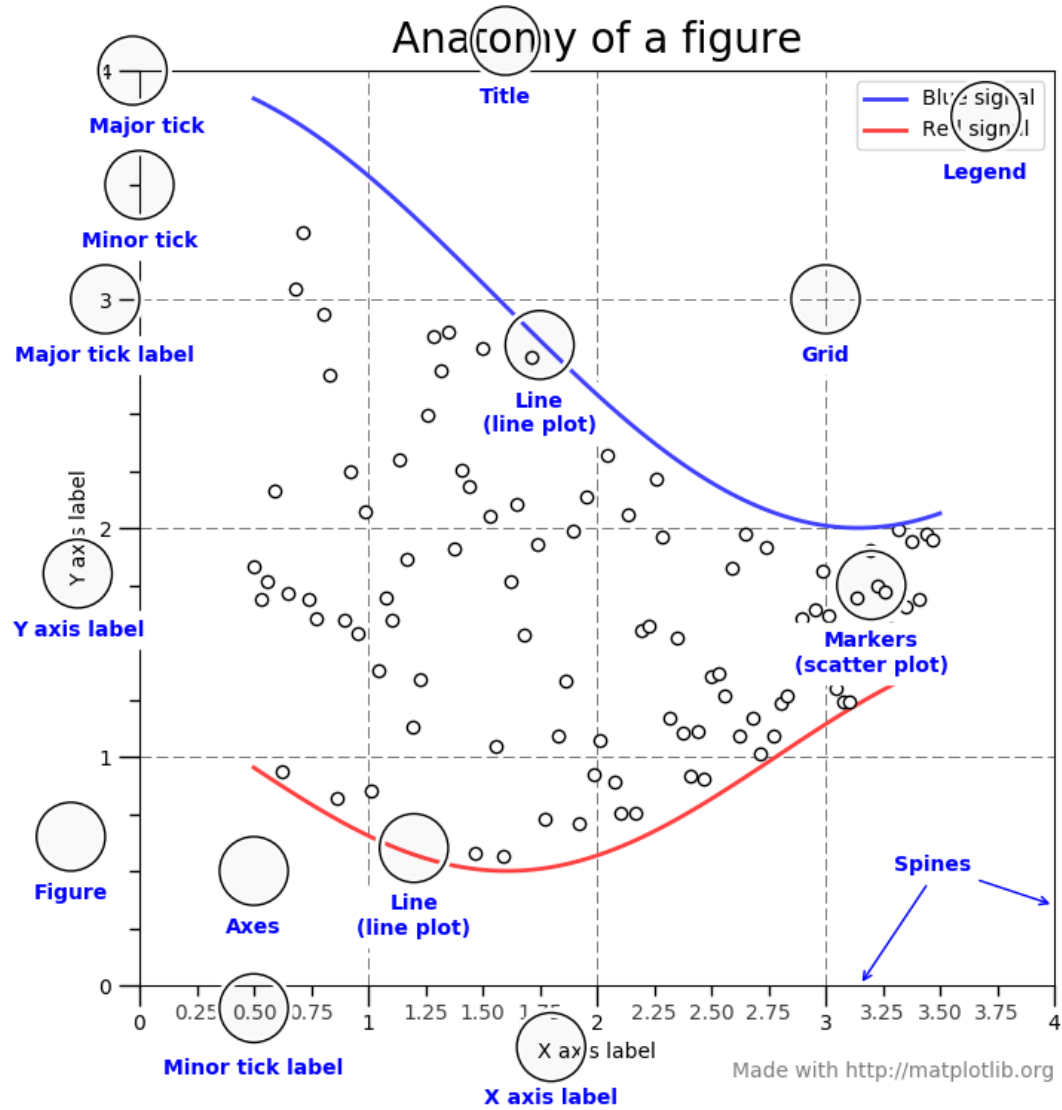


Chart galleries

<https://matplotlib.org/gallery/index.html>

<https://python-graph-gallery.com/>

<https://github.com/rasbt/matplotlib-gallery>

<https://seaborn.pydata.org/examples/index.html>

IDEA:

1) Import libraries

2) "Extracting" vectors into plot coordinates

3) "Plot options"

4) "Drawing functions"

5) "Add-ons"

6) Save to file

7) "show()"

27 Matplotlib - line chart

A line chart is one of the most powerful data visualization tools, allowing us to spot relationships and trends that might otherwise go unnoticed in raw data. Below is a detailed explanation of when and why it's worth reaching for this type of chart.

A line chart works best when we want to show **changes in values over time** or as a **function of another continuous variable**. Its strength lies in its ability to reveal continuous relationships between data points, making it easy to track trends and patterns.

1. Analysis of changes over time

A line chart perfectly illustrates how data changes across successive units of time. It is invaluable for presenting: - Economic trends (GDP growth, inflation, unemployment rates) - Changes in financial markets (exchange rates, stock prices, interest rates) - Climate and weather data (temperature changes, precipitation, pollution levels) - Health indicators (heart rate, blood sugar levels, blood pressure)

2. Showing relationships between variables

When we want to examine how one variable influences another, a line chart allows for an intuitive representation of these relationships: - The relationship between level of education and average earnings - The correlation between age and specific skills - The dependence between advertising spending and sales results

3. Comparing multiple trends simultaneously

A line chart enables effective comparison of several data series on a single chart: - Analysis of sales of different products over time - Comparison of results across different regions, teams, or countries - Comparison of actual results with planned targets - Comparison of different economic indicators over the same period

4. Analysis of correlation and causality

Lines on a chart help to see how changes in one variable may influence others: - Examining the impact of fuel prices on sales of different types of vehicles - Analyzing the relationship between ambient temperature and energy consumption - Assessing the impact of marketing campaigns on brand awareness

5. Identifying anomalies and turning points

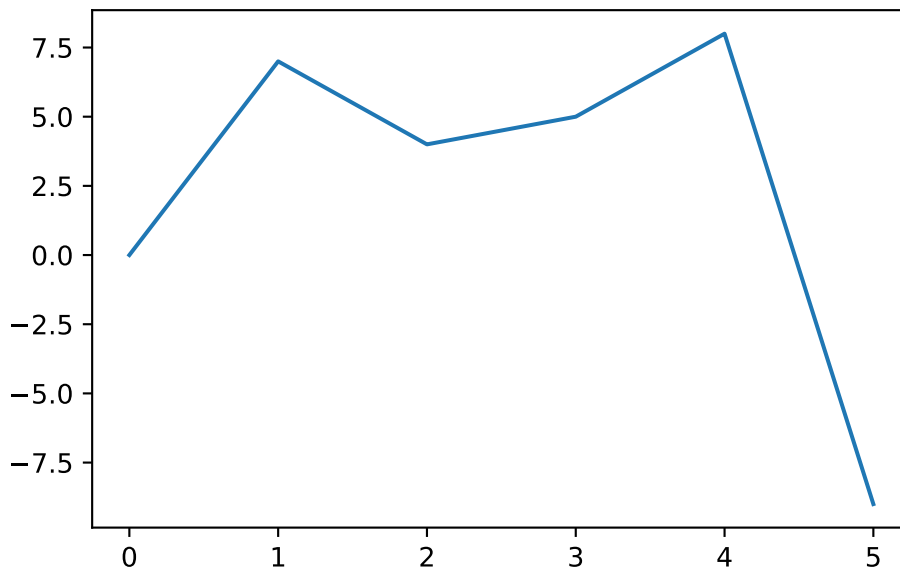
A line chart makes it easy to quickly notice outliers and moments of significant change: - Detecting unusual patterns in financial data - Identifying turning points in social trends - Recognizing seasonal fluctuations in data

Although line charts are most suitable for continuous data, they can also be used to present discrete data, as long as there is a logical connection between successive points. For example, monthly sales results are discrete data, but presenting them as a line will help reveal the annual trend.

It should be remembered, however, that connecting points with a line suggests continuity between them. If there is no logical continuity between data points (e.g., when comparing unrelated categories), a better choice will be a bar chart or scatter plot.

```
import matplotlib.pyplot as plt
```

```
x = [0, 7, 4, 5, 8, -9]  
plt.plot(x)  
plt.show()
```



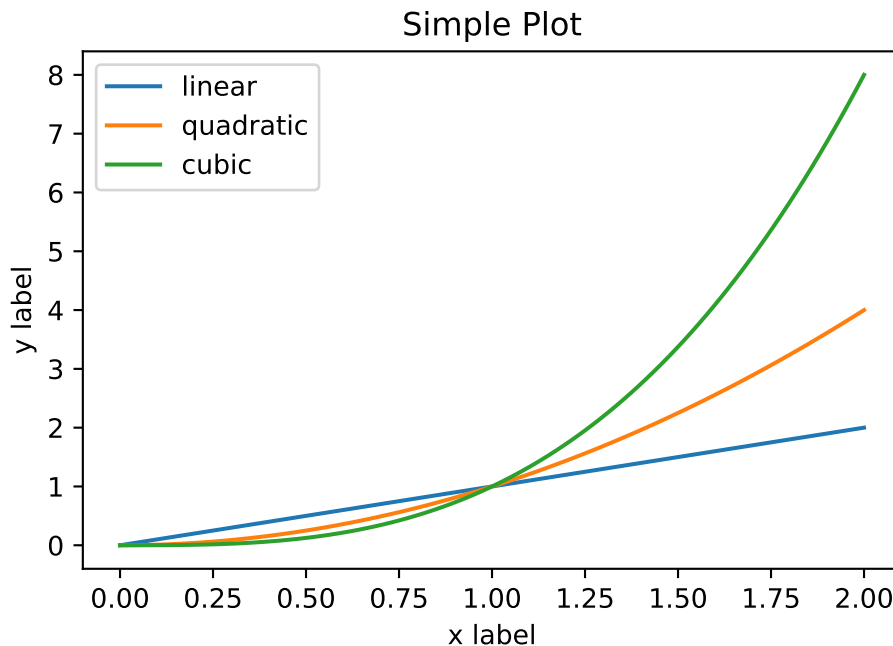
```
import matplotlib.pyplot as plt  
import numpy as np
```

```
x = np.linspace(0, 2, 100)  
plt.plot(x, x, label='linear')  
plt.plot(x, x ** 2, label='quadratic')
```

①
②
③

```
plt.plot(x, x ** 3, label='cubic') ④
plt.xlabel('x label') ⑤
plt.ylabel('y label') ⑥
plt.title("Simple Plot") ⑦
plt.legend() ⑧
plt.show() ⑨
```

- ① `x = np.linspace(0, 2, 100)`: creates an array `x` with 100 evenly spaced values from 0 to 2 (inclusive), using the `linspace` function from the `numpy` library.
- ② `plt.plot(x, x, label='linear')`: draws a linear chart ($y = x$) using the values from the `x` array.
- ③ `plt.plot(x, x**2, label='quadratic')`: draws a quadratic chart ($y = x^2$) using the values from the `x` array.
- ④ `plt.plot(x, x**3, label='cubic')`: draws a cubic chart ($y = x^3$) using the values from the `x` array.
- ⑤ `plt.xlabel('x label')`: adds a label to the X axis.
- ⑥ `plt.ylabel('y label')`: adds a label to the Y axis.
- ⑦ `plt.title("Simple Plot")`: gives the chart the title “Simple Plot”.
- ⑧ `plt.legend()`: adds a legend to the chart, which shows the labels (label) for the individual lines.
- ⑨ `plt.show()`: displays the chart.



Object-oriented version:

```

import matplotlib.pyplot as plt
import numpy as np

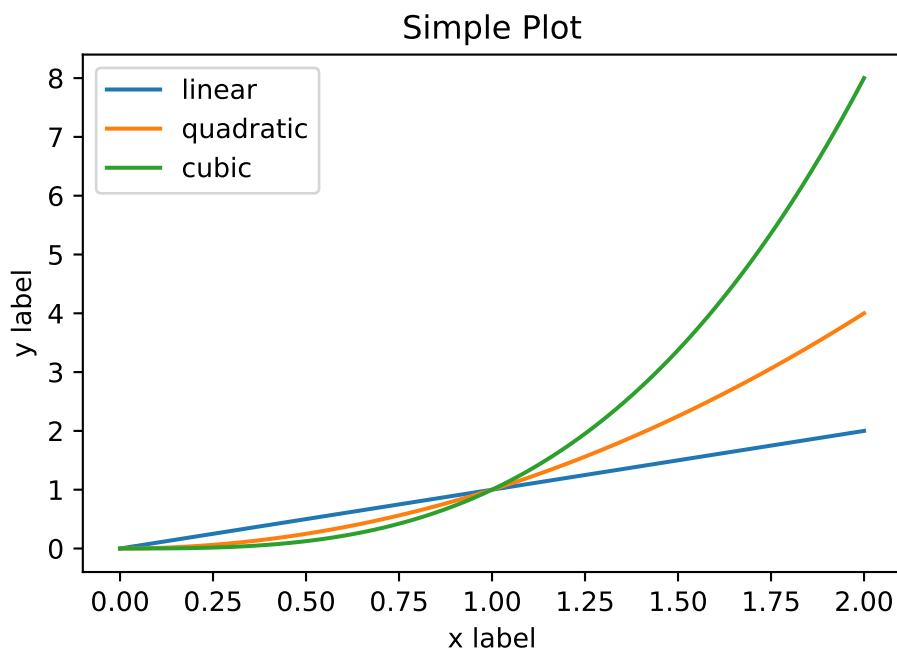
fig, ax = plt.subplots()
x = np.linspace(0, 2, 100)

ax.plot(x, x, label='linear')
ax.plot(x, x ** 2, label='quadratic')
ax.plot(x, x ** 3, label='cubic')

ax.set_xlabel('x label')
ax.set_ylabel('y label')
ax.set_title("Simple Plot")

ax.legend()
plt.show()

```



Version with dates:

```

import matplotlib.pyplot as plt
import numpy as np

```

```

dates = np.arange('2024-01', '2025-01', dtype='datetime64')

index = np.arange(len(dates))

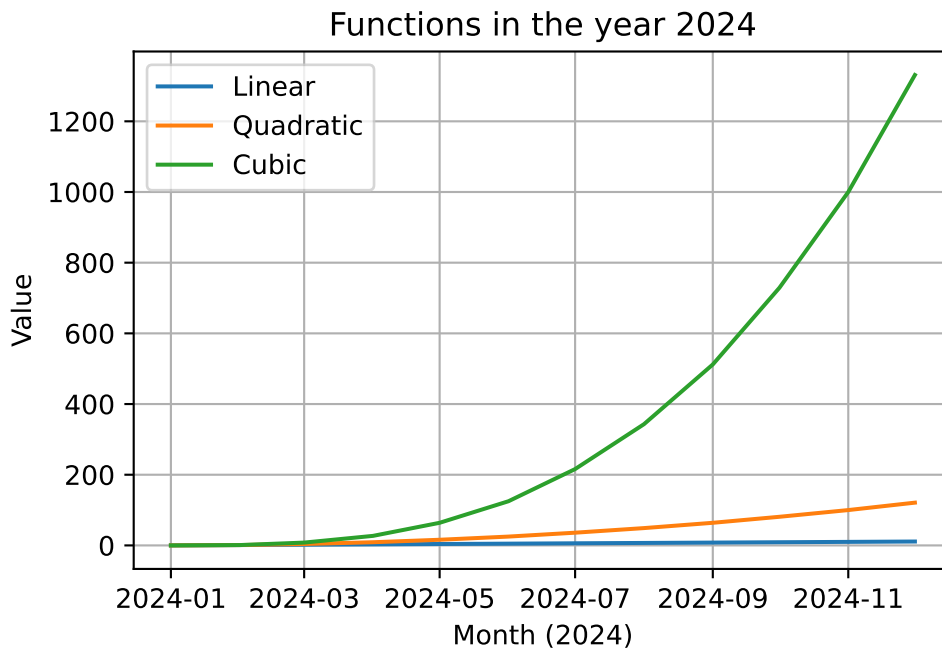
y_1 = index
y_2 = index ** 2
y_3 = index ** 3

plt.plot(dates, y_1, label='Linear')
plt.plot(dates, y_2, label='Quadratic')
plt.plot(dates, y_3, label='Cubic')

plt.xlabel('Month (2024)')
plt.ylabel('Value')
plt.title("Functions in the year 2024")
plt.legend()
plt.grid(True)

plt.show()

```



```

import matplotlib.pyplot as plt
import numpy as np

```

```

# Creating the data
dates = np.arange('2024-01', '2025-01', dtype='datetime64')
index = np.arange(len(dates))
y_1 = index
y_2 = index**2
y_3 = index**3

# Creating figure and axes in the object-oriented approach
fig, ax = plt.subplots()

# Drawing lines on the axes
ax.plot(dates, y_1, label='Linear')
ax.plot(dates, y_2, label='Quadratic')
ax.plot(dates, y_3, label='Cubic')

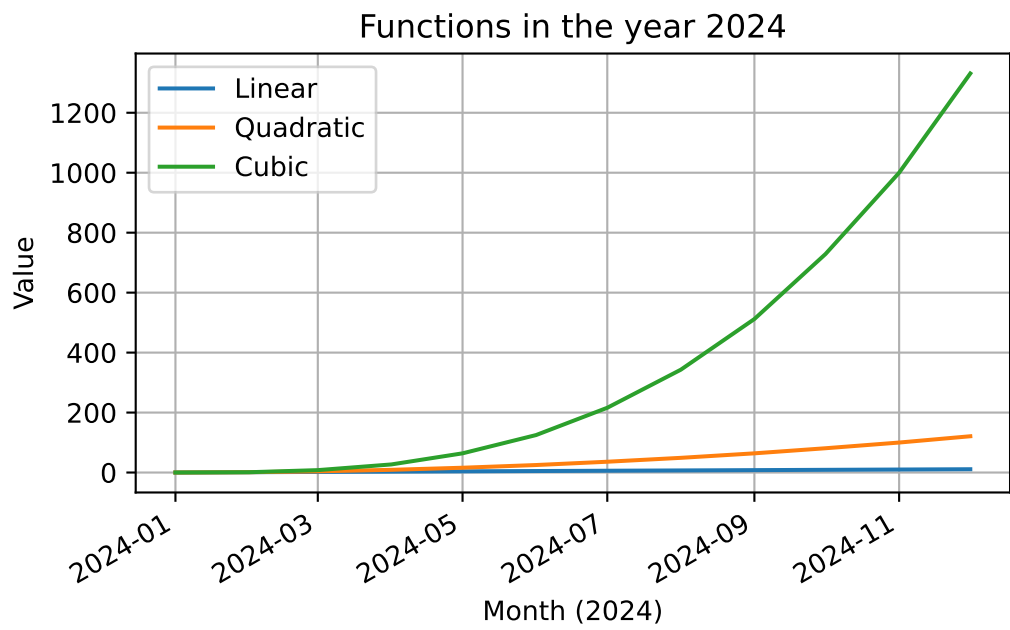
# Adding labels and title
ax.set_xlabel('Month (2024)')
ax.set_ylabel('Value')
ax.set_title('Functions in the year 2024')

# Adding legend and grid
ax.legend()
ax.grid(True)

# Adjusting appearance - optional improvements
fig.autofmt_xdate() # Automatic formatting of dates on the X axis for better readability
plt.tight_layout()  # Automatic adjustment of chart size

# Displaying the chart
plt.show()

```



28 Matplotlib - extras part 1

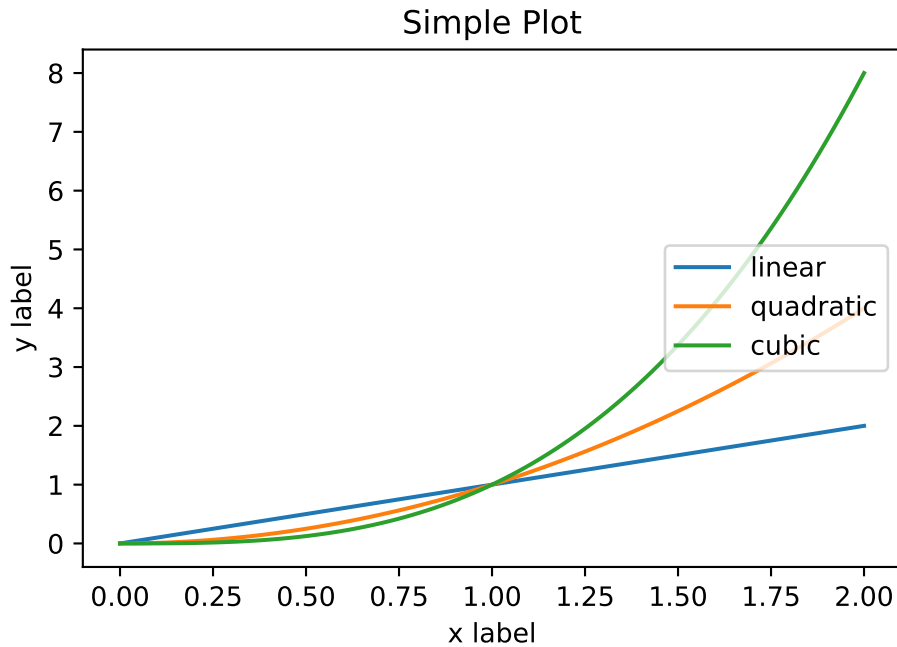
28.1 Legend parameters

Location String	Location Code
'best'	0
'upper right'	1
'upper left'	2
'lower left'	3
'lower right'	4
'right'	5
'center left'	6
'center right'	7
'lower center'	8
'upper center'	9
'center'	10

[**Legend location parameters**

```
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0, 2, 100)
plt.plot(x, x, label='linear')
plt.plot(x, x ** 2, label='quadratic')
plt.plot(x, x ** 3, label='cubic')
plt.xlabel('x label')
plt.ylabel('y label')
plt.title("Simple Plot")
plt.legend(loc = 5)
plt.show()
```

```
import matplotlib.pyplot as plt
import numpy as np

# We create the Figure object and the axes (Axes)
fig = plt.figure()
ax = fig.add_subplot(111) # 111 means: 1 row, 1 column, first chart

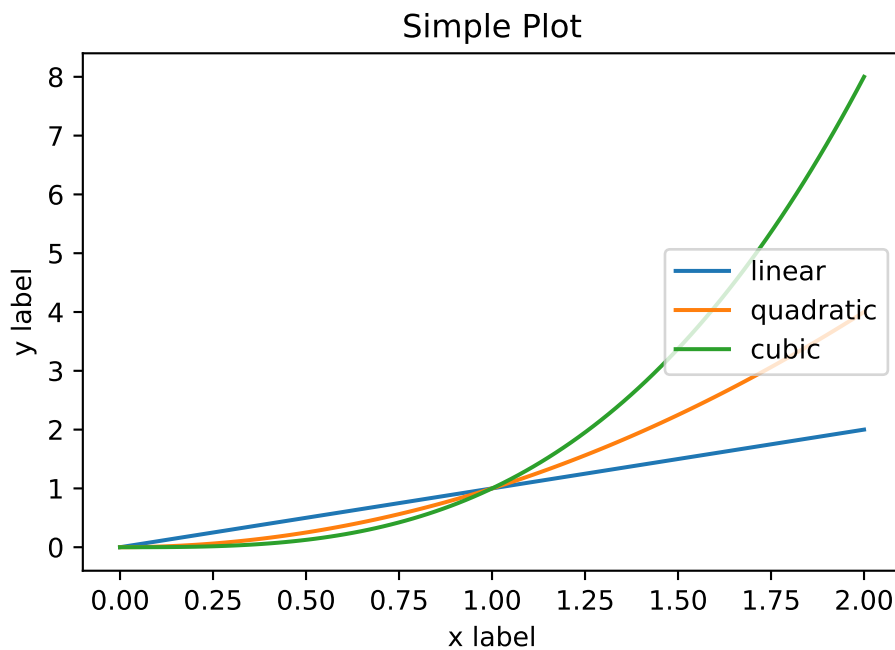
# We generate the data
x = np.linspace(0, 2, 100)

# We draw the plots on the axis
ax.plot(x, x, label='linear')
ax.plot(x, x**2, label='quadratic')
ax.plot(x, x**3, label='cubic')

# We add labels and a title
ax.set_xlabel('x label')
ax.set_ylabel('y label')
ax.set_title("Simple Plot")

# We add a legend
ax.legend(loc=5) # loc=5 means position "right"
```

```
# We display the chart
plt.show()
```



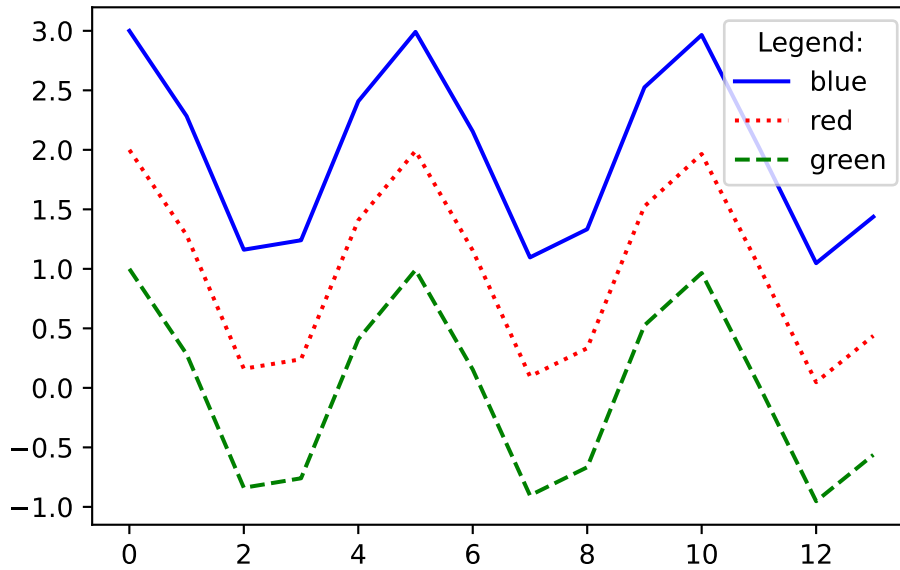
28.2 Styles, line colors

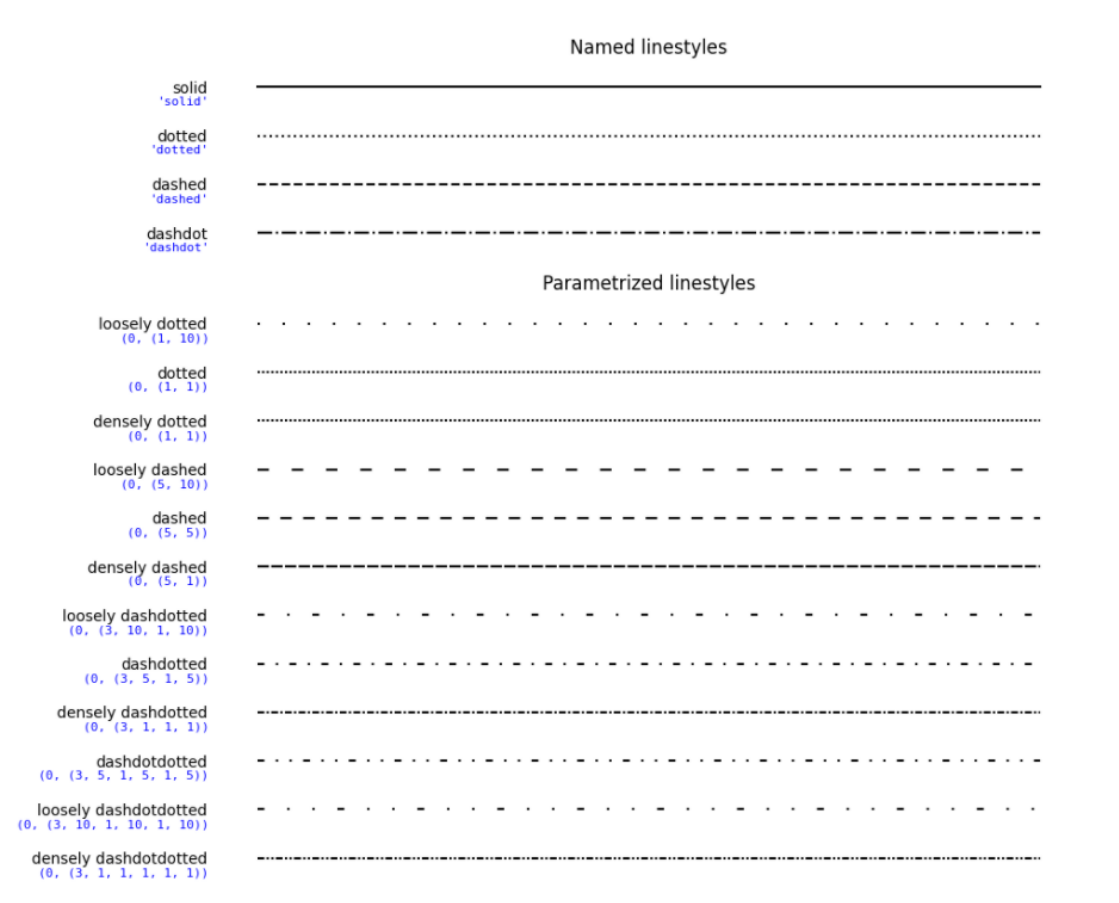
```
import numpy as np
import matplotlib.pyplot as plt

x = np.arange(14)
y = np.cos(5 * x)
plt.plot(x, y + 2, 'blue', linestyle="-", label="blue")
plt.plot(x, y + 1, 'red', linestyle=":", label="red")
plt.plot(x, y, 'green', linestyle="--", label="green")
plt.legend(title='Legend:')
plt.show()
```

- ① `x = np.arange(14)`: creates an array `x` with values from 0 to 13 (inclusive of 13), using the `arange` function from the `numpy` library.
- ② `y = np.cos(5 * x)`: calculates the cosine function values for each value of `x`, multiplied by 5. The resulting values are stored in the array `y`.

- ③ `plt.plot(x, y + 2, 'blue', linestyle="-", label="blue")`: draws a blue chart with values from the array `x`, and the `y` values shifted upward by 2. The line is solid (`linestyle="-"`).
- ④ `plt.plot(x, y + 1, 'red', linestyle=":", label="red")`: draws a red chart with values from the array `x`, and the `y` values shifted upward by 1. The line is dotted (`linestyle=":"`).
- ⑤ `plt.plot(x, y, 'green', linestyle="--", label="green")`: draws a green chart with values from the array `x` and the `y` values. The line is dashed (`linestyle="--"`).





Linestyle	Description
'-' or 'solid'	solid line
'--' or 'dashed'	dashed line
'-.' or 'dashdot'	dash-dotted line
':' or 'dotted'	dotted line
'None' or ' ' or ''	draw nothing

Simplified

(offset, (on_off_seq))

(0, (3, 10, 1, 15)) - (3pt line, 10pt space, 1pt line, 15pt space)

line *space* *line* *space*

```
import numpy as np
import matplotlib.pyplot as plt

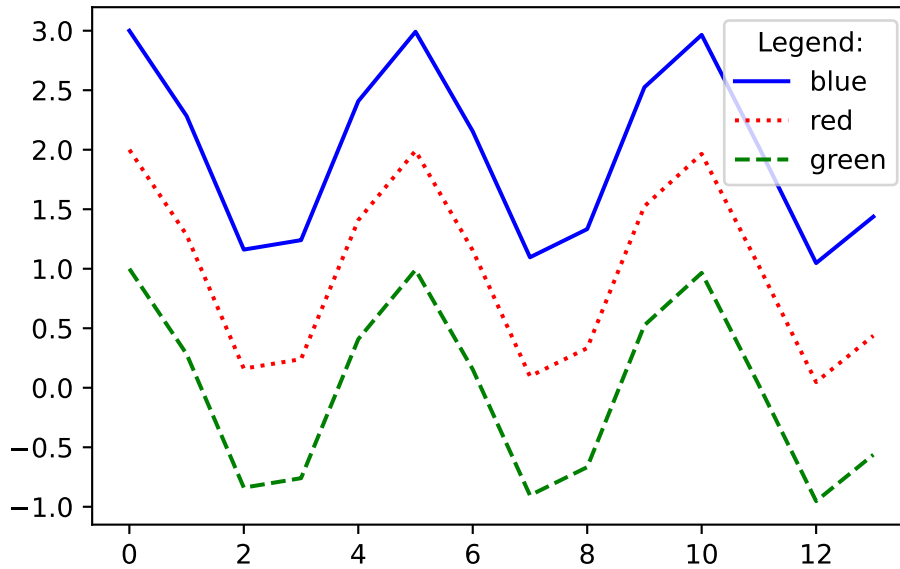
fig = plt.figure()
ax = fig.add_subplot(111)

x = np.arange(14)
y = np.cos(5 * x)

ax.plot(x, y + 2, 'blue', linestyle="-", label="blue")
ax.plot(x, y + 1, 'red', linestyle=":", label="red")
ax.plot(x, y, 'green', linestyle="--", label="green")

ax.legend(title='Legend:')

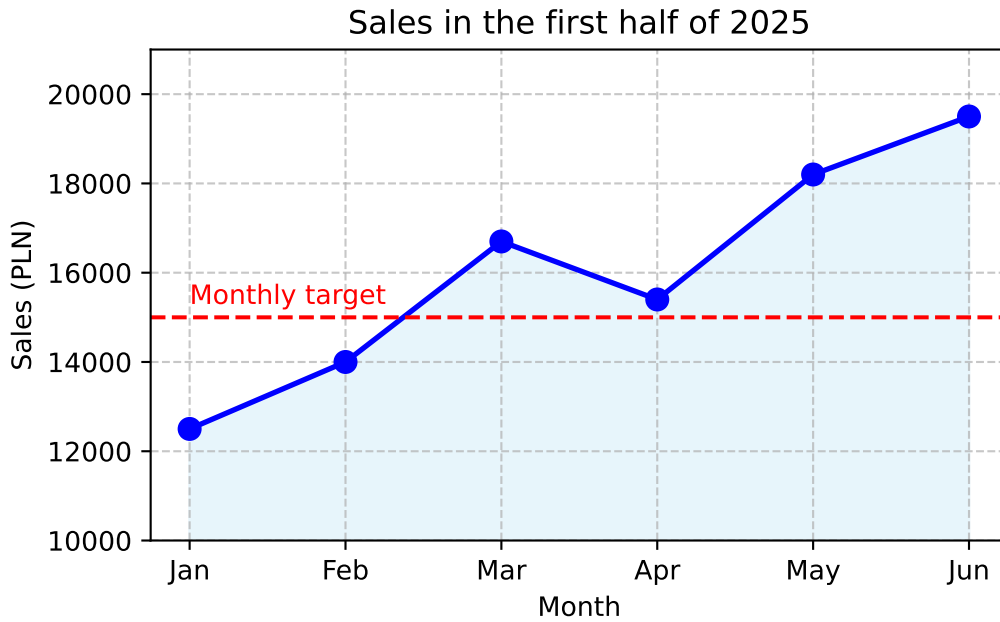
plt.show()
```



```
import matplotlib.pyplot as plt
import numpy as np

months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun']
sales = [12500, 14000, 16700, 15400, 18200, 19500]

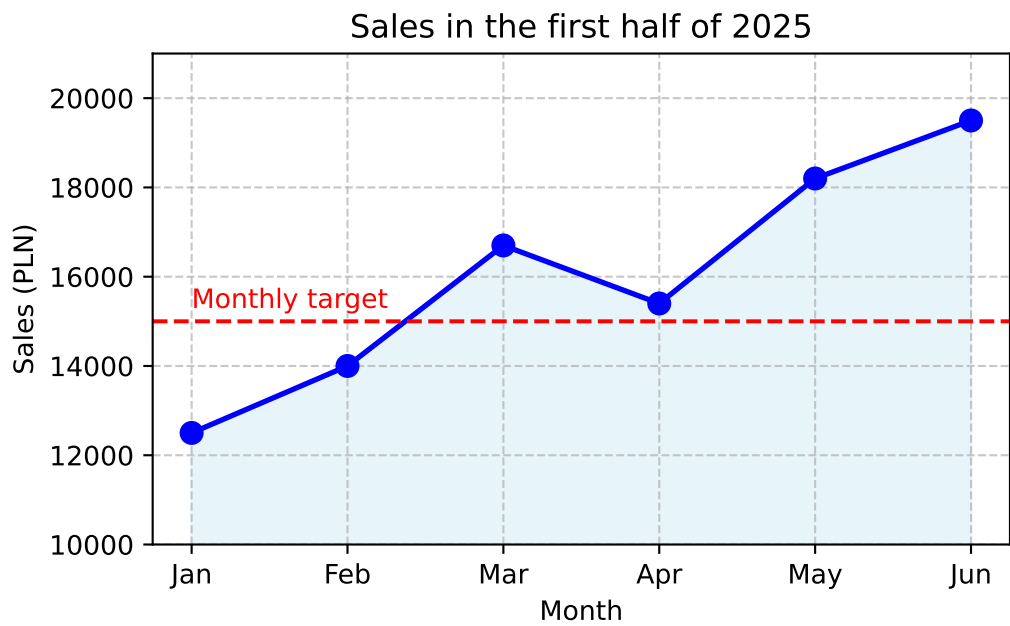
plt.plot(months, sales, 'bo-', linewidth=2, markersize=8)
plt.grid(True, linestyle='--', alpha=0.7)
plt.title('Sales in the first half of 2025')
plt.xlabel('Month')
plt.ylabel('Sales (PLN)')
plt.ylim([10000, 21000])
plt.fill_between(months, sales, 10000, alpha=0.2, color='skyblue')
plt.axhline(y=15000, color='red', linestyle='--')
plt.text(0, 15300, 'Monthly target', color='red')
plt.tight_layout()
plt.show()
```



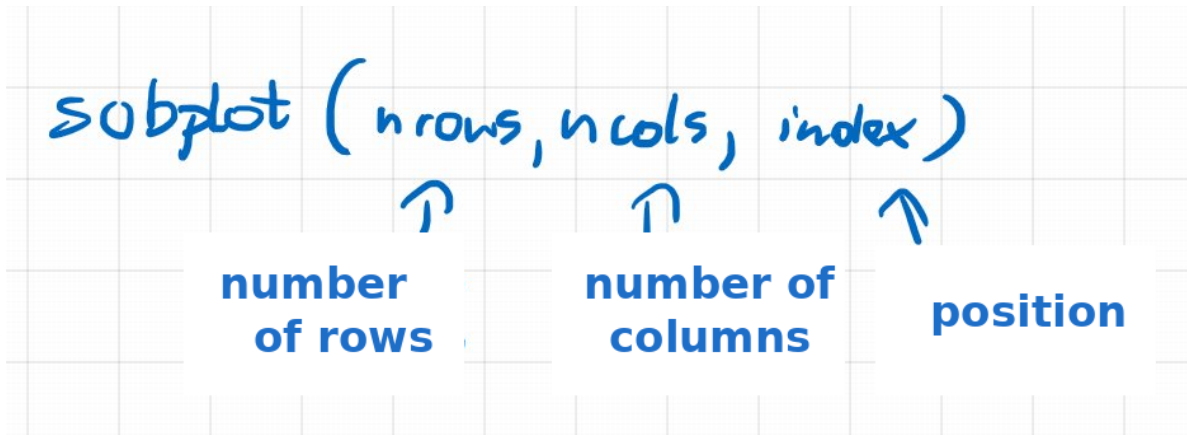
```
import matplotlib.pyplot as plt
import numpy as np

months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun']
sales = [12500, 14000, 16700, 15400, 18200, 19500]

fig, ax = plt.subplots()
ax.plot(months, sales, 'bo-', linewidth=2, markersize=8)
ax.grid(True, linestyle='--', alpha=0.7)
ax.set_title('Sales in the first half of 2025')
ax.set_xlabel('Month')
ax.set_ylabel('Sales (PLN)')
ax.set_ylim([10000, 21000])
ax.fill_between(months, sales, 10000, alpha=0.2, color='skyblue')
ax.axhline(y=15000, color='red', linestyle='--')
ax.text(0, 15300, 'Monthly target', color='red')
plt.tight_layout()
plt.show()
```



29 Matplotlib - subplots



The `subplot` function allows you to create multiple plots within a single window or figure. This makes it possible to compare different plots that share a common context, or to present different aspects of the data.

The function signature is `plt.subplot(nrows, ncols, index, **kwargs)`, where:

- `nrows` - the number of rows in the plot grid.
- `ncols` - the number of columns in the plot grid.
- `index` - the index of the current plot to be created (indexing starts at 1). Indices are numbered row by row, i.e. the next plot in a row will have an index one greater.
- `**kwargs` - additional arguments related to plot formatting.

```
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0, 2 * np.pi, 100)

# Creating a 2x2 grid of plots
# First plot (top-left corner)
plt.subplot(2, 2, 1)
plt.plot(x, np.sin(x))
plt.title('sin(x)')
```

```

# Second plot (top-right corner)
plt.subplot(2, 2, 2)
plt.plot(x, np.cos(x))
plt.title('cos(x)')

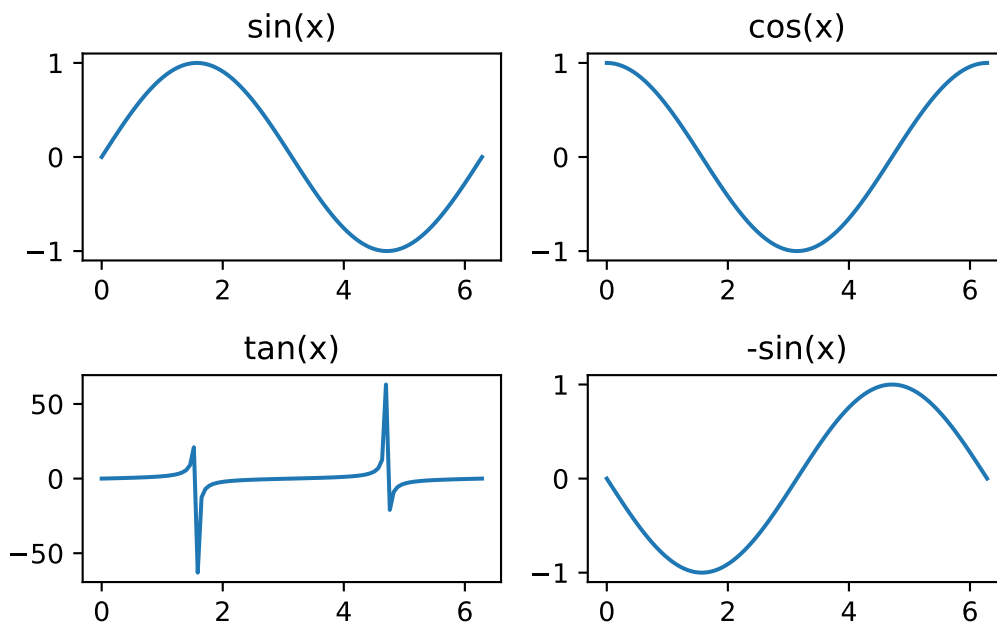
# Third plot (bottom-left corner)
plt.subplot(2, 2, 3)
plt.plot(x, np.tan(x))
plt.title('tan(x)')

# Fourth plot (bottom-right corner)
plt.subplot(2, 2, 4)
plt.plot(x, -np.sin(x))
plt.title('-sin(x)')

# Adjusting the spacing between plots
plt.tight_layout()

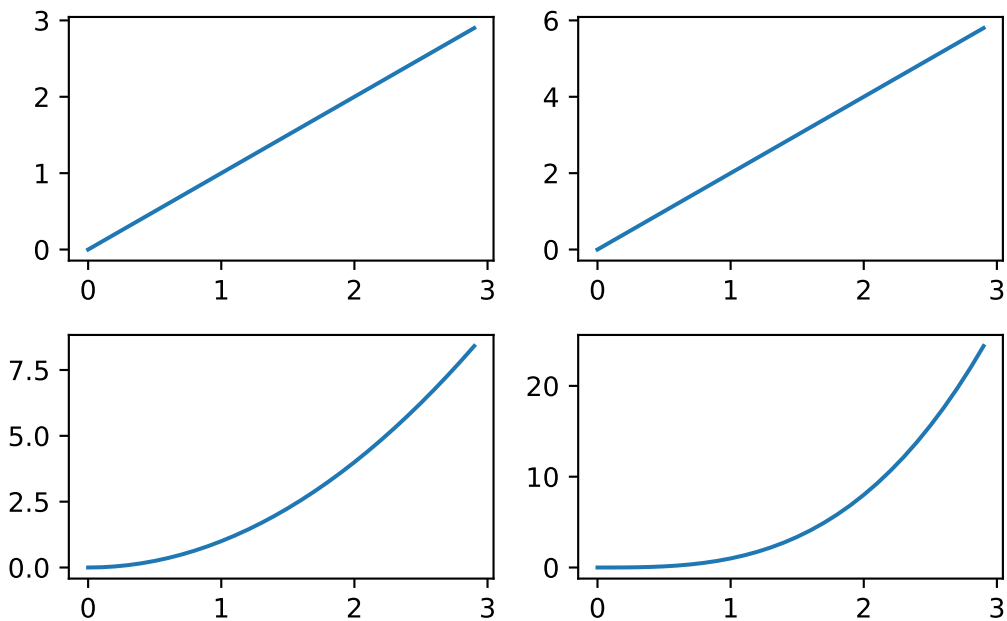
# Displaying the plots
plt.show()

```



```
import numpy as np
import matplotlib.pyplot as plt

x = np.arange(0, 3, 0.1)
plt.subplot(2, 2, 1)
plt.plot(x, x)
plt.subplot(2, 2, 2)
plt.plot(x, x * 2)
plt.subplot(2, 2, 3)
plt.plot(x, x * x)
plt.subplot(2, 2, 4)
plt.plot(x, x ** 3)
plt.tight_layout()
plt.show()
```



```
import numpy as np
import matplotlib.pyplot as plt

x = np.arange(0, 3, 0.1)

fig, axes = plt.subplots(2, 2)

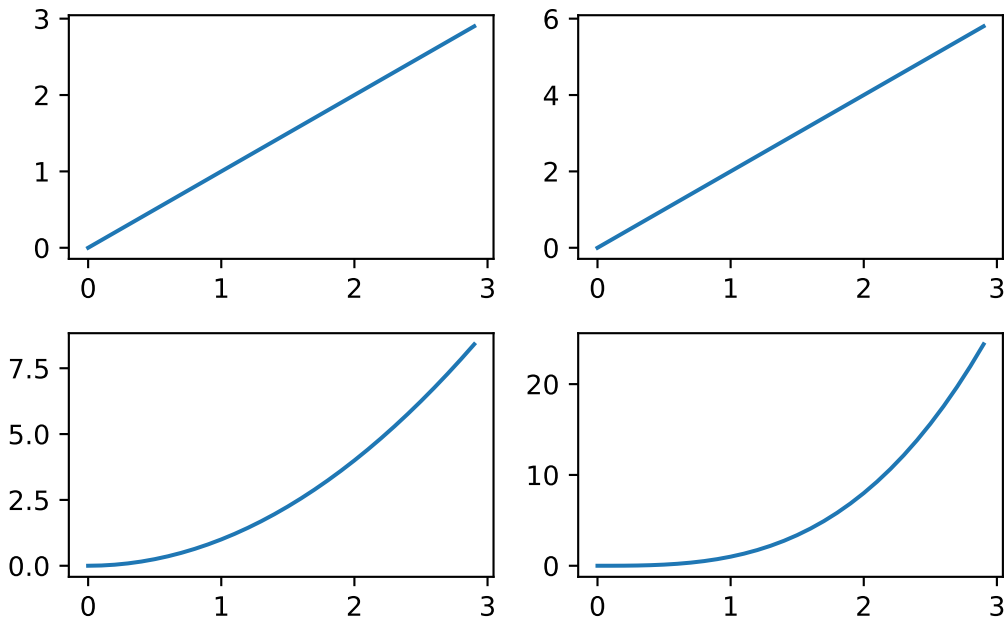
axes[0, 0].plot(x, x)
```

```

axes[0, 1].plot(x, x * 2)
axes[1, 0].plot(x, x * x)
axes[1, 1].plot(x, x ** 3)

fig.tight_layout()
plt.show()

```



```

import numpy as np
import matplotlib.pyplot as plt

# Preparing data for visualization
x = np.linspace(0, 10, 100) # Create 100 points in the range from 0 to 10
y1 = np.sin(x)              # Sine function
y2 = np.cos(x)              # Cosine function
y3 = np.exp(-0.2*x) * np.sin(x) # Damped sine function
y4 = x**2 / 20              # Quadratic function

# Creating the figure and subplots (2 rows, 2 columns)
# In the procedural approach we use pyplot directly
plt.figure(figsize=(12, 8)) # Set the size of the entire figure (width, height in inches)

# First subplot (top-left)
plt.subplot(2, 2, 1) # 2 rows, 2 columns, position 1

```

```

plt.plot(x, y1, 'r-', linewidth=2) # Red line
plt.title('Sine function')
plt.xlabel('X')
plt.ylabel('sin(x)')
plt.grid(True) # Add a grid
plt.axhline(y=0, color='k', linestyle='--', alpha=0.3) # Horizontal line at y=0

# Second subplot (top-right)
plt.subplot(2, 2, 2) # 2 rows, 2 columns, position 2
plt.plot(x, y2, 'b-', linewidth=2) # Blue line
plt.title('Cosine function')
plt.xlabel('X')
plt.ylabel('cos(x)')
plt.grid(True)
plt.axhline(y=0, color='k', linestyle='--', alpha=0.3)

# Third subplot (bottom-left)
plt.subplot(2, 2, 3) # 2 rows, 2 columns, position 3
plt.plot(x, y3, 'g-', linewidth=2) # Green line
plt.title('Damped sine')
plt.xlabel('X')
plt.ylabel('exp(-0.2x) * sin(x)')
plt.grid(True)
plt.axhline(y=0, color='k', linestyle='--', alpha=0.3)

# Fourth subplot (bottom-right)
plt.subplot(2, 2, 4) # 2 rows, 2 columns, position 4
plt.plot(x, y4, 'orange', linewidth=2, linestyle='--', marker='o', markevery=10, markersize=16)
plt.title('Quadratic function')
plt.xlabel('X')
plt.ylabel('x2/20')
plt.grid(True)

# Adding an annotation on the last plot
plt.annotate('Point (5, 1.25)', xy=(5, 1.25), xytext=(6, 1.8),
            arrowprops=dict(facecolor='black', shrink=0.05, width=1.5))

# Adjusting the layout - prevents overlapping
plt.tight_layout()

# Adding an overall title for the whole figure
plt.suptitle('Example of plots with subplots - procedural approach', fontsize=16)

```

```
plt.subplots_adjust(top=0.9) # Add space for the main title

# Displaying the plot
plt.show()
```

Example of plots with subplots - procedural approach

